Formális nyelv = másodrendű algebrai elmélet

Kaposi Ambrus*

akaposi@inf.elte.hu

Kivonat

A formális nyelvek közé tartoznak a logikai és a programozási nyelvek. Ebben a jegyzetben példákon keresztül megmutatjuk, hogyan lehet redundáns információ nélkül, magas szinten ilyen nyelveket megadni, és bennük programozni.

Tartalomjegyzék

1. Elsőrendű algebrai elméletek **1**
1.1. Univerzális algebra . . . 5

2. Elsőrendű általánosított algebrai elméletek **6**
2.1. További GAT-ok . . . 9

3. Másodrendű algebrai elméletek **10**

4. Másodrendű általánosított algebrai elméletek **14**
4.1. Függvény típus . . . 14
4.2. System T, definiálhatóság . . . 18
4.3. Fixpont-kombinátor . . . 21
4.4. Véges típusok . . . 22
4.5. Generikus programozás . . . 24
4.6. Induktív típusok . . . 26
4.7. Koinduktív típusok . . . 28
4.8. Polimorfizmus . . . 30
4.9. Függő típusok . . . 33
4.10. Elsőrendű logika . . . 35

5. Másodrendű általánosított algebra szignatúrák megadása másodrendű általánosított algebrai elmélettel **36**

Ennek a jegyzetnek az eredeti változata az Eötvös Collegium Informatika Műhelyének 20 éves évfordulójára készült. Köszönöm szépen Lócsi Leventének a biztatást, sok hibajavítást és szerkesztést. Köszönöm az alábbi hallgatóknak a hibajavításokat: Schaum Béla.

A jegyzet legfrissebb változata a szerző honlapján elérhető.

A jegyzet metanyelve Martin-Löf típuselmélete egyenlőségi reflexióval (equality reflection). A 3. fejezettől kezdve mindig egy adott nyelv szintaxisa fölötti előkéve modell belső nyelvében dolgozunk, de ennek ismerete nem szükséges a jegyzet megértéséhez (részletekért lásd a 75. definíció környékét). Azon állításokat, melyek bizonyításához ki kell lépni az előkéve-nyelvből, meta-állításnak jelöljük. A nem-meta állításokat mind formalizálni lehet a típuselmélet valamely implementációjában (pl. Agda, Coq, Idris, Lean).

1. Elsőrendű algebrai elméletek

Ebben a fejezetben a félcsoport és a kombinátor-kalkulus példáin keresztül bemutatjuk az algebrai elméleteket. Az algebrai elméletek (algebraic theory, AT) nagyon egyszerű programozási nyelvek: nincsenek bennük változók és típusok.

A legegyszerűbb Turing-teljes programozási nyelv Moses Schönfinkel kombinátor-kalkulusa [Sch24]. Az S és K kombinátorokat tudjuk egymás után írni zárójelezetten (például $(\mathsf{SK})\mathsf{K}$ illetve $\mathsf{S}(\mathsf{KK})$ két különböző kombinátor term), és két átírási szabályunk van: $(\mathsf{K}u)f \rightsquigarrow u$ illetve $((\mathsf{S}f)g)u \rightsquigarrow (fu)(gu)$, ahol u, f és g tetszőleges kombinátor termek.

Hogyan írjuk le ezt a nyelvet teljesen precízen? Először is, mi az, hogy kombinátor-term? Itt van alább néhány lehetséges válasz, a konkréttól az absztrakt felé:

1. Tetszőleges sztring (karaktersorozat).

* ELTE Informatikai Kar, Programozási Nyelvek és Fordítóprogramok Tanszék

2. Sztring, melyben csak S, K, nyitó és csukó zárójelek szerepelhetnek (lexikális elemek sorozata).

3a. A fentiek közül csak azon sztringek, melyekre a következő induktívan megadott P predikátum teljesül: $P(\text{"}\mathsf{S}\text{"})$, $P(\text{"}\mathsf{K}\text{"})$, illetve ha $P(s)$ és $P(s')$, akkor $P\big(\text{"(" } +\!\!+\ s +\!\!+\ s' +\!\!+ \text{ ")"}\big)$.

3b. Bináris fák, melyeknek kétféle levelük lehet (absztrakt szintaxisfa, AST).

4. Kétféle levelű bináris fák ekvivalenciaosztályai, melyek a fenti két átírási szabályból képzett ekvivalencia-kongruencia reláció alapján vannak képezve.

Az 1.→2. lépést lexikális elemzésnek, a 2.→3. lépést szintaktikus elemzésnek (parsing) nevezik.

1. Feladat. *A fenti 3a. és 3b. pontosan ugyanazokat az objektumokat írják le.*

2. Feladat. *Írjunk lexikális és szintaktikus elemzőt a kombinátor-kalkulusra!*

3. Feladat. *A nem-címkézett bináris fák ugyanazokat az objektumokat írják le, mint a csak zárójeleket tartalmazó jól zárójelezett sztringek. Egy sztring jól zárójelezett, ha a következő sztring-feldolgozó algoritmus 0-t ad: végigmegyünk a sztringen, a számlálót eggyel megnöveljük, ha nyitó-, és eggyel csökkentjük, ha csukó zárójelet találunk; a számláló az elején 0, ha bármikor a 0-t szeretnénk currcsökkenteni, hibával leállunk.*

Ebben a jegyzetben a 4. változatot fogjuk használni, a következőképp: a kombinátor-kalkulus egy algebrai elmélet, melyben egy bináris művelet van (applikáció, egymás mellé írás), két nulláris művelet van (S és K), és két egyenlőség $(\mathsf{K}u)f = u$ és $((\mathsf{S}f)g)u = (fu)(gu)$. Konvenció, hogy az applikáció balra zárójeleződik, tehát a két egyenlőség rövidebben így is írható: $\mathsf{K}uf = u$ és $\mathsf{S}fgu = fu(gu)$. Az ilyen, három művelettel ellátott halmazokat, melyekre teljesül a fenti két egyenlőség, kombinátor-algebráknak nevezzük.

Mielőtt a kombinátor-algebrákat tárgyalnánk, megnézünk egy másik, jól ismert algebrai elméletet.

4. Definíció (félcsoport)**.** *Egy félcsoport a következő komponensekből áll:*

$$\begin{aligned}
&\mathsf{C} && : \mathsf{Set}\\
&- \cdot - && : \mathsf{C} \to \mathsf{C} \to \mathsf{C}\\
&\mathsf{ass} && : x \cdot (y \cdot z) = (x \cdot y) \cdot z
\end{aligned}$$

Egy félcsoport tehát egy halmaz, rajta egy bináris művelet (mely „curry-zett" módon, $\mathsf{C} \to (\mathsf{C} \to \mathsf{C})$ függvénnyel van megadva $\mathsf{C} \times \mathsf{C} \to \mathsf{C}$ helyett), mely asszociatív. Például félcsoportot alkotnak a természetes számok az összeadással ($\mathsf{C} := \mathbb{N}, x \cdot y := x + y$), hiszen az összeadás asszociatív. További példák: természetes számok a szorzással; logikai értékek (kételemű halmaz) az és/vagy/xor/xnor művelettel; sztringek az összefűzéssel (konkatenáció); $n \times n$-es valós mátrixok a mátrix-szorzással; $A \to A$ függvények a függvénykompozícióval tetszőleges A halmazra; A részhalmazai a metszettel/unióval; az egyelemű halmaz (mi a művelet?); az üres halmaz (mi a művelet?). Nem példa: természetes számok a kivonással vagy hatványozással.

5. Feladat. *Adjuk meg formálisan a fenti félcsoportokat!*

A kombinátor-algebrákat is megadjuk ugyanezzel a jelöléssel.

6. Definíció (Kombinátor-algebra)**.**

$$\begin{aligned}
&\mathsf{Tm} && : \mathsf{Set}\\
&- \cdot - && : \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm} && \textit{(balra zárójeleződik)}\\
&\mathsf{K} && : \mathsf{Tm}\\
&\mathsf{S} && : \mathsf{Tm}\\
&\mathsf{K}\beta && : \mathsf{K} \cdot u \cdot f = u\\
&\mathsf{S}\beta && : \mathsf{S} \cdot f \cdot g \cdot u = f \cdot u \cdot (g \cdot u)
\end{aligned}$$

A kombinátor-algebrákat szokás a kombinátor-kalkulus modelljeinek, a félcsoportokat a félcsoport-elmélet modelljeinek nevezni. A következőkben kombinátor-kalkulusban fogunk programozni, ez azt jelenti, hogy egy tetszőleges kombinátor-algebrában dolgozunk. Ehhez feltételezünk egy kombinátor-algebrát (egy Tm halmazt egy bináris és két nulláris művelettel, melyre teljesül a két fenti egyenlőség).

7. Feladat (Identitás)**.** *Van a* Tm*-nek egy* I *eleme, melyre* $\mathsf{I} \cdot u = u$ *minden* u*-ra.*

8. Feladat (kompozíció)**.** *Van olyan* B *term, melyre* $\mathsf{B} \cdot f \cdot g \cdot u = f \cdot (g \cdot u)$ *minden* f*,* g*,* u*-ra.*

9. Feladat (C kombinátor)**.** *Van olyan* C *term, melyre* $\mathsf{C} \cdot f \cdot u \cdot g = f \cdot g \cdot u$ *minden* f*,* g*,* u*-ra.*

Kombinátor termekkel reprezentálhatók a természetes számok: az alapötlet, hogy egy n szám egy olyan függvény (term), melyre $n \cdot u \cdot f = f \cdot (f \cdot (\dots (f \cdot u) \dots))$, ahol f-et n-szer alkalmazzuk u-ra. A nulla nem más, mint K, a rákövetkező, összeadás, szorzás stb. definiálható.

10. Feladat (Rákövetkező). *Van olyan* suc *term, melyre* $\mathsf{suc} \cdot a \cdot f \cdot x = f \cdot (a \cdot f \cdot x)$ *minden* a, f, x*-re.*

11. Feladat (Összeadás). *Van olyan* plus *term, melyre* $\mathsf{plus} \cdot a \cdot b \cdot f \cdot x = a \cdot f \cdot (b \cdot f \cdot x)$.

12. Feladat (Extenzionalitás). *Adjuk meg máshogyan az* I *kombinátort, tehát egy olyan másik*[1] I' *termet, melyre* $\mathsf{I} \cdot u = u$ *minden* u*-ra!*

13. Feladat (Fixpont-kombinátor). *Megadjuk a következő termeket:* $\mathsf{M} := \mathsf{S} \cdot \mathsf{I} \cdot \mathsf{I}$, $\mathsf{L} := \mathsf{C} \cdot \mathsf{B} \cdot \mathsf{M}$, $\mathsf{Y} := \mathsf{S} \cdot \mathsf{L} \cdot \mathsf{L}$. *Mutassuk meg, hogy* $\mathsf{M} \cdot f = f \cdot f$, $\mathsf{L} \cdot f \cdot g = f \cdot (g \cdot g)$, $\mathsf{Y} \cdot f = f \cdot (\mathsf{Y} \cdot f)$.

Ennek segítségével tetszőleges rekurzív függvény implementálható.

14. Feladat (Fixpont-kombinátor használata). *Az összeadás megadható a fixpont-kombinátort használva.*

15. Feladat (Kettes projekciók). *Adjuk meg* proj_1, proj_2 *termeket úgy, hogy* $\mathsf{proj_i} \cdot u_1 \cdot u_2 = u_i$ *teljesüljön.*

16. Feladat (Hármas projekciók). *Adjuk meg* proj_1, proj_2, proj_3 *termeket úgy, hogy* $\mathsf{proj_i} \cdot u_1 \cdot u_2 \cdot u_3 = u_i$ *teljesüljön.*

Most egy tetszőleges kombinátor-algebrában való programozásról áttérünk a kombinátor-algebrák összességének vizsgálatára.

A triviális kombinátor-algebrában Tm az egyelemű halmaz.

17. Állítás. *Nincs kételemű kombinátor-algebra.*

Bizonyítás. A 16. feladat segítségével megadjuk proj_i termeket, melyekre $\mathsf{proj_i} \cdot u_1 \cdot u_2 \cdot u_3 = u_i$ teljesül. Tegyük fel, hogy a {0,1} halmaz kombinátor-algebra. Ebben a modellben a skatulya-elv alapján kettő proj_i megegyezik, az általánosság elvesztése nélkül feltesszük, hogy $\mathsf{proj}_1 = \mathsf{proj}_2$. Ekkor $0 = \mathsf{proj}_1 \cdot 0 \cdot 1 \cdot 1 = \mathsf{proj}_2 \cdot 0 \cdot 1 \cdot 1 = 1$. □

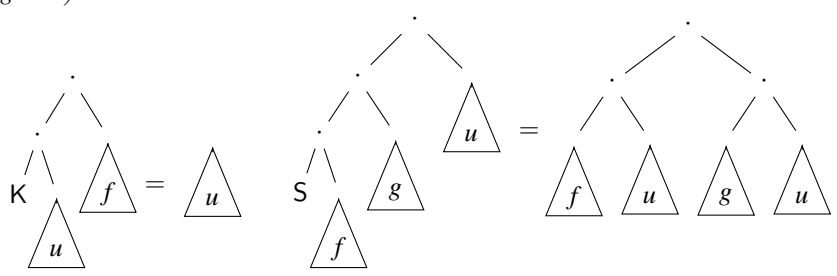
18. Feladat. *Nincs nemtriviális véges kombinátor-algebra (melyben* Tm *véges halmaz).*

Nemtriviális kombinátor-algebrákat nem olyan könnyű találni, mint félcsoportokat. Ez nem meglepő, hiszen ez egy Turing-teljes nyelv – egy nemtriviális kombinátor-algebrában elkódolható az összes Turing-gép.

19. Gondolkodnivaló. *Van-e olyan kombinátor-algebra, melyben* Tm *a Turing-gépek kódjainak halmaza? Esetleg ennek valamely kvóciens halmaza?*

Van egy módszer, amellyel tetszőleges algebrai elméletnek megadható egy modellje (ami néha megegyezik a triviális modellel), ezt szabad algebrának vagy szintaxisnak nevezzük. A kombinátor-kalkulus szintaxisa a következő.

20. Definíció (Kombinátor-kalkulus szintaxisa). *Vesszük a bináris fákat, melyek leveleinél a kételemű halmaz valamely eleme szerepel (ezt a két elemet* K*-val illetve* S*-el fogjuk jelölni), és ezek közül a következő fákat megegyezőnek tekintjük (itt a háromszögekkel tetszőleges fákat jelölünk, melyeket elnevezünk* u*-nak,* f*-nek illetve* g*-nek):*



A szintaxis „szabadsága” azt fejezi ki, hogy ez egy olyan algebra, mely csak annyit tud, amennyit az algebrai elmélet követel, sem többet (például nincs benne több egyenlőség), sem kevesebbet (hiszen az adott elmélet algebrája). A szintaxis elemeit szabadon generálja a három művelet, és az egyenlőségek szerint veszünk egy kvócienst. Ezt úgy is ki lehet fejezni, hogy a szintaxis elemei valójában ekvivalencia-osztályok: két term egy ekvivalenciaosztályban van, ha a két egyenlőség által generált ekvivalencia-kongruencia reláció szerint relációban állnak egymással. Ha két term a szintaxisban egyenlő, arra úgy gondolhatunk, hogy a két term (mint program) ugyanazt a végeredményt adja (vagy ugyanúgy nem adnak végeredményt, ha végtelen sokáig futó programok).

[1] Lásd a 26. feladatot.

Ha valamit meg tudunk csinálni a szintaxisban, azt meg tudjuk csinálni tetszőleges kombinátor-algebrában is: ha egy termet fel tudunk építeni a szintaxisban, azt fel tudjuk építeni tetszőleges algebrában. Ha két term egyenlő a szintaxisban, a megfelelő termek egyenlőek tetszőleges kombinátor-algebrában is. A programozást nevezhetjük a szintaxisban való munkának is.

A szintaxisról a priori nem tudjuk, hogy triviális-e, tehát igaz-e, hogy bármely két szintaktikus term egyenlő. Ahhoz, hogy belássuk, hogy például $\mathsf{S} \neq \mathsf{K}$ a szintaxisban, szükségünk van egy modellre, amiben $\mathsf{S} \neq \mathsf{K}$. Véges nemtriviális modellünk nincs, ezért végtelen modellre van szükség.

21. Probléma. *A kombinátor kalkulusnak van nemtriviális modellje.*

22. Jelölés (Függvény-applikáció)**.** *Ha van egy $f : A \to B$ függvényünk és hozzá egy $a : A$ bemenetünk, a függvény alkalmazását a bemenetre a matematikában legtöbbször $f(a)$ módon jelöljük. Ebben a jegyzetben a funkcionális programozásban szokásos módon ehelyett $f\,a$-t írunk.*

(Eddig csak infix függvényeink voltak, például $- \cdot -$, ezért nem kellett az applikációról külön beszélni. Ne tévessze meg az olvasót, hogy most már két különböző applikációról is beszélünk: a metanyelvi (metaelméleti) applikációt egyszerűen egymás mellé írással jelöljük, a tárgynyelvi (kombinátor-kalkulusbeli) applikációt $- \cdot -$-tal.)

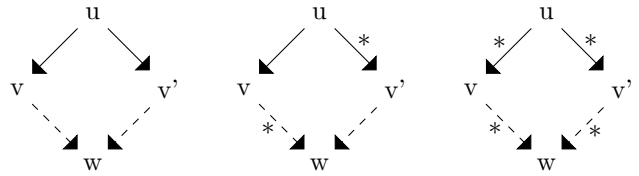
Konstrukció. Az AST-n (S-el ill. K-val levél-címkézett bináris fákon) megadjuk az alábbi $- \rightsquigarrow -$ induktív relációt (egylépéses párhuzamos átírási reláció):

$$\frac{}{\mathsf{K} \rightsquigarrow \mathsf{K}} \qquad \frac{}{\mathsf{S} \rightsquigarrow \mathsf{S}} \qquad \frac{f \rightsquigarrow f' \qquad u \rightsquigarrow u'}{f \cdot u \rightsquigarrow f' \cdot u'} \qquad \frac{u \rightsquigarrow u'}{\mathsf{K} \cdot u \cdot f \rightsquigarrow u'} \qquad \frac{f \rightsquigarrow f' \qquad g \rightsquigarrow g' \qquad u \rightsquigarrow u'}{\mathsf{S} \cdot f \cdot g \cdot u \rightsquigarrow f' \cdot u' \cdot (g' \cdot u')}$$

Ennek reflexív tranzitív lezártját $- \rightsquigarrow^* -$-vel jelöljük. Megadunk az AST-termeken egy endofüggvényt az alábbi módon:

$$\begin{aligned}
&\mathsf{step}\,\mathsf{K} && := \mathsf{K}\\
&\mathsf{step}\,\mathsf{S} && := \mathsf{S}\\
&\mathsf{step}\,(\mathsf{K} \cdot u \cdot f) && := \mathsf{step}\,u\\
&\mathsf{step}\,(\mathsf{S} \cdot f \cdot g \cdot u) && := \mathsf{step}\,f \cdot \mathsf{step}\,u \cdot (\mathsf{step}\,g \cdot \mathsf{step}\,u)\\
&\mathsf{step}\,(f \cdot g) && := \mathsf{step}\,f \cdot \mathsf{step}\,g
\end{aligned}$$

Az átírási reláció szerinti indukcióval belátjuk, hogy $u \rightsquigarrow v$-ből következik $u \rightsquigarrow \mathsf{step}\,v$. Belátjuk, hogy $\rightsquigarrow$ konfluens (tehát $u \rightsquigarrow v$ és $u \rightsquigarrow v'$-ből következik, hogy van olyan w, melyre $v \rightsquigarrow w$ és $v' \rightsquigarrow w$). Belátjuk, hogy bal oldalt $\rightsquigarrow$, jobb oldalt $\rightsquigarrow^*$ is konfluens ($u \rightsquigarrow v$-ből és $u \rightsquigarrow^* v'$-ből következik, hogy van olyan w, melyre $v \rightsquigarrow^* w$ és $v' \rightsquigarrow w$), majd ennek segítségével megmutatjuk, hogy $\rightsquigarrow^*$ is konfluens. Tehát az alábbi sorrendben bizonyítjuk a háromféle konfluenciát.



Megadjuk a következő relációt az AST-termeken: $u \sim v :=$ létezik olyan w, melyre $u \rightsquigarrow^* w$ és $v \rightsquigarrow^* w$. $\sim$ triviálisan reflexív és szimmetrikus, és $\rightsquigarrow^*$ konfluenciája miatt tranzitív. A modellünkben a termek az AST-termek kvóciensezve $\sim$-mal. □

23. Feladat. $\mathsf{K} \neq \mathsf{S}$. *Tipp: mutassuk meg, hogy ha $\mathsf{K} \rightsquigarrow^* u$, akkor $u = \mathsf{K}$, és hasonlóan $\mathsf{S} - re$.*

24. Gondolkodnivaló. $\mathsf{I}' \neq \mathsf{I}$.

25. Gondolkodnivaló. *Ha u kombinátor-normálforma és $u \rightsquigarrow^* v$, akkor $u = v$.*

26. Gondolkodnivaló. *Vannak olyan kombinátor termek, melyeknek nincs normálformája (nem egyenlők egy normálformával).*

27. Gondolkodnivaló (Egyenlőségének eldönthetetlensége)**.** *Feltételezve, hogy a metaelméletünk konstruktív (pl. Martin-Löf típuselmélet), a kombinátor-kalkulus szintaxisában az egyenlőség eldönthetetlen. Tehát nem igaz, hogy bármely t_0, t_1 termekre $t_0 = t_1$ vagy $t_0 \neq t_1$. Ezt úgy tudjuk belátni, hogy megadjuk a metaelméletünk egy modelljét, amiben eldönthető (ez könnyű, tetszőleges klasszikus modell megfelel), és megadjuk egy modelljét, melyben ez az állítás hamis (ez már érdekesebb).*

28. Feladat. *Tetszőleges két kombinátor-algebrának képezhető a szorzat algebrája, ahol a termek halmaza a két halmaz Descartes-szorzata (meg kell mutatnunk, hogy a termek Descartes-szorzatán megadható egy kombinátor-algebra).*

29. Feladat. *Mutassuk meg, hogy van olyan kombinátor szintaktikus termek közötti egyenlőség, amely teljesül bizonyos kombinátor-algebrában, de nem a szintaxisban.*

30. Feladat. *Mutassuk meg, hogy ha egy egyenlőség teljesül az összes kombinátor-algebrában, akkor a szintaxisban is!*

31. Gondolkodnivaló (Üzemanyag szemantika)**.**
A kombinátor-normálformák ilyen alakú termek: K, $\mathsf{K}\cdot n$, S, $\mathsf{S}\cdot n$, $\mathsf{S}\cdot n\cdot n'$, *ahol* n *és* n' *normálformák. Meg szeretnénk adni egy függvényt, ami szintaktikus kombinátor termekről velük egyenlő normálformákra képez. Ez a függvény parciális lesz, hiszen például a fixpont-kombinátor nem egyenlő semmilyen normálformával. (Ezt hogyan bizonyítjuk?) Másrészt az nem eldönthető, hogy egy termnek van-e normálformája (Rice tétele). Emiatt egy olyan szemantikát tudunk megadni, ahol üzemanyag (egy természetes szám) mennyiségű lépést hajthatunk létre, és azt adjuk meg, hogy ennyi lépésben megkapjuk-e a normálformát. Minden, a szintaxison megadott függvénynek meg kell tartania az egyenlőségeket, ezért a végeredményt faktorizálni kell a teljes relációval.*

32. Gondolkodnivaló. *Mutassuk meg, hogy a fixpont-kombinátornak (13. feladat) nincs normálformája!*

1.1. Univerzális algebra

Néhány további, jól ismert algebrai elmélet: egységelemes félcsoport (angolul monoid), csoport, gyűrű, testszőleges test feletti vektortér. Nagyon sok algebrai elmélet van: annyi, ahányféle művelet és egyenlőség adható meg. Egy algebrai elméletet a *szignatúrája*[2] határoz meg: szignatúra az operátorok aritását és az egyenlőségek oldalait tartalmazza. Egy egyszerű definíció ezekre a következő.

33. Definíció (Algebra szignatúra)**.** *A szignatúra egyrészt egy* $\nu : \mathbb{N} \to \mathbb{N}$ *függvényből áll, ahol* $\nu\, n$ *megadja, hogy hány darab* n*-paraméteres műveletünk van. Másrészt tekintsük az alábbi szintaxist, amelynek segítségével az egyenlőségek két oldalát fogjuk elkódolni.*

$$\begin{aligned}
&\mathsf{tm} &&: \mathsf{Set}\\
&\mathsf{var} &&: \mathbb{N} \to \mathsf{tm}\\
&\mathsf{op} &&: (n : \mathbb{N}) \to \overline{\nu\, n} \to \underbrace{\mathsf{tm} \to \mathsf{tm} \to \ldots \to}_{n\ darab} \mathsf{tm}
\end{aligned}$$

Itt az $\bar{i}$ *az* i*-elemű halmazt jelöli. Ez egy olyan algebrai elmélet, melyben nincsenek egyenlőségek. Végtelen sok nulláris operátor van (minden természetes számhoz egy), ezek felelnek meg a változóknak, melyek az egyenlőségekben szerepelnek.* $\nu\, n$ *darab* n*-áris műveletünk van. A* ν *függvényen túl a szignatúra* tm*-párok egy halmazát tartalmazza.*

Például félcsoport szignatúrájában $\nu\, 2 = 1$ és minden $n \neq 2$-re $\nu\, n = 0$; a kombinátor-kalkulus szignatúrájában $\nu\, 0 = 2$, $\nu\, 2 = 1$, és minden egyéb n-re $\nu\, n = 0$. A félcsoport szignatúrájában egy egyenlőség van, melyet a következő pár határoz meg (0_1-el jelöljük az egyelemű halmaz egyetlen elemét):

$$\Big(\mathsf{op}\, 2\, 0_1\, \big(\mathsf{op}\, 2\, 0_1\, (\mathsf{var}\, 0)\, (\mathsf{var}\, 1)\big)\, (\mathsf{var}\, 2),\\
\mathsf{op}\, 2\, 0_1\, (\mathsf{var}\, 0)\, \big(\mathsf{op}\, 2\, 0_1\, (\mathsf{var}\, 1)\, (\mathsf{var}\, 2)\big)\Big)$$

A kombinátor-kalkulus szignatúrájában két egyenlőség van, a $\mathsf{K}\beta$ egyenlőséget az alábbi pár kódolja (a kételemű halmaz elemeit 0_2-vel és 1_2-vel jelöljük):

$$\Big(\mathsf{op}\, 2\, 0_1\, \big(\mathsf{op}\, 2\, 0_1\, (\mathsf{op}\, 0\, 0_2)\, (\mathsf{var}\, 0)\big)\, (\mathsf{var}\, 1), \mathsf{var}\, 0\Big)$$

Az $\mathsf{S}\beta$ egyenlőséget az alábbi pár kódolja:

$$\Big(\mathsf{op}\, 2\, 0_1\, \big(\mathsf{op}\, 2\, 0_1\, \big(\mathsf{op}\, 2\, 0_1\, (\mathsf{op}\, 0\, 1_2)\, (\mathsf{var}\, 0)\big)\, (\mathsf{var}\, 1)\big)\, (\mathsf{var}\, 2),\\
\mathsf{op}\, 2\, 0_1\, \big(\mathsf{op}\, 2\, 0_1\, (\mathsf{var}\, 0)\, (\mathsf{var}\, 2)\big)\, \big(\mathsf{op}\, 2\, 0_1\, (\mathsf{var}\, 1)\, (\mathsf{var}\, 2)\big)\Big)$$

[2] A szignatúrába beleértjük az egyenlőségeket is, mert az általánosított algebrai elméleteknél (2. fejezet) a műveletek és az egyenlőségek nem választhatók el: lesznek olyan műveleteink, melyek aritása csak bizonyos egyenlőségek megléte miatt értelmes.

34. Feladat. *Kódoljuk el a csoport szignatúráját a fenti módon!*

35. Feladat. *Adjunk meg olyan algebrai elméletet, amelynek csak triviális algebrája van!*

36. Feladat. *Adjuk meg az algebra-fogalmat tetszőleges szignatúrára!*

37. Feladat. *Mutassuk meg, hogy minden algebrai elméletnek van triviális algebrája!*

38. Feladat. *Mutassuk meg, hogy minden algebrai elméletben tetszőleges két algebrának van szorzat-algebrája!*

39. Megjegyzés. *Az 5. fejezetben megadjuk egy szebb leírását az algebrai elméleteknek.*

Az algebra homomorfizmus egy függvény a két algebra alaphalmaza között, mely megtartja a műveleteket. Például A és B félcsoportok között ez egy $f : \mathsf{C}_A \to \mathsf{C}_B$ függvény, melyre teljesül, hogy $f\,(x \cdot_A y) = f\,x \cdot_B f\,y$ minden x, y-ra. Hasonlóképp, A és B kombinátor-algebrákra ez egy $f : \mathsf{Tm}_A \to \mathsf{Tm}_B$ függvény, melyre $f\,(x \cdot_A y) = f\,x \cdot_B f\,y$, és $f\,\mathsf{K}_A = \mathsf{K}_B$ és $f\,\mathsf{S}_A = \mathsf{S}_B$.

40. Feladat. *Adjunk meg félcsoport-homomorfizmust* $(\mathbb{N}, +)$*-ból* $(\mathsf{Bool}, \mathit{xnor})$*-ba, két különböző homomorfizmust* $(\mathbb{N}, +)$*-ból* $(\mathsf{Bool}, \mathit{és})$*-be, és végtelen sok homomorfizmust* $(\mathbb{N}, +)$*-ból* $(\mathbb{N}, +)$*-ba.*

41. Feladat. *Mutassuk meg, hogy minden félcsoportból megy önmagába egy identitás homomorfizmus, a homomorfizmusok komponálhatók, és minden félcsoportból pontosan egy homomorfizmus megy az egyelemű félcsoportba.*

42. Feladat. *Minden algebrai elmélethez megadható a homomorfizmus fogalom. A homomorfizmusok komponálhatók, és az identitás függvény mindig homomorfizmus.*

43. Definíció (Szintaxis)**.** *A szintaxis az iniciális algebra: ez azt jelenti, hogy minden más algebrába pontosan egy homomorfizmus megy a szintaxisból.*

44. Feladat. *Mutassuk meg, hogy a szintaxis egyértelmű: bármely két iniciális algebra között van egy izomorfizmus (oda-vissza homomorfizmusok, melyek kompozíciói identitások).*

45. Feladat. *Mutassuk meg, hogy a félcsoportok szintaxisa az üres halmaz!*

46. Feladat. *Mutassuk meg, hogy a 20. definícióban megadott algebra valóban a kombinátor-kalkulus szintaxisa!*

A szintaxisból egy adott algebrába menő homomorfizmust kiértékelésnek (interpretációnak) nevezzük.

47. Feladat. *Minden algebrai elméletnek van szintaxisa.*

48. Feladat. *Adjunk meg olyan algebrai elméletet, amelynek a szintaxisa triviális! Egészítsük ki a kombinátor-kalkulust egy egyenlőséggel, amely a szintaxisát triviálissá teszi! Adjunk meg olyan algebrai elméletet, amelynek a szintaxisa triviális, de van nemtriviális modellje! Adjunk meg olyan algebrai elméletet, melynek csak triviális modellje van!*

Az induktívan megadott halmazok egyenlőségek nélküli algebrai elméletek szintaxisai: például a Peano természetes számokat az alábbi algebrai elmélet adja meg: $\mathsf{N} : \mathsf{Set}, \mathsf{zero} : \mathsf{N}, \mathsf{suc} : \mathsf{N} \to \mathsf{N}$.

Egy adott algebrai elmélet többféleképpen is prezentálható. Például a kombinátor-kalkulushoz hozzávehetjük az I műveletet (lásd a 7. feladatot), és ezzel egy vele izomorf algebrai elméletet kapunk, melynek modelljei egy az egyben megfeleltethetők a kombinátor-algebráknak. Vagy a félcsoportokhoz hozzávehetünk egy ternáris műveletet és egy egyenlőséget, mely kimondja, hogy a ternáris művelet egyenlő a bináris művelet kétszeri alkalmazásával. Általánosságban nehéz kérdés, hogy van-e egyszerűbb prezentációja egy adott algebrai elméletnek, és néha nem világos, hogy melyik a „jobb" prezentáció.

A testek nem adhatók meg közvetlenül a szokásos módon algebrai elméletként, mert az osztás csak a nemnulla elemekre működik, de a fenti definíció szerint minden műveletnek működnie kell a teljes alaphalmazra. Ez még nem jelenti azt, hogy nincs valamilyen okosabb, izomorf megadása a testeknek. Ilyen azonban nincs.

49. Gondolkodnivaló. *A testek nem adhatók meg algebrai elméletként, mert tetszőleges algebrai elméletre vannak szorzat-algebrák, és van 2-elemű test és van 3-elemű test, de nincsen 6-elemű test.*

2. Elsőrendű általánosított algebrai elméletek

Ebben a fejezetben megadunk egy egyszerű típusos kifejezésnyelvet és tanulmányozzuk a típusos kombinátor-kalkulust. Ezek már nem egyszerű algebrai elméletek, hanem általánosított algebrai elméletek (generalised algebraic theory, GAT).

Egy általánosított algebrai elméletben nem csak egy alaphalmaz, hanem az alaphalmaz feletti indexelt család is van. A kifejezésnyelvünkben az alaphalmaz a típusok halmaza, és minden típushoz tartozik egy külön halmaz, mely az olyan típusú termek halmaza. A típusok dokumentálják, hogy mit csinál egy operátor, de korlátozzák is, hogy milyen jellegű bemeneteket kaphat.

50. Definíció (Razor)**.**

$$
\begin{array}{ll}
\mathsf{Ty} & : \mathsf{Set} \\
\mathsf{Tm} & : \mathsf{Ty} \to \mathsf{Set} \\
\mathsf{Bool} & : \mathsf{Ty} \\
\mathsf{Nat} & : \mathsf{Ty} \\
\mathsf{true} & : \mathsf{Tm}\,\mathsf{Bool} \\
\mathsf{false} & : \mathsf{Tm}\,\mathsf{Bool} \\
\mathsf{ite} & : \mathsf{Tm}\,\mathsf{Bool} \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A \\
\mathsf{num} & : \mathbb{N} \to \mathsf{Tm}\,\mathsf{Nat} \\
- + - & : \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Nat} \\
\mathsf{isZero} & : \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Bool} \\
\mathsf{ite}\beta_1 & : \mathsf{ite}\,\mathsf{true}\,u\,v = u \\
\mathsf{ite}\beta_2 & : \mathsf{ite}\,\mathsf{false}\,u\,v = v \\
+\beta & : \mathsf{num}\,m + \mathsf{num}\,n = \mathsf{num}\,(m+n) \\
\mathsf{isZero}\beta_1 & : \mathsf{isZero}\,(\mathsf{num}\,0) = \mathsf{true} \\
\mathsf{isZero}\beta_2 & : \mathsf{isZero}\,(\mathsf{num}\,(1+n)) = \mathsf{false}
\end{array}
$$

Például az isZero művelet csak Nat típusú termet kaphat bemenetként, és a kimenete Bool típusú lesz. Az $\mathsf{isZero}\,\mathsf{true}$-nak nincs értelme, ezt nem lehet kimondani a Razor nyelven. Az ite műveletnek az A egy implicit paramétere, ha ezt explicit módon kiírnánk, a teljes típus így nézne ki:

$$\mathsf{ite} : (A : \mathsf{Ty}) \to \mathsf{Tm}\,\mathsf{Bool} \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A.$$

A nyelvet Huttont [Hut23] követve nevezzük Razor-nak. A nyelv egyenlőségei azt mondják meg, hogy hogyan kell a programokat futtatni, például:

$$
\begin{array}{ll}
(\mathsf{num}\,1 + \mathsf{num}\,2) + (\mathsf{num}\,3 + \mathsf{num}\,4) & =(+\beta) \\
(\mathsf{num}\,(1+2)) + (\mathsf{num}\,3 + \mathsf{num}\,4) & = \\
\mathsf{num}\,3 + (\mathsf{num}\,3 + \mathsf{num}\,4) & =(+\beta) \\
\mathsf{num}\,3 + \mathsf{num}\,(3+4) & = \\
\mathsf{num}\,3 + \mathsf{num}\,7 & =(+\beta) \\
\mathsf{num}\,(3+7) & = \\
\mathsf{num}\,10 &
\end{array}
$$

Vegyük észre, hogy a $\mathsf{num}\,(1+2)$ kifejezésben a $+$ a metanyelvi összeadásra vonatkozik, míg a $\mathsf{num}\,1 + \mathsf{num}\,2$ kifejezésben a $+$ a tárgynyelvi összeadás operátorunk (tehát a $+$ szimbólum túl van terhelve). A $\mathsf{num}\,1 + \mathsf{num}\,2 = \mathsf{num}\,(1+2)$ lépéshez a tárgynyelvünk $+\beta$ szabállyát használtuk, míg a $\mathsf{num}\,(1+2) = \mathsf{num}\,3$ lépéshez a metanyelvünkből azt az információt, hogy $1+2=3$.

51. Feladat. *Ha* $\mathsf{true} = \mathsf{false}$*, akkor bármely* A*-ra és* $u, v : \mathsf{Tm}\,A$*-ra* $u = v$*.*

52. Feladat. *Ha* $\mathsf{num}\,0 = \mathsf{num}\,1$*, akkor bármely* A*-ra és* $u, v : \mathsf{Tm}\,A$*-ra* $u = v$*.*

53. Feladat. *Van olyan Razor modell, melyben* $\mathsf{num}\,1 = \mathsf{num}\,2$*, de* $\mathsf{true} \neq \mathsf{false}$*.*

54. Feladat. *Van olyan Razor modell, melyben* $\mathsf{Tm}\,\mathsf{Bool}$ *háromelemű (nemsztenderd modell).*

A Razor AST-szintű megadása nem tartalmazza az egyenlőségeket és nincsenek benne típusok.

55. Definíció (Razor-AST)**.**

$$
\begin{array}{ll}
\mathsf{Tm} & : \mathsf{Set} \\
\mathsf{true} & : \mathsf{Tm} \\
\mathsf{false} & : \mathsf{Tm} \\
\mathsf{ite} & : \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm} \\
\mathsf{num} & : \mathbb{N} \to \mathsf{Tm} \\
- + - & : \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm} \\
\mathsf{isZero} & : \mathsf{Tm} \to \mathsf{Tm}
\end{array}
$$

A Razor-AST szintaxisán indukcióval megadunk egy típuskikövetkeztető függvényt, amely egy Razor-AST termből megad egy Razor típust és egy olyan típusú Razor-termet, vagy hibát ad. A Razor-AST és Razor operátorokat ugyanúgy nevezzük, előbbiek csak a bal, utóbbiak csak a jobb oldalon vannak. $\mathsf{Tm}_{\mathrm{AST}}$ a Razor-AST szintaxisát jelöli, Ty és Tm Razor szintaktikus típusokat és termeket.

$$
\begin{aligned}
&\mathsf{infer} : \mathsf{Tm}_{\mathrm{AST}} \to \mathsf{Maybe}\,((A : \mathsf{Ty}) \times \mathsf{Tm}\,A)\\
&\mathsf{infer}\,\mathsf{true} &&:= \mathsf{just}\,(\mathsf{Bool}, \mathsf{true})\\
&\mathsf{infer}\,\mathsf{false} &&:= \mathsf{just}\,(\mathsf{Bool}, \mathsf{false})\\
&\mathsf{infer}\,(\mathsf{ite}\,b\,u\,v) &&:= \mathsf{match}\,(\mathsf{infer}\,b, \mathsf{infer}\,u, \mathsf{infer}\,v)\,\mathsf{with}\\
&&&\quad (\mathsf{just}(\mathsf{Bool}, b'), \mathsf{just}\,(\mathsf{Bool}, u'), \mathsf{just}\,(\mathsf{Bool}, v')).\mathsf{just}\,(\mathsf{Bool}, \mathsf{ite}\,b'\,u'\,v')\\
&&&\quad (\mathsf{just}(\mathsf{Bool}, b'), \mathsf{just}\,(\mathsf{Nat}, u'), \mathsf{just}\,(\mathsf{Nat}, v')).\mathsf{just}\,(\mathsf{Nat}, \mathsf{ite}\,b'\,u'\,v')\\
&&&\quad \mathrm{otherwise}.\mathsf{nothing}\\
&\mathsf{infer}\,(\mathsf{num}\,n) &&:= \mathsf{just}\,(\mathsf{Nat}, \mathsf{num}\,n)\\
&\mathsf{infer}\,(u + v) &&:= \mathsf{match}\,(\mathsf{infer}\,u, \mathsf{infer}\,v)\,\mathsf{with}\\
&&&\quad (\mathsf{just}(\mathsf{Nat}, u'), \mathsf{just}\,(\mathsf{Nat}, v')).\mathsf{just}\,(\mathsf{Nat}, u' + v')\\
&&&\quad \mathrm{otherwise}.\mathsf{nothing}\\
&\mathsf{infer}\,(\mathsf{isZero}\,t) &&:= \mathsf{match}\,(\mathsf{infer}\,t)\,\mathsf{with}\\
&&&\quad (\mathsf{just}(\mathsf{Nat}, t')).\mathsf{just}\,(\mathsf{Bool}, \mathsf{isZero}\,t')\\
&&&\quad \mathrm{otherwise}.\mathsf{nothing}
\end{aligned}
$$

56. Feladat. *Számítsuk ki, hogy az alábbi Razor-AST termekre milyen végeredményt ad a típuskikövetkeztető!*

$$
\begin{aligned}
&\mathsf{ite}\,(\mathsf{isZero}\,(\mathsf{num}\,1))\,(\mathsf{num}\,2)\,(\mathsf{num}\,2 + \mathsf{num}\,3)\\
&\mathsf{ite}\,(\mathsf{isZero}\,\mathsf{true})\,(\mathsf{num}\,2)\,(\mathsf{num}\,2 + \mathsf{num}\,3)\\
&\mathsf{ite}\,\mathsf{true}\,(\mathsf{num}\,2)\,\mathsf{false}\\
&\mathsf{ite}\,\mathsf{true}\,(\mathsf{num}\,2)\,(\mathsf{num}\,2 + \mathsf{false})\\
&\mathsf{isZero}\,(\mathsf{ite}\,\mathsf{true}\,\mathsf{true}\,\mathsf{true})\\
&\mathsf{isZero}\,(\mathsf{ite}\,\mathsf{true}\,(\mathsf{num}\,1)\,\mathsf{true})\\
&\mathsf{isZero}\,(\mathsf{ite}\,\mathsf{true}\,(\mathsf{num}\,1)\,(\mathsf{num}\,2))
\end{aligned}
$$

A jóltípusozott Razor nyelv ugyanaz, mint a Razor, csak az egyenlőségeket kivesszük belőle.

57. Definíció (Razor-WT)**.**

$$
\begin{aligned}
&\mathsf{Ty} &&: \mathsf{Set}\\
&\mathsf{Tm} &&: \mathsf{Ty} \to \mathsf{Set}\\
&\mathsf{Bool} &&: \mathsf{Ty}\\
&\mathsf{Nat} &&: \mathsf{Ty}\\
&\mathsf{true} &&: \mathsf{Tm}\,\mathsf{Bool}\\
&\mathsf{false} &&: \mathsf{Tm}\,\mathsf{Bool}\\
&\mathsf{ite} &&: \mathsf{Tm}\,\mathsf{Bool} \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A\\
&\mathsf{num} &&: \mathbb{N} \to \mathsf{Tm}\,\mathsf{Nat}\\
&_ + _ &&: \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Nat}\\
&\mathsf{isZero} &&: \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Bool}
\end{aligned}
$$

A jóltípusozott szintaxisból ki tudjuk törölni a típusokat. $\mathsf{Tm}_{\mathrm{AST}}$ a Razor-AST szintaxisát jelöli, Ty és Tm Razor-WT szintaktikus típusokat és termeket.

$$
\begin{aligned}
&\mathsf{erase} : \mathsf{Tm}\,A \to \mathsf{Tm}_{\mathrm{AST}}\\
&\mathsf{erase}\,\mathsf{true} &&:= \mathsf{true}\\
&\mathsf{erase}\,\mathsf{false} &&:= \mathsf{false}\\
&\mathsf{erase}\,(\mathsf{ite}\,b\,u\,v) &&:= \mathsf{ite}\,(\mathsf{erase}\,b)\,(\mathsf{erase}\,u)\,(\mathsf{erase}\,v)\\
&\mathsf{erase}\,(\mathsf{num}\,n) &&:= \mathsf{num}\,n\\
&\mathsf{erase}\,(u + v) &&:= \mathsf{erase}\,u + \mathsf{erase}\,v\\
&\mathsf{erase}\,(\mathsf{isZero}\,t) &&:= \mathsf{isZero}\,(\mathsf{erase}\,t)
\end{aligned}
$$

A típuskikövetkeztetés helyessége a következőt jelenti: minden $u : \mathsf{Tm}\,A$-ra $\mathsf{infer}\,(\mathsf{erase}\,u) = \mathsf{just}\,(A, u)$.

58. Feladat. *A Razor szintaxisán adjunk meg egy kiértékelő függvényt, amely* $\mathsf{Tm}\,\mathsf{Bool}$*-ról* $\mathbb{2}$*-re,* $\mathsf{Tm}\,\mathsf{Nat}$*-ról* $\mathbb{N}$*-re képez.* $\mathbb{2} = \{\mathsf{tt}, \mathsf{ff}\}$.

59. Feladat. *Az előző feladat segítségével adjunk meg egy normalizáló függvényt a Razor szintaxisán:*

$$\begin{aligned}
&\mathsf{Nf} : \mathsf{Ty} \to \mathsf{Set}\\
&\mathsf{Nf}\,\mathsf{Bool} := \mathbb{2}\\
&\mathsf{Nf}\,\mathsf{Nat} := \mathbb{N}\\
&\mathsf{norm} : \mathsf{Tm}\,A \to \mathsf{Nf}\,A
\end{aligned}$$

Bizonyítsuk be a normalizálás teljességét, tehát azt, hogy minden t*-re* $\mathsf{quote}\,(\mathsf{norm}\,t) = t$*, ahol* quote *az alábbi „idézés" függvény, mely normál formákat szintaktikus termekre képez.*

$$\begin{aligned}
&\mathsf{quote} : \mathsf{Nf}\,A \to \mathsf{Tm}\,A\\
&\mathsf{quote}\,\mathsf{tt} := \mathsf{true}\\
&\mathsf{quote}\,\mathsf{ff} := \mathsf{false}\\
&\mathsf{quote}\,n := \mathsf{num}\,n
\end{aligned}$$

60. Feladat. *A Razor szintaxisán nem működik a fent megadott* erase *függvény, mert nem tartja meg az egyenlőségeket. Helyette adjunk meg a normalizálás segítségével egy* $\mathsf{Tm}\,A \to \mathsf{Tm}_{AST}$ *függvényt, mely mindig normál formájú AST-termet ad vissza.*

2.1. További GAT-ok

A típusokat dokumentációnak is gondolhatjuk: a kombinátor-kalkulus típusos leírása érthetőbb, mint a típus nélküli, hiszen a típusok elmondják, hogy valójában mit csinálnak a K és az S operátorok.

61. Definíció (Típusos kombinátor-algebra)**.**

$$\begin{aligned}
&\mathsf{Ty} && : \mathsf{Set}\\
&\mathsf{Tm} && : \mathsf{Ty} \to \mathsf{Set}\\
&\iota && : \mathsf{Ty}\\
&- \Rightarrow - && : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty}\\
&- \cdot - && : \mathsf{Tm}\,(A \Rightarrow B) \to \mathsf{Tm}\,A \to \mathsf{Tm}\,B && \textit{(balra zárójeleződik)}\\
&\mathsf{K} && : \mathsf{Tm}\,(A \Rightarrow B \Rightarrow A)\\
&\mathsf{S} && : \mathsf{Tm}\,((A \Rightarrow B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C)\\
&\mathsf{K}\beta && : \mathsf{K} \cdot u \cdot f = u\\
&\mathsf{S}\beta && : \mathsf{S} \cdot f \cdot g \cdot u = f \cdot u \cdot (g \cdot u)
\end{aligned}$$

A $- \cdot -$ applikáció műveletnek van két implicit paramétere, ezek az A és a B. Az applikáció explicit leírása a következő lenne:

$$(A : \mathsf{Ty}) \to (B : \mathsf{Ty}) \to \mathsf{Tm}\,(A \Rightarrow B) \to \mathsf{Tm}\,A \to \mathsf{Tm}\,B$$

Nem akarjuk mindig kiírni az A-t és a B-t, ezért ezeket implicit módon mindig odaértjük. Az értékük általában kikövetkeztethető a másik két paraméterből. Hasonlóan, a K *műveletnek két, az* S *műveletnek három implicit* Ty*-paramétere van. Az egyenlőségeknél is odaértjük az implicit* Ty/Tm*-paramétereket.*

Az ilyen, általánosított algebrai elméletekre is igaz minden, amit a közönséges algebrai elméletekről elmondtunk:

- Megadható a szignatúrájuk (lásd az 5. fejezetben).
- Minden szignatúrához megadhatók a következő fogalmak: algebra/modell, homomorfizmus.
- Minden általánosított algebrai elméletnek van szintaxisa.
- Algebráknak van Descartes-szorzata, van triviális algebra.

62. Feladat. *Mutassuk meg, hogy minden típus nélküli kombinátor-algebra megad egy típusos kombinátor-algebrát! A másik irány miért nem teljesül?*

63. Gondolkodnivaló. *Mutassuk meg, hogy minden típusos kombinátor termnek van normálformája! (A normálformák ugyanazok, mint a típus nélküli esetben (31. gondolkodnivaló); Tait logikai predikátumok módszerét kell használni a bizonyításra.)*

Megadunk néhány további általánosított algebrai elméletet.

64. Definíció (Gráf)**.**

$$\begin{aligned}&\mathsf{V} : \mathsf{Set}\\&\mathsf{E} : \mathsf{V} \to \mathsf{V} \to \mathsf{Set}\end{aligned}$$

A gráfban csúcsok (vertex) és élek (edge) vannak.

65. Definíció (Előrendezés (preorder))**.**

$$\begin{aligned}&\mathsf{Ob} && : \mathsf{Set}\\&\mathsf{Mor} && : \mathsf{Ob} \to \mathsf{Ob} \to \mathsf{Set}\\&- \circ - && : \mathsf{Mor}\, J\, I \to \mathsf{Mor}\, K\, J \to \mathsf{Mor}\, K\, I\\&\mathsf{id} && : \mathsf{Mor}\, I\, I\\&\mathsf{irr} && : (f\ g : \mathsf{Mor}\, J\, I) \to f = g\end{aligned}$$

A preorder egy tranzitív és reflexív egyszerű gráf.

66. Definíció (Kategória)**.**

$$\begin{aligned}&\mathsf{Ob} && : \mathsf{Set}\\&\mathsf{Mor} && : \mathsf{Ob} \to \mathsf{Ob} \to \mathsf{Set}\\&- \circ - && : \mathsf{Mor}\, J\, I \to \mathsf{Mor}\, K\, J \to \mathsf{Mor}\, K\, I\\&\mathsf{id} && : \mathsf{Mor}\, I\, I\\&\mathsf{ass} && : (f \circ g) \circ h = f \circ (g \circ h)\\&\mathsf{idl} && : \mathsf{id} \circ f = f\\&\mathsf{idr} && : f \circ \mathsf{id} = f\end{aligned}$$

A kategória egy tranzitív és reflexív gráf, melyben az élek kompozíciója asszociatív, és a reflexív élekkel való kompozíció az identitás.

67. Feladat. *Minden egységelemes félcsoport megad egy kategóriát (ahol egy darab objektum van).*

68. Feladat. *Adjuk meg a gráf-, preorder- és kategória-homomorfizmus fogalmát! Utóbbit funktornak szokás nevezni.*

69. Feladat. *Adjuk meg a halmazok kategóriáját, melyben az objektumok halmazok, a morfizmusok függvények az adott halmazok között.*

70. Megjegyzés. *Az egyenlőségek nélküli általánosított algebrai elméletek induktív-induktív típusoknak felelnek meg. Ha egyenlőségeket is tartalmaznak, ezeket kvóciens induktív-induktív típusoknak nevezzük [Kov23]. Az általánosított algebrai elméleteket először Cartmell írta le [Car86].*

3. Másodrendű algebrai elméletek

Ebben a fejezetben másodrendű algebrai elméleteket (second-order algebraic theory, SOAT) vizsgálunk.

A legtöbb nyelvben vannak változók, vagy más néven kötéssel rendelkező operátorok. Például az $\int_a^b \frac{1}{x^2}\,\mathrm{d}x$ kifejezésben az integrálás $\int -\mathrm{d}-$ operátor köti az x változót az $\frac{1}{x^2}$ kifejezésben, ez utóbbit nevezzük a kötés hatáskörének. Ez a kifejezés megegyezik az $\int_a^b \frac{1}{y^2}\,\mathrm{d}y$ kifejezéssel, hiszen a kötött változó neve nem számít, az csak egy mutató a kötés helyére. Hasonlóan, a $\sum_{x=1}^{10} x^2$ kifejezésben a $\sum$ a kötő operátor, a kötött változó az x, a kötés hatásköre az x^2 kifejezés. Az `int f(int i) { return(i+1); }` kifejezésben a függvénydefiníció a kötő operátor, a kötött név az `i`, a kötés hatásköre a kapcsos zárójelek (`{` és `}`) közötti rész.

71. Jelölés (Névnélküli függvény)**.** *A „plusz egy” $\mathbb{N} \to \mathbb{N}$ függvényt megadhatjuk az alábbi két, ekvivalens módon:* $\mathsf{pluszegy}\, n := n + 1$ *illetve* $\lambda n.n + 1$*. Utóbbinak előnye, hogy nem kell nevet adnunk a függvénynek.*

Az integrálás operátor például a következő halmazban van (ahol $a, b : \mathbb{R}$): $\int_a^b : (\mathbb{R} \to \mathbb{R}) \to \mathbb{R}$. A fenti példa kifejezést az alábbi módon írhatjuk: $\int_a^b (\lambda x.\frac{1}{x^2})$, erre a szokásos jelölés a fenti, ahol $\lambda x.$ helyett a kifejezés végére írunk dx-et. Hasonlóan, az összegzés operátor precízen például mint $\sum : \mathbb{N} \to \mathbb{N} \to (\mathbb{N} \to \mathbb{N}) \to \mathbb{N}$ adható meg, és a fenti kifejezés $\sum\ 1\ 10\ (\lambda x.x^2)$-et jelent.

A legegyszerűbb, kötésekkel rendelkező nyelv a lambda-kalkulus, amely a típus nélküli kombinátor-kalkulushoz hasonlóan Turing-teljes. Egy kötő operátor van, a $\mathsf{lam} : (\mathsf{Tm} \to \mathsf{Tm}) \to \mathsf{Tm}$, például az identitás függvényt a következőképp adjuk meg: $\mathsf{lam}\,(\lambda x.x)$. Ne tévessze meg az olvasót a két különböző „lambda": a λ a metanyelvi (metaelméleti) lambda, hiszen a metanyelvünkben is tudunk függvényeket megadni, míg a lam a tárgynyelvi lambda.

72. Definíció (Lambda-kalkulus)**.** *Egy másodrendű lambda-kalkulus modell (algebra) a következő komponensekből áll.*

$$\begin{aligned}
\mathsf{Tm} &: \mathsf{Set}\\
- \cdot - &: \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm}\\
\mathsf{lam} &: (\mathsf{Tm} \to \mathsf{Tm}) \to \mathsf{Tm}\\
\beta &: \mathsf{lam}\, f \cdot u = f\, u
\end{aligned}$$

A lambda-kalkulus hasonló a kombinátor-kalkulushoz, de a K és S helyett csak egy operátorunk van, a lam, és ennek megfelelően csak egy számítási szabályuk van. A lam azonban egy másodrendű operátor, hiszen a bemenete egy függvény. Ilyen másodrendű operátorok nem szerepelhetnek algebrai elméletekben, a 33. definícióval nem fejezhetők ki. A számítási szabály azt fejezi ki, hogy egy lam-mal megadott függvény tárgynyelvi applikációja ugyanaz, mint a metanyelvi applikáció.

Egy másodrendű lambda-kalkulus modellből készíthető egy kombinátor-algebra a következőképp.

$$\begin{aligned}
\mathsf{Tm} &:= \mathsf{Tm}\\
f \cdot u &:= f \cdot u\\
\mathsf{K} &:= \mathsf{lam}\left(\lambda u.\mathsf{lam}\left(\lambda f.u\right)\right)\\
\mathsf{S} &:= \mathsf{lam}\left(\lambda f.\mathsf{lam}\left(\lambda g.\mathsf{lam}\left(\lambda u.f \cdot u \cdot (g \cdot u)\right)\right)\right)
\end{aligned}$$

Az egyenlőségek is teljesülnek:

$$\begin{aligned}
\mathsf{K}\beta : \mathsf{K} \cdot u \cdot f & && = (\mathsf{K}\text{ definíciója})\\
\left(\left(\mathsf{lam}\left(\lambda u.\mathsf{lam}\left(\lambda f.u\right)\right)\right) \cdot u\right) \cdot f & && = (\beta)\\
\left(\left(\lambda u.\mathsf{lam}\left(\lambda f.u\right)\right) u\right) \cdot f & && = (\text{metanyelvi függvényalkalmazás})\\
\left(\mathsf{lam}\left(\lambda f.u\right)\right)\right) \cdot f & && = (\beta)\\
(\lambda f.u)\, f & && = (\text{metanyelvi függvényalkalmazás})\\
u &&&
\end{aligned}$$

73. Feladat. *Mutassuk meg, hogy a fent megadott modellben az $\mathsf{S}\beta$ szabály is teljesül!*

74. Jelölés (Levezetési szabályok)**.** *A lambda-kalkulus operátorainak a megadására egy alternatív jelölés levezetési szabályokkal történik. Az elsőrendű függvénytér konklúziója helyett vízszintes vonalat írunk, a bemeneteket összegyűjtük (uncurry-zzük), a másodrendű függvénytér helyett $\vdash$ (turnstyle) szimbólumot.*

$$\frac{f : \mathsf{Tm} \qquad u : \mathsf{Tm}}{f \cdot u : \mathsf{Tm}} \qquad \frac{x : \mathsf{Tm} \vdash f : \mathsf{Tm}}{\mathsf{lam}\, x.f : \mathsf{Tm}} \qquad \frac{x : \mathsf{Tm} \vdash f : \mathsf{Tm} \qquad u : \mathsf{Tm}}{\beta : (\mathsf{lam}\, x.\, f) \cdot u = f[x \mapsto u]}$$

A vízszintes vonal feletti komponensek a bemenetek, az alatta levő rész a kimenet. Az első szabályt például a következőképp olvassuk: ha f és u termek, akkor $f \cdot u$ is term. A levezetési fás jelölésben a másodrendű függvénytér ($\vdash$) használata esetén nevet adunk a másodrendű paraméternek, fent erre x-et használtunk. A választott név megjelenik a konklúzióban is. A metanyelvi applikációt $f\,u$ helyett $f[x \mapsto u]$-val jelöljük, amit úgy olvasunk, hogy f-ben az x értékét u-val helyettesítjük. A nevek nem számítanak, hiszen az $x : \mathsf{Tm} \vdash f : \mathsf{Tm}$ ugyanazt jelenti, mint hogy $f : \mathsf{Tm} \to \mathsf{Tm}$. A β szabály konklúzióját írhattuk volna például úgy is, hogy $\beta : \mathsf{lam}\, y.\, f \cdot u = f[y \mapsto u]$, a második szabály helyett írhattunk volna az alábbit is:

$$\frac{y : \mathsf{Tm} \vdash f : \mathsf{Tm}}{\mathsf{lam}\, y.f : \mathsf{Tm}}$$

Ha csak egy szortunk van, akkor a sok $: \mathsf{Tm}$-et nem szoktuk kiírni, hanem az alábbi, tömörebb módon is megadhatjuk a szabályokat.

$$\frac{f \qquad u}{f \cdot u} \qquad \frac{x \vdash f}{\mathsf{lam}\, x.f} \qquad \frac{x \vdash f \qquad u}{\beta : (\mathsf{lam}\, x.f) \cdot u = f[x \mapsto u]}$$

A levezetési szabályos jelölés egyik előnye, hogy nem kell λ-kat írni a kötő operátorok számítási szabályainál, hátránya, hogy minden kötésnél meg kell adni egy nevet. Egy másik előny, hogy kikényszeríti a másodrendűséget, vagyis harmadrendű műveletek (például $((\mathsf{Tm} \to \mathsf{Tm}) \to \mathsf{Tm}) \to \mathsf{Tm}$) nem adhatók meg, mert a $\vdash$ bal oldalán nem szerepelhet függvény, vízszintes vonal vagy $\vdash$.

A levezetési szabályok között nincsenek változókra vonatkozóak, mint például az alábbi.

$$\frac{}{x : \mathsf{Tm} \vdash x : \mathsf{Tm}}$$

Ez a szabály a másodrendű algebrai leírásban egyszerűen a $\mathsf{Tm} \to \mathsf{Tm}$ identitás függvény lenne. Az ilyen és ehhez hasonló szabályokat implicit módon beleértjük a levezetési szabályos leírásba, hiszen az ilyen szabályok formális változata a másodrendű függvényekkel megadott. A levezetési szabályos változatot azért használjuk, mert a programozási nyelvek és bizonyításelméleti nyelvek ilyen jelölést szoktak használni.

Másodrendű algebrai elméleteknél nincs értelmes homomorfizmus-fogalom, például hogyan mondanánk ki azt, hogy egy $f : \mathsf{Tm}_A \to \mathsf{Tm}_B$ függvény megőrzi a lam operátort? Azt szeretnénk, hogy $f\,(\mathsf{lam}_A\,g) = \mathsf{lam}_B\,(f \circ g \circ ?)$, de nem tudunk a ? helyére mit írni, hiszen $\mathsf{Tm}_B \to \mathsf{Tm}_A$-ban nincs függvényünk ($\circ$ a szokásos függvénykompozíció). Ha f izomorfizmus, tehát egy olyan homomofizmus, melynek van inverze, akkor ez működik, de általánosságban szeretnénk tetszőleges modellbe kiértékelni a szintaxist, nem csak izomorf modellekbe. Emiatt a következő módszer [KX24] működik a homomorfizmus-fogalomra, illetve a szintaxisra: a másodrendű elméletet lefordítjuk elsőrendűre, és ott tekintjük ezeket a fogalmakat. A fordítás alapötlete, hogy bevezetjük a környezetek és a helyettesítések szortját, és minden operátort és egyenlőséget ezekkel is indexelünk. A környezetek gyűjtik össze a másodrendű paramétereket. Ha van egy másodrendű modellünk, abból mindig képezhető egy modellje a megfelelő elsőrendű elméletnek. Például a 72. definíció elsőrendű változata, melyet [KX24] algoritmusa ad, a következő.

75. Definíció (Lambda-kalkulus, elsőrendű modell)**.**

$$\begin{array}{ll}
\mathsf{Con} & : \mathsf{Set} \\
\mathsf{Sub} & : \mathsf{Con} \to \mathsf{Con} \to \mathsf{Set} \\
- \circ - & : \mathsf{Sub}\,\Delta\,\Gamma \to \mathsf{Sub}\,\Theta\,\Delta \to \mathsf{Sub}\,\Theta\,\Gamma \\
\mathsf{ass} & : (\gamma \circ \delta) \circ \theta = \gamma \circ (\delta \circ \theta) \\
\mathsf{id} & : \mathsf{Sub}\,\Gamma\,\Gamma \\
\mathsf{idl} & : \mathsf{id} \circ \gamma = \gamma \\
\mathsf{idr} & : \gamma \circ \mathsf{id} = \gamma \\
\diamond & : \mathsf{Con} \\
\epsilon & : \mathsf{Sub}\,\Gamma\,\diamond \\
\diamond\eta & : (\sigma : \mathsf{Sub}\,\Gamma\,\diamond) \to \sigma = \epsilon \\
\mathsf{Tm} & : \mathsf{Con} \to \mathsf{Set} \\
-[-] & : \mathsf{Tm}\,\Gamma \to \mathsf{Sub}\,\Delta\,\Gamma \to \mathsf{Tm}\,\Delta \\
[\circ] & : t[\gamma \circ \delta] = t[\gamma][\delta] \\
[\mathsf{id}] & : t[\mathsf{id}] = t \\
-\triangleright & : \mathsf{Con} \to \mathsf{Con} \\
-, - & : \mathsf{Sub}\,\Delta\,\Gamma \to \mathsf{Tm}\,\Delta \to \mathsf{Sub}\,\Delta\,(\Gamma\triangleright) \\
\mathsf{p} & : \mathsf{Sub}\,(\Gamma\triangleright)\,\Gamma \\
\mathsf{q} & : \mathsf{Tm}\,(\Gamma\triangleright) \\
\triangleright\beta_1 & : \mathsf{p} \circ (\gamma, t) = \mathsf{p} \\
\triangleright\beta_2 & : \mathsf{q}[\gamma, t] = t \\
\triangleright\eta & : (\sigma : \mathsf{Sub}\,\Delta\,(\Gamma\triangleright)) \to \sigma = (\mathsf{p} \circ \sigma, \mathsf{q}[\sigma]) \\
- \cdot - & : \mathsf{Tm}\,\Gamma \to \mathsf{Tm}\,\Gamma \to \mathsf{Tm}\,\Gamma \\
\cdot[] & : (t \cdot u)[\gamma] = (t[\gamma]) \cdot (u[\gamma]) \\
\mathsf{lam} & : \mathsf{Tm}\,(\Gamma\triangleright) \to \mathsf{Tm}\,\Gamma \\
\mathsf{lam}[] & : (\mathsf{lam}\,t)[\gamma] = \mathsf{lam}\,(t[\gamma \circ \mathsf{p}, \mathsf{q}]) \\
\beta & : \mathsf{lam}\,t \cdot u = t[\mathsf{id}, u]
\end{array}$$

A fenti definíciót a következőképp adhatjuk meg rövidebben: kategória terminális objektummal ($\mathsf{Con}, \ldots, \diamond\eta$), egy előkéve (presheaf) ezen a kategórián ($\mathsf{Tm}, \ldots, [\mathsf{id}]$), Tm lokálisan reprezentálható ($-\triangleright, \ldots, \triangleright\eta$), két Γ-val

indexelt művelet $-\cdot-$ és lam, melyek természetesek ($\cdot[]$ és $\mathsf{lam}[]$), végül van β egyenlőségünk. A kategória objektumai a környezetek (contexts), morfizmusai a helyettesítések (substitutions), a termek környezetekkel vannak indexelve, p-t gyengítésnek (weakening) nevezik, q a nullás De Bruijn index (a legutolsó változó a környezetben). Az összes változó az alábbi módon adható meg: $0 := \mathsf{q}, 1 := \mathsf{q}[\mathsf{p}], 2 := \mathsf{q}[\mathsf{p}][\mathsf{p}], 3 := \mathsf{q}[\mathsf{p}][\mathsf{p}]$ stb. Minden SOGAT elsőrendű modellje „kategória terminális objektummal" kezdődik, a szortok lokálisan reprezentálható előkévék lesznek ezen a kategórián, a műveletek természetes transzformációk, az egyenőségek pedig egyenlőségek. Érdemes megfigyelni a lam és a β szabályok másodrendű és elsőrendű megadása közötti különbségeket.

A további nyelveket csak SOGAT-ként adjuk meg, de mindegyik lefordítható GAT-ra a fentivel analóg módon [KX24].

76. Megjegyzés. *A legtöbb programozási nyelvekről szóló tankönyv nem másodrendű nyelvként, hanem elsőrendű nyelvként adja meg a nyelveket, tehát lényegileg közvetlenül a másodrendű → elsőrendű fordítás eredményét adják meg. Sőt, legtöbbször még ennél is alacsonyabb szinten, típusozatlan pretermekként adják meg a szintaxist – nem világos azonban, hogy az általunk használt magas szinten tárgyalható-e minden fontos tulajdonsága a programozási nyelveknek. A következő tankönyveket javasoljuk az érdeklődőknek [Har16, WKS22, Pie02, PdAC+23, LN15]. Bizonyos nyelvekben a változók használatára korlátozások vannak: például egy változó maximum egyszer vagy pontosan egyszer használható fel: ezeket szubstrukturális nyelveknek nevezzük. Ilyenek például a lineáris logika [Gir87], lineáris típusrendszer [Atk18], modális típuselméletek [GKNB21]. Ilyen nyelveket nem tárgyalunk ebben a jegyzetben, a másodrendű algebrai leíráshoz további kiegészítésekre lenne szükség.*

A nyelv fogalma illetve a szintaxis használata (a szintaxisban való programozás) azonban teljesen jól működik másodrendben is, ezt fogjuk itt illusztrálni. A programozás úgy működik, hogy feltételezünk egy másodrendű modellt, és ennek a komponenseit használjuk.

Például (feltételezve egy tetszőleges másodrendű lambda-kalkulus modellt, és annak komponenseit „levezetési szabályok" jelöléssel használva) az Y-kombinátor a következőképp adható meg.

$$\mathsf{Y} := \mathsf{lam}\, f.\big(\mathsf{lam}\, x.f \cdot (x \cdot x)\big) \cdot \big(\mathsf{lam}\, x.f \cdot (x \cdot x)\big)$$

Megmutatjuk, hogy ez egy fixpont-kombinátor:

$$\begin{aligned}
&\mathsf{Y} \cdot f && = (\mathsf{Y}\text{ definíciója})\\
&\Big(\mathsf{lam}\, f.\big(\mathsf{lam}\, x.f \cdot (x \cdot x)\big) \cdot \big(\mathsf{lam}\, x.f \cdot (x \cdot x)\big)\Big) \cdot f && = (\beta)\\
&\big(\mathsf{lam}\, x.f \cdot (x \cdot x)\big) \cdot \big(\mathsf{lam}\, x.f \cdot (x \cdot x)\big) && = (\beta)\\
&f \cdot \Big(\big(\mathsf{lam}\, x.f \cdot (x \cdot x)\big) \cdot \big(\mathsf{lam}\, x.f \cdot (x \cdot x)\big)\Big) && = (\mathsf{Y}\text{ definíciója})\\
&f \cdot (\mathsf{Y} \cdot f)
\end{aligned}$$

Feltételezve, hogy vannak $\mathsf{isZero} : \mathsf{Tm} \to \mathsf{Tm}$, kivonás $-\!-\!- : \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm}$, logikai elágazás $\mathsf{if}-\mathsf{then}-\mathsf{else}-$ műveleteink és számliteráljaink (ezek mind definiálhatók, csakúgy mint a típus nélküli kombinátor-kalkulus esetén), szeretnénk megadni egy fac termet, mely a faktoriális függvényt implementálja, vagyis az alábbi egyenlőség igaz rá (feltételezzük, hogy az $m * n$ szorzást már megadtuk):

$$\mathsf{fac} \cdot n = \mathsf{if}\, \mathsf{isZero}\, n\, \mathsf{then}\, 1\, \mathsf{else}\, n * \big(\mathsf{fac} \cdot (n - 1)\big)$$

Először is megadunk egy f termet, amely a rekurzív hívás struktúráját adja meg.

$$\mathsf{f} := \mathsf{lam}\, r.\mathsf{lam}\, n.\mathsf{if}\, \mathsf{isZero}\, n\, \mathsf{then}\, 1\, \mathsf{else}\, n * \big(r \cdot (n - 1)\big)$$

Ennek segítségével a faktoriális függvény a következő.

$$\mathsf{fac} := \mathsf{Y} \cdot \mathsf{f}$$

És ez a definíció teljesíti a fenti egyenlőséget:

$$\begin{aligned}
&\mathsf{fac} \cdot n && = (\mathsf{fac}\text{ definíciója})\\
&\mathsf{Y} \cdot \mathsf{f} \cdot n && = (\mathsf{Y}\text{ fixpont-kombinátor})\\
&\mathsf{f} \cdot (\mathsf{Y} \cdot \mathsf{f}) \cdot n && = (\mathsf{f}\text{ definíciója})\\
&\big(\mathsf{lam}\, r.\mathsf{lam}\, n.\mathsf{if}\, \mathsf{isZero}\, n\, \mathsf{then}\, 1\, \mathsf{else}\, n * (r \cdot (n - 1))\big) \cdot (\mathsf{Y} \cdot \mathsf{f}) \cdot n && = (\beta\text{ kétszer})\\
&\mathsf{if}\, \mathsf{isZero}\, n\, \mathsf{then}\, 1\, \mathsf{else}\, n * \big(\mathsf{Y} \cdot \mathsf{f} \cdot (n - 1)\big) && = (\mathsf{fac}\text{ definíciója})\\
&\mathsf{if}\, \mathsf{isZero}\, n\, \mathsf{then}\, 1\, \mathsf{else}\, n * \big(\mathsf{fac} \cdot (n - 1)\big)
\end{aligned}$$

77. Megjegyzés. *A lambda-kalkulus (elsőrendű) szintaxisa nem teljesen izomorf a kombinátor-kalkulus szintaxisával, de ha mindkettőhöz hozzáveszünk néhány újabb egyenlőséget, akkor már izomorfak lesznek [AKSV23].*

4. Másodrendű általánosított algebrai elméletek

A legtöbb nyelv tartalmaz változókat, ezért a megadásához másodrendű algebrai elméletre van szükségünk. A legtöbb nyelvben van valamilyen típus-fogalom, ezért általánosított algebrai elméletre van szükségünk. Ebben a fejezetben ezt a két általánosítást kombinálva leírunk néhány alapvető logikai és programozási nyelvet másodrendű általánosított algebrai elméletként (second-order generalised algebraic theory, SOGAT).

4.1. Függvény típus

A Razor nyelvet kiegészítjük függvény típusokkal.

78. Definíció (Razor függvényekkel)**.**

$$\frac{}{\mathsf{Ty} : \mathsf{Set}} \qquad \frac{A : \mathsf{Ty}}{\mathsf{Tm}\, A : \mathsf{Set}}$$

$$\frac{}{\mathsf{Bool} : \mathsf{Ty}} \qquad \frac{}{\mathsf{true} : \mathsf{Tm}\,\mathsf{Bool}} \qquad \frac{}{\mathsf{false} : \mathsf{Tm}\,\mathsf{Bool}} \qquad \frac{t : \mathsf{Tm}\,\mathsf{Bool} \qquad u : \mathsf{Tm}\, A \qquad v : \mathsf{Tm}\, A}{\mathsf{ite}\, t\, u\, v : \mathsf{Tm}\, A}$$

$$\frac{}{\mathsf{ite}\beta_1 : \mathsf{ite}\,\mathsf{true}\, u\, v = u} \qquad \frac{}{\mathsf{ite}\beta_2 : \mathsf{ite}\,\mathsf{false}\, u\, v = v}$$

$$\frac{}{\mathsf{Nat} : \mathsf{Ty}} \qquad \frac{n : \mathbb{N}}{\mathsf{num}\, n : \mathsf{Tm}\,\mathsf{Nat}} \qquad \frac{u : \mathsf{Tm}\,\mathsf{Nat} \qquad v : \mathsf{Tm}\,\mathsf{Nat}}{u + v : \mathsf{Tm}\,\mathsf{Nat}} \qquad \frac{t : \mathsf{Tm}\,\mathsf{Nat}}{\mathsf{isZero}\, t : \mathsf{Tm}\,\mathsf{Bool}}$$

$$\frac{}{+\beta : \mathsf{num}\, m + \mathsf{num}\, n = \mathsf{num}\,(m+n)} \qquad \frac{}{\mathsf{isZero}\beta_1 : \mathsf{isZero}\,(\mathsf{num}\, 0) = \mathsf{true}} \qquad \frac{}{\mathsf{isZero}\beta_2 : \mathsf{isZero}\,(\mathsf{num}\,(1+n)) = \mathsf{false}}$$

$$\frac{A : \mathsf{Ty} \qquad B : \mathsf{Ty}}{A \Rightarrow B : \mathsf{Ty}} \qquad \frac{x : \mathsf{Tm}\, A \vdash b : \mathsf{Tm}\, B}{\mathsf{lam}\, x.b : \mathsf{Tm}\,(A \Rightarrow B)} \qquad \frac{f : \mathsf{Tm}\,(A \Rightarrow B) \qquad a : \mathsf{Tm}\, A}{f \cdot a : \mathsf{Tm}\, B} \qquad \frac{}{\Rightarrow\beta : (\mathsf{lam}\, x.b) \cdot a = b[x \mapsto a]}$$

$$\frac{f : \mathsf{Tm}\,(A \Rightarrow B)}{\Rightarrow\eta : f = \mathsf{lam}\, x.f \cdot x}$$

Az olyan nyelveknél, melyeknél ugyanez a két szort van, megadhatunk egy rövidebb jelölést is. Egyszerűen elhagyjuk a Ty-t és a Tm-et. Egyértelmű, hogy adott esetben melyikről van szó, mert a termeknek mindig oda van írva a típusa kettősponttal elválasztva. Ezenkívül az egyenlőségeknek nem adunk nevet. Ezzel a spórolósabb jelöléssel ugyanez a kalkulus a következő.

$$\frac{}{\mathsf{Bool}} \qquad \frac{}{\mathsf{true} : \mathsf{Bool}} \qquad \frac{}{\mathsf{false} : \mathsf{Bool}} \qquad \frac{t : \mathsf{Bool} \qquad u : A \qquad v : A}{\mathsf{ite}\, t\, u\, v : A} \qquad \frac{}{\mathsf{ite}\,\mathsf{true}\, u\, v = u} \qquad \frac{}{\mathsf{ite}\,\mathsf{false}\, u\, v = v}$$

$$\frac{}{\mathsf{Nat}} \qquad \frac{n : \mathbb{N}}{\mathsf{num}\, n : \mathsf{Nat}} \qquad \frac{u : \mathsf{Nat} \qquad v : \mathsf{Nat}}{u + v : \mathsf{Nat}} \qquad \frac{t : \mathsf{Nat}}{\mathsf{isZero}\, t : \mathsf{Bool}} \qquad \frac{}{\mathsf{num}\, m + \mathsf{num}\, n = \mathsf{num}\,(m+n)}$$

$$\frac{}{\mathsf{isZero}\,(\mathsf{num}\, 0) = \mathsf{true}} \qquad \frac{}{\mathsf{isZero}\,(\mathsf{num}\,(1+n)) = \mathsf{false}}$$

$$\frac{A \qquad B}{A \Rightarrow B} \qquad \frac{x : A \vdash b : B}{\mathsf{lam}\, x.b : A \Rightarrow B} \qquad \frac{f : A \Rightarrow B \qquad a : A}{f \cdot a : B} \qquad \frac{}{(\mathsf{lam}\, x.b) \cdot a = b[x \mapsto a]} \qquad \frac{f : A \Rightarrow B}{f = \mathsf{lam}\, x.f \cdot x}$$

Ugyanez algebrai jelöléssel:

$$\begin{array}{ll}
\mathsf{Ty} & : \mathsf{Set} \\
\mathsf{Tm} & : \mathsf{Ty} \to \mathsf{Set} \\
\mathsf{Bool} & : \mathsf{Ty} \\
\mathsf{true} & : \mathsf{Tm}\,\mathsf{Bool} \\
\mathsf{false} & : \mathsf{Tm}\,\mathsf{Bool} \\
\mathsf{ite} & : \mathsf{Tm}\,\mathsf{Bool} \to \mathsf{Tm}\, A \to \mathsf{Tm}\, A \to \mathsf{Tm}\, A \\
\mathsf{Nat} & : \mathsf{Ty} \\
\mathsf{num} & : \mathbb{N} \to \mathsf{Tm}\,\mathsf{Nat} \\
- + - & : \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Nat} \\
\mathsf{isZero} & : \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Bool} \\
\mathsf{ite}\beta_1 & : \mathsf{ite}\,\mathsf{true}\, u\, v = u \\
\mathsf{ite}\beta_2 & : \mathsf{ite}\,\mathsf{false}\, u\, v = v \\
+\beta & : \mathsf{num}\, m + \mathsf{num}\, n = \mathsf{num}\,(m+n) \\
\mathsf{isZero}\beta_1 & : \mathsf{isZero}\,(\mathsf{num}\, 0) = \mathsf{true}
\end{array}$$

$$
\begin{array}{ll}
\mathsf{isZero}\beta_2 & : \mathsf{isZero}\,(\mathsf{num}\,(1+n)) = \mathsf{false} \\
- \Rightarrow - & : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty} \\
\mathsf{lam} & : (\mathsf{Tm}\,A \to \mathsf{Tm}\,B) \to \mathsf{Tm}\,(A \Rightarrow B) \\
- \cdot - & : \mathsf{Tm}\,(A \Rightarrow B) \to \mathsf{Tm}\,A \to \mathsf{Tm}\,B \\
\Rightarrow\beta & : \mathsf{lam}\,b \cdot a = b\,a \\
\Rightarrow\eta & : f = \mathsf{lam}\,\lambda x.f \cdot x
\end{array}
$$

79. Jelölés. *Az utolsó négy sort még rövidebben is meg tudjuk adni, a fentivel megegyezőt jelent az alábbi jelölés.*

$$\mathsf{lam} : (\mathsf{Tm}\,A \to \mathsf{Tm}\,B) \cong \mathsf{Tm}\,(A \Rightarrow B) : - \cdot -$$

Azt mondjuk, hogy a $(\mathsf{Tm}\,A \to \mathsf{Tm}\,B)$ *és a* $\mathsf{Tm}\,(A \Rightarrow B)$ *halmazok izomorfak. Az izomorfizmus két oldalára írjuk az operátorokat, amelyek a két oldal között képeznek. Általánosságban:*

$$(f : X \cong Y : g) \;:=\; (f : X \to Y) \times (g : Y \to X) \times \big((x : X) \to g\,(f\,x) = x\big) \times \big((y : Y) \to f\,(g\,y) = y\big).$$

Az egyes típusokra vonatkozó operátorok és egyenlőségek a következőképp csoportosíthatók: típuslétrehozó szabályok, konstruktorok (bevezető szabályok, introduction rules), destruktorok (eliminátorok, kivezető szabályok, elimination rules), számítási szabályok (β), egyediség szabályok (η). A β szabályok azt mondják, mi történik, ha destruktort konstruktorra alkalmazunk (pontosabban, ha a destruktor fő paraméterei konstruktorok). Az η szabályok elmondják, mi történik ha konstruktort destruktorra alkalmazunk. A három típusunkra a következőképp csoportosítjuk a szabályokat.

típus bevezető	konstruktor	destruktor	számítás	egyediség
Bool	$\mathsf{true}, \mathsf{false}$	ite	$\mathsf{ite}\beta_1, \mathsf{ite}\beta_2$	
Nat	num	$- + -, \mathsf{isZero}$	$+\beta, \mathsf{isZero}\beta_1, \mathsf{isZero}\beta_2$	
$- \Rightarrow -$	lam	$- \cdot -$	$\Rightarrow\beta$	$\Rightarrow\eta$

80. Példa. *Levezetjük az alábbi „kétszer" függvény típusát rövid jelöléssel.*

$$\dfrac{\dfrac{}{f : A \Rightarrow A, x : A \vdash f : A \Rightarrow A} \quad \dfrac{\dfrac{}{f : A \Rightarrow A, x : A \vdash f : A \Rightarrow A} \quad \dfrac{}{f : A \Rightarrow A, x : A \vdash x : A}}{f : A \Rightarrow A, x : A \vdash f \cdot x : A}}{\dfrac{f : A \Rightarrow A, x : A \vdash f \cdot (f \cdot x) : A}{\dfrac{f : A \Rightarrow A \vdash \mathsf{lam}\,\lambda x.f \cdot (f \cdot x) : A \Rightarrow A}{\mathsf{lam}\,\lambda f.\mathsf{lam}\,\lambda x.f \cdot (f \cdot x) : (A \Rightarrow A) \Rightarrow A \Rightarrow A}}}$$

81. Példa. *Levezetjük a függvénykompozíciót.*

$$\dfrac{\dfrac{}{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash f : B \Rightarrow A} \quad \dfrac{\dfrac{}{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash g : C \Rightarrow B} \quad \dfrac{}{\ldots, x : C \vdash x : C}}{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash g \cdot x : B}}{\dfrac{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash f \cdot (g \cdot x) : A}{\dfrac{f : B \Rightarrow A, g : C \Rightarrow B \vdash \mathsf{lam}\,\lambda x.f \cdot (g \cdot x) : C \Rightarrow A}{\dfrac{f : B \Rightarrow A \vdash \mathsf{lam}\,\lambda g.\mathsf{lam}\,\lambda x.f \cdot (g \cdot x) : (C \Rightarrow B) \Rightarrow C \Rightarrow A}{\mathsf{lam}\,\lambda f.\mathsf{lam}\,\lambda g.\mathsf{lam}\,\lambda x.f \cdot (g \cdot x) : (B \Rightarrow A) \Rightarrow (C \Rightarrow B) \Rightarrow C \Rightarrow A}}}}$$

82. Példa. *A „kettő hozzáadása" függvényt az alábbi módon adjuk meg.*

$$\begin{array}{l}
\mathsf{add2} : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat}) \\
\mathsf{add2} := \mathsf{lam}\,\lambda x.x + \mathsf{num}\,2
\end{array}$$

Működése:

$$
\begin{array}{ll}
\mathsf{add2} \cdot \mathsf{num}\,3 & =(\textit{definíció feloldása}) \\
(\mathsf{lam}\,\lambda x.x + \mathsf{num}\,2) \cdot \mathsf{num}\,3 & =(\Rightarrow\beta) \\
(\lambda x.x + \mathsf{num}\,2)\,(\mathsf{num}\,3) & =(\textit{metanyelvi függvényalkalmazás}) \\
(\mathsf{num}\,3 + \mathsf{num}\,2) & =(+\beta) \\
(\mathsf{num}\,(3+2)) & =(\textit{metaelméleti összeadás}) \\
(\mathsf{num}\,5) &
\end{array}
$$

83. Példa. *Megmutatjuk, hogy a típuslevezetés hogyan alkalmazható a típus kitalálására: levezetjük a*

$$\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z$$

kifejezés típusát, vagyis felépítjük a levezetési fát az aljáról kezdve. Egyelőre azt mondjuk, hogy a típus valamilyen tetszőleges A.

$$\frac{?}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : A}$$

Mivel a levezetendő kifejezés gyökerénél lam *van, ezért az arra vonatkozó levezetési szabályt használjuk, és megtanuljuk, hogy $A = B \Rightarrow C$ valamilyen B-ra és C-re*

$$\frac{\dfrac{?}{x : B \vdash \mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : C}}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : B \Rightarrow C}$$

A ? alatti kifejezés lam*, megint ugyanazt a szabályt használjuk, és megtanuljuk, hogy $C = D \Rightarrow E$, C minden előfordulását lecseréljük $D \Rightarrow E$-re.*

$$\frac{\dfrac{\dfrac{?}{x : B, y : D \vdash \mathsf{lam}\,\lambda z.y\cdot x\cdot z : E}}{x : B \vdash \mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : D \Rightarrow E}}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : B \Rightarrow (D \Rightarrow E)}$$

A következő lépés újra a $\Rightarrow$ típus bevezető szabálya, és megtanuljuk, hogy $E = F \Rightarrow G$:

$$\frac{\dfrac{\dfrac{\dfrac{?}{x : B, y : D, z : F \vdash y\cdot x\cdot z : G}}{x : B, y : D \vdash \mathsf{lam}\,\lambda z.y\cdot x\cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : D \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

Most alkalmazás-kifejezést kell ?-el levezetni, és bevezetünk egy új H metaváltozót, mert az alkalmazás értelmezési tartománya nem látszik a konklúziójából. Az alkalmazás levezetési szabályának két feltétele van, szóval most már két befejezetlen levezetési fát jelölünk ?-el. Vegyük észre, hogy $y\cdot x\cdot z = (y\cdot x)\cdot z$.

$$\frac{\dfrac{\dfrac{\dfrac{\dfrac{?}{x : B, y : D, z : F \vdash y\cdot x : H \Rightarrow G} \qquad \dfrac{?}{x : B, y : D, z : F \vdash z : H}}{x : B, y : D, z : F \vdash y\cdot x\cdot z : G}}{x : B, y : D \vdash \mathsf{lam}\,\lambda z.y\cdot x\cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : D \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

Először befejezzük a jobb oldali ?-t: z egy változó, melynek típusa a környezetben ($(x : B, y : D, z : F)$-ban) meg van határozva, tehát megtanuljuk, hogy $H = F$:

$$\frac{\dfrac{\dfrac{\dfrac{\dfrac{?}{x : B, y : D, z : F \vdash y\cdot x : F \Rightarrow G} \qquad \dfrac{}{x : B, y : D, z : F \vdash z : F}}{x : B, y : D, z : F \vdash y\cdot x\cdot z : G}}{x : B, y : D \vdash \mathsf{lam}\,\lambda z.y\cdot x\cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : D \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

Bal oldalon alkalmazás van, bevezetünk egy új I metaváltozót:

$$\frac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{?}{x : B, y : D, z : F \vdash y : I \Rightarrow F \Rightarrow G} \qquad \dfrac{?}{x : B, y : D, z : F \vdash x : I}}{x : B, y : D, z : F \vdash y\cdot x : F \Rightarrow G} \qquad \dfrac{}{x : B, y : D, z : F \vdash z : F}}{x : B, y : D, z : F \vdash y\cdot x\cdot z : G}}{x : B, y : D \vdash \mathsf{lam}\,\lambda z.y\cdot x\cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : D \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

A jobb oldali ?-t megoldva látjuk, hogy $I = B$, mert x-nek ez a típusa a környezetben:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{?}{x : B, y : D, z : F \vdash y : B \Rightarrow F \Rightarrow G} \quad \dfrac{}{x : B, y : D, z : F \vdash x : B}}{x : B, y : D, z : F \vdash y \cdot x : F \Rightarrow G} \quad \dfrac{}{x : B, y : D, z : F \vdash z : F}}{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : D \vdash \mathsf{lam}\, \lambda z.y \cdot x \cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\, \lambda x.\mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

Bal oldalon azt tanuljuk meg, hogy $D = B \Rightarrow F \Rightarrow G$.

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y : B \Rightarrow F \Rightarrow G} \quad \dfrac{}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash x : B}}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y \cdot x : F \Rightarrow G} \quad \dfrac{}{\ldots, z : F \vdash z : F}}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : B \Rightarrow F \Rightarrow G \vdash \mathsf{lam}\, \lambda z.y \cdot x \cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : (B \Rightarrow F \Rightarrow G) \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\, \lambda x.\mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : B \Rightarrow ((B \Rightarrow F \Rightarrow G) \Rightarrow (F \Rightarrow G))}$$

Eltávolítva a felesleges zárójeleket a következőt kapjuk:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y : B \Rightarrow F \Rightarrow G} \quad \dfrac{}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash x : B}}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y \cdot x : F \Rightarrow G} \quad \dfrac{}{\ldots, z : F \vdash z : F}}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : B \Rightarrow F \Rightarrow G \vdash \mathsf{lam}\, \lambda z.y \cdot x \cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : (B \Rightarrow F \Rightarrow G) \Rightarrow F \Rightarrow G}}{\mathsf{lam}\, \lambda x.\mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : B \Rightarrow (B \Rightarrow F \Rightarrow G) \Rightarrow F \Rightarrow G}$$

84. Feladat. *Adjuk meg Schönfinkel S és K kombinátorait, vezessük le a típusukat, és mutassuk meg, hogy teljesülnek a számítási szabályaik!*

85. Feladat. *Adjuk meg a függvénykompozíciót a $\mathsf{Tm}\left((B \Rightarrow A) \Rightarrow (C \Rightarrow B) \Rightarrow (C \Rightarrow A)\right)$ halmaz elemeként, ahol A, B és C tetszőleges típusok!*

86. Feladat. *Mutassuk meg, hogy a függvénykompozíció asszociatív!*

87. Feladat. *Adjuk meg az identitás függvényt a $\mathsf{Tm}\,(A \Rightarrow A)$ halmaz elemeként minden A-ra!*

88. Feladat. *Mutassuk meg, hogy $\mathsf{Tm}\,(A \Rightarrow A)$ minden A-ra egységelemes félcsoportot alkot a kompozícióval és az identitással!*

89. Feladat. *Mutassuk meg, hogy a típusok és termek kategóriát alkotnak (66. definíció), ahol az objektumok a típusok, és egy A-ból B-be menő morfizmus egy $\mathsf{Tm}\,(A \Rightarrow B)$ term!*

90. Feladat. *Ha $\mathsf{num}\,0 = \mathsf{num}\,1$, akkor bármely A-ra és $u, v : \mathsf{Tm}\,A$-ra $u = v$.*

91. Meta-állítás. *A függvényekkel kiegészített Razor nyelv egyenlőségi konziszenciája: van olyan A és $u, v : \mathsf{Tm}\,A$, hogy $u \neq v$.*

Bizonyítás. Megadjuk a nyelvünk sekély beágyazását a metanyelvbe. Ezt metacirkuláris interpetációnak is nevezik, hiszen a tárgynyelvi konstrukciókat a nekik pontosan megfelelő metanyelvi konstrukcióval adjuk meg. A sekély beágyazás egy kiértékelőnek is tekinthető, a metanyelvünkbe értékeljük ki a tárgynyelvünket.

$$\begin{array}{ll}
\mathsf{Ty} & := \mathsf{Set} \\
\mathsf{Tm}\,A & := A \\
\mathsf{Bool} & := \{\mathsf{tt}, \mathsf{ff}\} \\
\mathsf{true} & := \mathsf{tt} \\
\mathsf{false} & := \mathsf{ff} \\
\mathsf{ite}\,t\,u\,v & := \mathsf{match}\,t\,\mathsf{with}\,\{\mathsf{tt}.u, \mathsf{ff}.v\} \\
\mathsf{Nat} & := \mathbb{N} \\
\mathsf{num}\,n & := n \\
m + n & := m + n
\end{array}$$

$$
\begin{array}{ll}
\mathsf{isZero}\,0 & := \mathsf{tt} \\
\mathsf{isZero}\,(1+n) & := \mathsf{ff} \\
\mathsf{ite}\beta_1 & : \mathsf{ite}\,\mathsf{true}\,u\,v = \mathsf{match}\,\mathsf{tt}\,\mathsf{with}\,\{\mathsf{tt}.u, \mathsf{ff}.v\} = u \\
\mathsf{ite}\beta_1 & : \mathsf{ite}\,\mathsf{false}\,u\,v = \mathsf{match}\,\mathsf{ff}\,\mathsf{with}\,\{\mathsf{tt}.u, \mathsf{ff}.v\} = v \\
+\beta & : \mathsf{num}\,m + \mathsf{num}\,n = m + n = \mathsf{num}\,(m+n) \\
\mathsf{isZero}\beta_1 & : \mathsf{isZero}\,(\mathsf{num}\,0) = \mathsf{isZero}\,0 = \mathsf{tt} = \mathsf{true} \\
\mathsf{isZero}\beta_2 & : \mathsf{isZero}\,(\mathsf{num}\,(1+n)) = \mathsf{isZero}\,(1+n) = \mathsf{ff} = \mathsf{false} \\
A \Rightarrow B & := A \to B \\
\mathsf{lam}\,t & := t \\
t \cdot u & := t\,u \\
\Rightarrow\beta & : \mathsf{lam}\,b \cdot a = b\,a \\
\Rightarrow\eta & : f = \lambda x.f\,x = \mathsf{lam}\,\lambda x.f \cdot x
\end{array}
$$

A típusokat metaelméleti típusokkal adjuk meg, Tm a metaelméleti identitás függvény, Bool a kételemű halmaz, Nat a természetes számok halmaza. A későbbi komponensek típusa függ az előző komponensek definícióitól, például ite típusa $\mathsf{Tm}\,\mathsf{Bool} \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A = \mathsf{Bool} \to A \to A \to A = \{\mathsf{tt}, \mathsf{ff}\} \to A \to A \to A$, $\Rightarrow$ típusa $\mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty} = \mathsf{Set} \to \mathsf{Set} \to \mathsf{Set}$, lam típusa $(\mathsf{Tm}\,A \to \mathsf{Tm}\,B) \to \mathsf{Tm}\,(A \Rightarrow B) = (A \to B) \to (A \Rightarrow B) = = (A \to B) \to (A \to B)$. Az egyenlőség-bizonyításoknál csak a definíciókat használtuk fel.

Az egyenlőségi konzisztenciához A-t Bool-nak választjuk, $u := \mathsf{true}$, $v := \mathsf{false}$. Ezek egyenlősége a sekély beágyazásban a metanyelvi kételemű típus két elemének (tt-nek és ff-nek) az egyenlőségét jelenti. □

Az egyszerű típuselmélet a fenti nyelvünk azon része, melyben csak függvény típus van, illetve egy alaptípus ι (csak azért, hogy ne legyen üres a típusok halmaza).

92. Definíció (Egyszerű típusos lambda kalkulus)**.**

$$
\begin{array}{ll}
\mathsf{Ty} & : \mathsf{Set} \\
\mathsf{Tm} & : \mathsf{Ty} \to \mathsf{Set} \\
\iota & : \mathsf{Ty} \\
- \Rightarrow - & : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty} \\
\mathsf{lam} & : (\mathsf{Tm}\,A \to \mathsf{Tm}\,B) \to \mathsf{Tm}\,(A \Rightarrow B) \\
- \cdot - & : \mathsf{Tm}\,(A \Rightarrow B) \to \mathsf{Tm}\,A \to \mathsf{Tm}\,B \\
\Rightarrow\beta & : \mathsf{lam}\,b \cdot a = b\,a \\
\Rightarrow\eta & : f = \mathsf{lam}\,\lambda x.f \cdot x
\end{array}
$$

4.2. System T, definiálhatóság

A következő nyelvből kiszedjük a Bool-t, és megjavítjuk Nat-ot: először is, nem a metanyelvi $\mathbb{N}$-t örökli meg a tárgynyelvünk, hanem a nulla (zero) és rákövetkező (suc, successor) műveletekkel vezetjük be az (unáris) természetes számokat. Például $2 = \mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero})$ $3 = \mathsf{suc}\,(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero}))$; másodszor, Razor-ban nem tudjuk definiálni a szorzást, a mostani nyelvben viszont (szinte) minden függvényt meg tudunk adni természetes számokon. A természetes számok eliminátorai most már nem ad-hoc módon vannak megadva összeadás és isZero műveletekkel, hanem elvszerűen: az iterátor (ite) egy általános megoldás, mellyel rekurzív függvényeket tudunk megadni. A függvényekre vonatkozó szabályok nem változnak.

93. Definíció (Gödel-féle System T, iterátoros változat)**.**

$$
\begin{array}{ll}
\mathsf{Ty} & : \mathsf{Set} \\
\mathsf{Tm} & : \mathsf{Ty} \to \mathsf{Set} \\
- \Rightarrow - & : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty} \\
\mathsf{lam} & : (\mathsf{Tm}\,A \to \mathsf{Tm}\,B) \cong \mathsf{Tm}\,(A \Rightarrow B) : - \cdot - \\
\mathsf{Nat} & : \mathsf{Ty} \\
\mathsf{zero} & : \mathsf{Tm}\,\mathsf{Nat} \\
\mathsf{suc} & : \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Nat} \\
\mathsf{ite} & : \mathsf{Tm}\,A \to (\mathsf{Tm}\,A \to \mathsf{Tm}\,A) \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A \\
\mathsf{Nat}\beta_1 & : \mathsf{ite}\,z\,s\,\mathsf{zero} = z \\
\mathsf{Nat}\beta_2 & : \mathsf{ite}\,z\,s\,(\mathsf{suc}\,t) = s\,(\mathsf{ite}\,z\,s\,t)
\end{array}
$$

Az $\mathsf{ite}\,u\,v\,t$ kifejezés eredménye a következő: a t természetes számban zero-t u-ra, a suc-okat pedig v-re cseréljük. A duplázás függvényt a következőképp adjuk meg:

$$\begin{aligned}&\mathsf{dup} : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat})\\&\mathsf{dup} := \mathsf{lam}\,\lambda x.\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,x\end{aligned}$$

Ha egy $\mathsf{suc}\,(\mathsf{suc}\,(\ldots(\mathsf{suc}\,\mathsf{zero})\ldots))$ számban a zero-t lecseréljük zero-ra és a suc-okat két darab suc-ra, akkor megkapjuk a szám kétszeresét. Például:

$$\begin{aligned}&\mathsf{dup}\cdot\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero}) && =(\text{rövidítés feloldása})\\&\big(\mathsf{lam}\,\lambda x.\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,x\big)\cdot\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero}) && =(\Rightarrow\beta)\\&\big(\lambda x.\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,x\big)\,(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero})) && =(\text{metanyelvi alkalmazás})\\&\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero})) && =(\mathsf{Nat}\beta_2)\\&(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,\big(\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,(\mathsf{suc}\;\mathsf{zero})\big) && =(\text{metanyelvi alkalmazás})\\&\mathsf{suc}\,\Big(\mathsf{suc}\,\big(\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,(\mathsf{suc}\;\mathsf{zero})\big)\Big) && =(\mathsf{Nat}\beta_2)\\&\mathsf{suc}\,\Big(\mathsf{suc}\,\big((\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,(\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,\mathsf{zero})\big)\Big) && =(\text{metanyelvi alkalmazás})\\&\mathsf{suc}\,\Big(\mathsf{suc}\,\big(\mathsf{suc}\,(\mathsf{suc}\,(\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,\mathsf{zero}))\big)\Big) && =(\mathsf{Nat}\beta_1)\\&\mathsf{suc}\,\Big(\mathsf{suc}\,\big(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero})\big)\big)\Big)\end{aligned}$$

A természetes számok beágyazhatók System T-be: megadunk egy $\ulcorner - \urcorner : \mathbb{N} \to \mathsf{Tm}\,\mathsf{Nat}$ függvényt, mely n-hez $\mathsf{suc}^n\,\mathsf{zero}$-t rendeli, például $\ulcorner 3 \urcorner = \mathsf{suc}\,(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero}))$.

94. Definíció (Definiálhatóság)**.** *Egy $f : \mathbb{N} \to \mathbb{N}$ függvény definiálható System T-ben, ha létezik olyan $t : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat})$, melyre minden n-re $\ulcorner f\,n \urcorner = t \cdot \ulcorner n \urcorner$. Egy $f : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ függvény definiálható System T-ben, ha létezik olyan $t : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat} \Rightarrow \mathsf{Nat})$, melyre minden m-re és n-re $\ulcorner f\,m\,n \urcorner = t \cdot \ulcorner m \urcorner \cdot \ulcorner n \urcorner$.*

95. Feladat. *Az összeadás, szorzás, hatványozás és az Ackermann-függvény definiálható System T-ben.*

96. Feladat. *A következő függvények közül melyek definiálják az identitás függvényt?*

$$\begin{aligned}&\mathsf{lam}\,\lambda x.x\\&\mathsf{lam}\,\lambda x.\mathsf{ite}\,\mathsf{zero}\,\mathsf{suc}\,x\\&\mathsf{lam}\,\lambda x.\mathsf{ite}\,x\,(\lambda y.y)\,x\\&\mathsf{lam}\,\lambda x.\mathsf{ite}\,\mathsf{zero}\,(\lambda y.y)\,x\\&\mathsf{lam}\,\lambda x.\mathsf{ite}\,(\mathsf{suc}\,\mathsf{zero})\,\mathsf{suc}\,x\\&\mathsf{lam}\,\lambda x.\mathsf{ite}\,(\mathsf{suc}\,\mathsf{zero})\,(\lambda y.y)\,x\end{aligned}$$

97. Feladat. *A következő függvények közül melyek definiálják az $x \mapsto 2 * x + 1$ függvényt?*

$$\begin{aligned}&\mathsf{lam}\,\lambda x.\mathsf{suc}\,x\\&\mathsf{lam}\,\lambda x.\mathsf{suc}\,(\mathsf{ite}\,x\,\mathsf{suc}\,x)\\&\mathsf{lam}\,\lambda x.\mathsf{ite}\,(\mathsf{suc}\,x)\,\mathsf{suc}\,x\\&\mathsf{lam}\,\lambda x.\mathsf{ite}\,(\mathsf{suc}\,\mathsf{zero})\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,x\\&\mathsf{lam}\,\lambda x.\mathsf{suc}\,(\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,x)\end{aligned}$$

A fenti nyelvben nem tudunk megadni olyan $\mathsf{pred} : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat})$ termet, melyre teljesül, hogy $\mathsf{pred}\,(\mathsf{suc}\,t) = t$ minden $t : \mathsf{Tm}\,\mathsf{Nat}$-ra [Par89]. Ha vannak szorzat típusaink, akkor pred definiálható, tehát $\mathsf{pred}\,(\mathsf{suc}\,\ulcorner n \urcorner) = \ulcorner n \urcorner$ minden $n : \mathbb{N}$-re, de az előbbi feltételt továbbra sem fogja teljesíteni (a termek között lehetnek változók is, melyre nem vonatkozik számítási szabály). Ezt kijavítjuk a System T alábbi, iterátor helyett rekurzort tartalmazó változatával.

98. Definíció (Gödel-féle System T, rekurzoros változat)**.**

$$\begin{aligned}&\mathsf{Ty} && : \mathsf{Set}\\&\mathsf{Tm} && : \mathsf{Ty} \to \mathsf{Set}\\&- \Rightarrow - && : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty}\\&\mathsf{lam} && : (\mathsf{Tm}\,A \to \mathsf{Tm}\,B) \cong \mathsf{Tm}\,(A \Rightarrow B) : - \cdot -\end{aligned}$$

$$
\begin{aligned}
&\mathsf{Nat} && : \mathsf{Ty}\\
&\mathsf{zero} && : \mathsf{Tm}\,\mathsf{Nat}\\
&\mathsf{suc} && : \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Nat}\\
&\mathsf{rec} && : \mathsf{Tm}\,A \to (\mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A) \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A\\
&\mathsf{Nat}\beta_1 && : \mathsf{rec}\,z\,s\,\mathsf{zero} = z\\
&\mathsf{Nat}\beta_2 && : \mathsf{rec}\,z\,s\,(\mathsf{suc}\,n) = s\,n\,(\mathsf{rec}\,z\,s\,n)
\end{aligned}
$$

A rec operátort leírjuk a levezetési szabályos jelöléssel is: ez azért érdekes, mert a rec a második paraméterében két változót is köt, ezeket ilyenkor vesszővel elválasztva soroljuk fel, a kötött változók nevét pedig az operátor jelölésekor ponttal választjuk el:

$$\frac{z : A \qquad i : \mathsf{Nat}, a : A \vdash s : A \qquad n : \mathsf{Nat}}{\mathsf{rec}\,z\,(i.a.s)\,n : A}$$

A számítási szabályok a levezetési szabályos jelöléssel a következők.

$$\frac{}{\mathsf{rec}\,z\,(i.a.s)\,\mathsf{zero} = z} \qquad \frac{}{\mathsf{rec}\,z\,(i.a.s)\,(\mathsf{suc}\,n) = s[i \mapsto n, a \mapsto \mathsf{rec}\,z\,(i.a.s)\,n]}$$

A suc-ra vonatkozó számítási szabályában ($\mathsf{Nat}\beta_2$) párhuzamosan helyettesítjük az i és az a változók értékét. Algebrai jelöléssel explicit változónevekkel ez a következőnek felel meg.

$$\mathsf{rec}\,z\,(\lambda i\,a.s\,i\,a)\,(\mathsf{suc}\,n) = s\,n\,(\mathsf{rec}\,z\,s\,n)$$

99. Feladat. *Az iterátor megadható a rekurzorral.*

A pred függvény a következő.

$$
\begin{aligned}
&\mathsf{pred} : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat})\\
&\mathsf{pred} := \mathsf{lam}\,\lambda x.\mathsf{rec}\,\mathsf{zero}\,(\lambda y\,_.y)\,x
\end{aligned}
$$

Működése:

$$
\begin{aligned}
&\mathsf{pred}\cdot\,\mathsf{suc}\,t && =(\text{rövidítés feloldása})\\
&(\mathsf{lam}\,\lambda x.\mathsf{rec}\,\mathsf{zero}\,(\lambda y\,_.y)\,x)\cdot\mathsf{suc}\,t && =(\Rightarrow\beta)\\
&(\lambda x.\mathsf{rec}\,\mathsf{zero}\,(\lambda y\,_.y)\,x)\,(\mathsf{suc}\,t) && =(\text{metanyelvi alkalmazás})\\
&\mathsf{rec}\,\mathsf{zero}\,(\lambda y\,_.y)\,(\mathsf{suc}\,t) && =(\mathsf{Nat}\beta_2)\\
&(\lambda y\,_.y)\,t\,(\mathsf{rec}\,\mathsf{zero}\,(\lambda y\,_.y)\,t) && =(\text{metanyelvi alkalmazás})\\
&t
\end{aligned}
$$

100. Feladat. *A faktoriális függvény definiálható System T-ben.*

101. Meta-állítás. *System T-ben nincs olyan $v : \mathsf{Tm}\,\mathsf{Nat}$, hogy $v = \mathsf{suc}\,v$.*

Bizonyítás. A System T sekélyen beágyazható a metanyelvünkbe ugyanazzal a technikával, mint a 91. állítás bizonyításában. Ebben a beágyazásban $\mathsf{Ty} = \mathsf{Set}$, $\mathsf{Tm}\,A = A$, $\mathsf{Nat} = \mathbb{N}$, $\mathsf{suc} = \lambda x.x + 1$. Tehát ha létezik ilyen v a nyelvben, akkor a metanyelvben kapunk egy $v : \mathbb{N}$-et, melyre $v = 1 + v$, ami lehetetlen. □

102. Következmény. *A System T egyenlőségileg konzisztens.*

103. Feladat. *Adjuk meg a System T sekély beágyazását a metanyelvbe, annak minden részletével!*

104. Állítás. *Van olyan függvény, mely ha definiálható System T-ben, akkor létezik olyan $v : \mathsf{Tm}\,\mathsf{Nat}$, hogy $v = \mathsf{suc}\,v$.*

Bizonyítás. Először rögzítünk egy Gödel-kódoló függvényt, mely tetszőleges $\mathsf{Nat} \Rightarrow \mathsf{Nat}$ típusú termet elkódol egy természetes számba: $\mathsf{code} : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat}) \to \mathbb{N}$.

A System T kiértékelője egy $\mathsf{eval} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ függvény a következő tulajdonsággal: minden u-ra és n-re $\ulcorner\mathsf{eval}\,(\mathsf{code}\,u)\,n\urcorner = u \cdot \ulcorner n\urcorner$. Ha az eval definiálható, akkor van olyan t, melyre $\ulcorner\mathsf{eval}\,m\,n\urcorner = t \cdot \ulcorner m\urcorner \cdot \ulcorner n\urcorner$ minden m-re és n-re. Az u termet a következőképp rögzítjük: $u := \mathsf{lam}\,\lambda x.\mathsf{suc}\,(t \cdot x \cdot x)$. Ekkor:

$$
\begin{aligned}
&t \cdot \ulcorner\mathsf{code}\,u\urcorner \cdot \ulcorner\mathsf{code}\,u\urcorner && =(\mathsf{eval}\text{ definiálható})\\
&\ulcorner\mathsf{eval}\,(\mathsf{code}\,u)\,(\mathsf{code}\,u)\urcorner && =(\text{a kiértékelő tulajdonsága})\\
&u \cdot \ulcorner\mathsf{code}\,u\urcorner && =(u\text{ definíciója})\\
&(\mathsf{lam}\,\lambda x.\mathsf{suc}\,(t \cdot x \cdot x)) \cdot \ulcorner\mathsf{code}\,u\urcorner && =(\Rightarrow\beta)\\
&\mathsf{suc}\,(t \cdot \ulcorner\mathsf{code}\,u\urcorner \cdot \ulcorner\mathsf{code}\,u\urcorner)
\end{aligned}
$$

Tehát $v := t \cdot \ulcorner\mathsf{code}\,u\urcorner \cdot \ulcorner\mathsf{code}\,u\urcorner$ és $v = \mathsf{suc}\,v$. □

105. Következmény. *Vannak System T-ben nem definiálható függvények.*

106. Gondolkodnivaló. *Definiálható iterátoros System T-ben (tehát rekurzor nélkül) a* pred *?*

4.3. Fixpont-kombinátor

A következő nyelv ismét Turing-teljes, csakúgy, mint a típus nélküli kombinátor/lambda-kalkulus, de vannak benne típusok. Lényegileg a System T kiegészítése egy fixpont-kombinátorral, de nincs η-szabály a függvényekre. És mivel van fixpont-kombinátor, ezért nincs szükség rekurzorra a természetes számokhoz, elég egy ifZero C-stílusú if-then-else elágazás. A PCF a Programming Computable Functions rövidítése, Gordon Plotkin adta meg [Plo77], mint a típusos funkcionális programozási nyelvek (pl. ML, Haskell, OCaml) absztrakcióját.

107. Definíció (PCF)**.** *Két szortunk van, a típusok és a típusokkal indexelt termek.*

$$\frac{A \qquad B}{A \Rightarrow B} \qquad \frac{x : A \vdash b : B}{\mathsf{lam}\, x.b : A \Rightarrow B} \qquad \frac{f : A \Rightarrow B \qquad a : A}{f \cdot a : B} \qquad \frac{}{(\mathsf{lam}\, x.b) \cdot a = b[x \mapsto a]}$$

$$\frac{x : A \vdash t : A}{\mathsf{fix}\, x.t : A} \qquad \frac{}{\mathsf{fix}\, x.t = t[x \mapsto \mathsf{fix}\, x.t]}$$

$$\frac{}{\mathsf{Nat}} \qquad \frac{}{\mathsf{zero} : \mathsf{Nat}} \qquad \frac{n : \mathsf{Nat}}{\mathsf{suc}\, n : \mathsf{Nat}} \qquad \frac{b : \mathsf{Nat} \qquad t : A \qquad f : A}{\mathsf{ifZero}\, b\, t\, f : A} \qquad \frac{}{\mathsf{ifZero}\, \mathsf{zero}\, t\, f = t} \qquad \frac{}{\mathsf{ifZero}\, (\mathsf{suc}\, n)\, t\, f = f}$$

$$\frac{n : \mathsf{Nat}}{\mathsf{pred}\, n : \mathsf{Nat}} \qquad \frac{}{\mathsf{pred}\, (\mathsf{suc}\, n) = n} \qquad \frac{}{\mathsf{pred}\, \mathsf{zero} = \mathsf{zero}}$$

Mivel $\mathsf{fix}\, f = f\, (\mathsf{fix}\, f)$, és $f\, (\mathsf{fix}\, f) = f\, (f\, (\mathsf{fix}\, f))$, a $\mathsf{fix}\, f$ kifejezésre úgy lehet gondolni, mint a következő végtelen kifejezésre: $f\, (f\, (f\, (\ldots$

108. Példa (Duplázás)**.** *Meg szeretnénk adni egy* $\mathsf{double} : \mathsf{Tm}\, (\mathsf{Nat} \Rightarrow \mathsf{Nat})$ *termet, melyre a következő teljesül:*

$$\mathsf{double} \cdot n = \mathsf{ifZero}\, n\, \mathsf{zero}\, \Big(\mathsf{suc}\, \big(\mathsf{suc}\, (\mathsf{double} \cdot \mathsf{pred}\, n)\big)\Big)$$

Először is átírjuk a fenti egyenlőséget úgy, hogy kiemeljük a rekurzív hívást, és ezt double'*-vel rövidítjük. A rekurzív hívás helyén a* double' *bemenetét használjuk.*

$$\mathsf{double}' : \mathsf{Tm}\, (\mathsf{Nat} \Rightarrow \mathsf{Nat}) \to \mathsf{Tm}\, (\mathsf{Nat} \Rightarrow \mathsf{Nat})$$
$$\mathsf{double}'\, f := \mathsf{lam}\, \lambda n.\mathsf{ifZero}\, n\, \mathsf{zero}\, \Big(\mathsf{suc}\, \big(\mathsf{suc}\, (f \cdot \mathsf{pred}\, n)\big)\Big)$$

És megadjuk a duplázás függvényt fix *segítségével:*

$$\mathsf{double} : \mathsf{Tm}\, (\mathsf{Nat} \Rightarrow \mathsf{Nat})$$
$$\mathsf{double} := \mathsf{fix}\, \mathsf{double}'$$

Kiszámítjuk 2 duplázását:

$$\begin{aligned}
&\mathsf{double} \cdot 2 && = \\
&\mathsf{double} \cdot \mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero}) && = \\
&\mathsf{fix}\, \mathsf{double}' \cdot \mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero}) && = \\
&\mathsf{double}'\, (\mathsf{fix}\, \mathsf{double}') \cdot \mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero}) && = \\
&\Big(\mathsf{lam}\, \lambda n.\mathsf{ifZero}\, n\, \mathsf{zero}\, \Big(\mathsf{suc}\, \big(\mathsf{suc}\, (\mathsf{fix}\, \mathsf{double}' \cdot \mathsf{pred}\, n)\big)\Big)\Big) \cdot \mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero}) && = \\
&\mathsf{ifZero}\, (\mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero}))\, \mathsf{zero}\, \Big(\mathsf{suc}\, \big(\mathsf{suc}\, (\mathsf{fix}\, \mathsf{double}' \cdot \mathsf{pred}\, (\mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero})))\big)\Big) && = \\
&\mathsf{suc}\, \big(\mathsf{suc}\, (\mathsf{fix}\, \mathsf{double}' \cdot \mathsf{pred}\, (\mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero})))\big) && = \\
&\mathsf{suc}\, \big(\mathsf{suc}\, (\mathsf{fix}\, \mathsf{double}' \cdot \mathsf{suc}\, \mathsf{zero})\big) && = \\
&\mathsf{suc}\, \big(\mathsf{suc}\, (\mathsf{double}'\, (\mathsf{fix}\, \mathsf{double}') \cdot \mathsf{suc}\, \mathsf{zero})\big) && = \\
&\mathsf{suc}\, \Big(\mathsf{suc}\, \Big(\Big(\mathsf{lam}\, \lambda n.\mathsf{ifZero}\, n\, \mathsf{zero}\, \Big(\mathsf{suc}\, \big(\mathsf{suc}\, (\mathsf{fix}\, \mathsf{double}' \cdot \mathsf{pred}\, n)\big)\Big)\Big) \cdot \mathsf{suc}\, \mathsf{zero}\Big)\Big) && = \\
&\mathsf{suc}\, \Big(\mathsf{suc}\, \Big(\mathsf{ifZero}\, (\mathsf{suc}\, \mathsf{zero})\, \mathsf{zero}\, \Big(\mathsf{suc}\, \big(\mathsf{suc}\, (\mathsf{fix}\, \mathsf{double}' \cdot \mathsf{pred}\, (\mathsf{suc}\, \mathsf{zero}))\big)\Big)\Big)\Big) && =
\end{aligned}$$

$$\begin{aligned}
&\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{fix}\,\mathsf{double}' \cdot \mathsf{pred}\,(\mathsf{suc}\,\mathsf{zero})\right)\right)\right)\right) &&= \\
&\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{fix}\,\mathsf{double}' \cdot \mathsf{zero}\right)\right)\right)\right) &&= \\
&\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{double}'\,(\mathsf{fix}\,\mathsf{double}') \cdot \mathsf{zero}\right)\right)\right)\right) &&= \\
&\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\left(\left(\left(\mathsf{lam}\,\lambda n.\mathsf{ifZero}\,n\,\mathsf{zero}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{fix}\,\mathsf{double}' \cdot \mathsf{pred}\,n\right)\right)\right)\right) \cdot \mathsf{zero}\right)\right)\right)\right) &&= \\
&\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{ifZero}\,\mathsf{zero}\,\mathsf{zero}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{fix}\,\mathsf{double}' \cdot \mathsf{pred}\,\mathsf{zero}\right)\right)\right)\right)\right)\right)\right) &&= \\
&\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\left(\mathsf{suc}\,\mathsf{zero}\right)\right)\right)
\end{aligned}$$

109. Feladat. *Az összeadás, szorzás, hatványozás és az Ackermann-függvény definiálható PCF-ben.*

110. Feladat. *Adjuk meg a természetes számok iterátorát és rekurzorát PCF-ben, és bizonyítsuk be a számítási szabályaikat!*

A System T-ről megmutattuk, hogy ha definiálható a kiértékelő System T-ben, akkor van olyan u, hogy $u = \mathsf{suc}\,u$. Ugyanez a bizonyítás működik PCF-re, is, viszont az eredmény nem meglepő, hiszen

$$\mathsf{fix}\,\mathsf{suc} = \mathsf{suc}\,(\mathsf{fix}\,\mathsf{suc}).$$

Mivel tetszőleges függvénynek van fixpontja, így az identitásnak is, emiatt minden típusnak van eleme:

$$\begin{aligned}
&\mathsf{bottom} : \mathsf{Tm}\,A \\
&\mathsf{bottom} := \mathsf{fix}\,(\lambda a.a)
\end{aligned}$$

111. Feladat. *Mutassuk meg, hogy ha* $\mathsf{zero} = \mathsf{suc}\,\mathsf{zero}$*, akkor minden* A*-ra és* $u, v : \mathsf{Tm}\,A$*-ra* $u = v$*!*

112. Feladat. *Mutassuk meg, hogy a* $\mathsf{lam}\,\lambda x.\mathsf{suc}\,(\mathsf{suc}\,(\mathsf{pred}\,x))$ *term nem definiálja az* $x \mapsto 1 + x$ *függvényt!*

113. Feladat. *Mutassuk meg, hogy PCF-be sekély módon beágyazható a System T-nek egy* $\Rightarrow\eta$ *szabály nélküli változata!*

114. Feladat. *Az alábbi PCF-termekre határozzuk meg, hogy melyik függvényt definiálják!*

$$\begin{aligned}
&\mathsf{fix}\,(\lambda t.\mathsf{lam}\,\lambda x.x) \\
&\mathsf{fix}\,(\lambda t.\mathsf{lam}\,\lambda x.\mathsf{ifZero}\,x\,x\,x) \\
&\mathsf{fix}\,(\lambda t.\mathsf{lam}\,\lambda x.\mathsf{ifZero}\,x\,x\,(\mathsf{suc}\,\mathsf{zero})) \\
&\mathsf{fix}\,(\lambda t.\mathsf{lam}\,\lambda x.\mathsf{ifZero}\,x\,(\mathsf{suc}\,\mathsf{zero})\,\mathsf{zero}) \\
&\mathsf{fix}\left(\lambda t.\mathsf{lam}\,\lambda x.\mathsf{ifZero}\,x\,x\left(\mathsf{suc}\,(t \cdot \mathsf{pred}\,x)\right)\right) \\
&\mathsf{fix}\left(\lambda t.\mathsf{lam}\,\lambda x.\mathsf{ifZero}\,x\,\mathsf{zero}\left(\mathsf{suc}\,(t \cdot \mathsf{pred}\,x)\right)\right) \\
&\mathsf{fix}\left(\lambda t.\mathsf{lam}\,\lambda x.\mathsf{ifZero}\,x\,(\mathsf{suc}\,\mathsf{zero})\left(\mathsf{suc}\,(t \cdot \mathsf{pred}\,x)\right)\right)
\end{aligned}$$

4.4. Véges típusok

115. Definíció (Összeg és szorzat típusok)**.** *Az egyszerű típusos lambda-kalkulust (92. definíció) az alábbi szabályokkal egészítjük ki.*

$$\frac{}{\bot} \qquad \frac{b : \bot}{\mathsf{exfalso}\,b : A} \qquad \frac{x : \bot \vdash a : A}{a = \mathsf{exfalso}\,x}$$

$$\frac{A \qquad B}{A + B} \qquad \frac{a : A}{\mathsf{inl}\,a : A + B} \qquad \frac{b : B}{\mathsf{inr}\,b : A + B} \qquad \frac{t : (A + B) \qquad x : A \vdash c_0 : C \qquad y : B \vdash c_1 : C}{\mathsf{case}\,t\,(x.c_0)\,(y.c_1) : C}$$

$$\frac{}{\mathsf{case}\,(\mathsf{inl}\,a)\,(x.c_0)\,(y.c_1) = c_0[x \mapsto a]} \qquad \frac{}{\mathsf{case}\,(\mathsf{inr}\,b)\,(x.c_0)\,(y.c_1) = c_1[y \mapsto b]}$$

$$\frac{z : A + B \vdash t : C}{t = \mathsf{case}\,z\,(x.t[z \mapsto \mathsf{inl}\,x])\,(y.t[z \mapsto \mathsf{inr}\,y])}$$

$$\frac{}{\top} \qquad \frac{}{\mathsf{tt} : \top} \qquad \frac{t : \top}{t = \mathsf{tt}}$$

$$\frac{A \qquad B}{A \times B} \qquad \frac{a : A \qquad b : B}{(a, b) : A \times B} \qquad \frac{t : A \times B}{\mathsf{fst}\,t : A} \qquad \frac{t : A \times B}{\mathsf{snd}\,t : B} \qquad \frac{}{\mathsf{fst}\,(a, b) = a} \qquad \frac{}{\mathsf{snd}\,(a, b) = b} \qquad \frac{t : A \times B}{t = (\mathsf{fst}\,t, \mathsf{snd}\,t)}$$

116. Feladat. *A fenti összes szabályra adjuk meg, hogy az alábbiak közül melyik csoportba tartozik: konstruktor, destruktor, β η szabály.*

117. Feladat. *Adjuk meg a logikai értékek típusát a következőképp:* $\mathsf{Bool} := \top + \top$*! Mutassuk meg, hogy az alábbi operátorok és egyenlőségek definiálhatóak!*

$$\frac{}{\mathsf{true} : \mathsf{Bool}} \qquad \frac{}{\mathsf{false} : \mathsf{Bool}} \qquad \frac{b : \mathsf{Bool} \qquad a_0 : A \qquad a_1 : A}{\mathsf{ite}\, b\, a_0\, a_1 : A}$$

$$\frac{}{\mathsf{ite}\,\mathsf{true}\, a_0\, a_1 = a_0} \qquad \frac{}{\mathsf{ite}\,\mathsf{false}\, a_0\, a_1 = a_1}$$

$$\frac{x : \mathsf{Bool} \vdash t : A}{t = \mathsf{ite}\, x\, (t[x \mapsto \mathsf{true}])\, (t[x \mapsto \mathsf{false}])}$$

118. Feladat. *Mutassuk meg, hogy a* $\mathsf{lam}\, \lambda x.\mathsf{ite}\, x\, \mathsf{false}\, \mathsf{true}$ *és a* $\mathsf{lam}\, \lambda x.(\mathsf{ite}\, x\, \mathsf{false}\, (\mathsf{ite}\, x\, \mathsf{true}\, \mathsf{true}))$ *termek egyenlők (szükségünk lesz a* Bool *típus* η *szabályára).*

119. Feladat. *Adjunk meg négy* $\mathsf{Bool} \Rightarrow \mathsf{Bool}$ *típusú termet, amelyek közül ha bármelyik kettő egyenlő, akkor* $\mathsf{true} = \mathsf{false}$*!*

120. Feladat. *Mutassuk meg, hogy bármely* $\mathsf{Bool} \Rightarrow \mathsf{Bool}$ *típusú term egyenlő az előző feladatban megadott négy term valamelyikével!*

121. Feladat. *Adjunk meg egy háromelemű típust és hozzá egy esetszétválasztás (case) operátort, ennek segítségével adjuk meg a modulo 3 összeadást.*

122. Feladat. *Adjuk meg* Bool *és szorzat típus segítségével a homogén bináris összeg típust (tehát tetszőleges A-ra az $A + A$ típust)! Adjuk meg a bevezető operátorait, az esetszétválasztás operátorát és vezessük le a β és η szabályokat.*

123. Feladat. *Adjuk meg az opció (option, talán, maybe) típuskonstruktort, mely egy meglevő típushoz ad hozzá egy újabb elemet.*

124. Feladat. *Minden $n : \mathbb{N}$-re adjuk meg a* $\mathsf{Fin}\, n$ *típust, melynek n eleme van. Adjuk meg az eliminációs szabályát.*

125. Feladat. *Adjuk meg minden n-re az n-szeres szorzat és összeg típusokat.*

126. Feladat. *Mutassuk meg, hogy pontosan egy $A \Rightarrow \top$ illetve pontosan egy $\bot \Rightarrow A$ típusú term van minden A-ra.*

127. Definíció (Típus-izomorfizmus)**.** *A és B típusok izomorfak, ha létezik $x : A \vdash t : B$ és $y : B \vdash t' : A$ term, melyekre $t[x \mapsto t'] = y$ és $t'[y \mapsto t] = x$. Ebben az esetben $A \cong B$-t írunk.*

128. Feladat. *Adjunk meg egy izomorfizmust $A \times \top$ és A között minden A-ra.*

129. Feladat. *Mutassuk meg, hogy tetszőleges A, B típusokra $A + B \cong B + A$.*

A következő feladat a típusos változata az algebrai $(a + b)^2 = a^2 + 2ab + b^2$ egyenletnek.

130. Feladat. *Mutassuk meg, hogy tetszőleges A, B típusokra* $\mathsf{Bool} \Rightarrow (A + B) \cong \mathsf{Bool} \Rightarrow A + \mathsf{Bool} \times A \times B + + \mathsf{Bool} \Rightarrow B$.

131. Feladat. *Mutassuk, meg hogy $a \cong$ reláció reflexív, szimmetrikus és tranzitív, valamint kongruencia a $\times$-ra és a $+$-ra vontakozóan! (Tehát például $A \cong A'$-ből következik $A \times B \cong A' \times B$.)*

132. Definíció (Kommutatív exponenciális félgyűrű)**.** *A kommutatív exponenciális félgyűrű (commutative exponential semiring, commutative exponential ring without negation, commutative exponential rig) egy* $\mathsf{C} : \mathsf{Set}$ *halmaz a következő műveletekkel és egyenlőségekkel. Mivel $a +$ és $a *$ operátorok kommutatívak, ezért műveletenként csak egy identitás és egy disztributivitás egyenlőséget listázunk. Az egyenlőségeknek nem adunk nevet.*

$$\begin{aligned}
-+- &: \mathsf{C} \to \mathsf{C} \to \mathsf{C} \\
0 &: \mathsf{C} \\
-*- &: \mathsf{C} \to \mathsf{C} \to \mathsf{C} \\
1 &: \mathsf{C} \\
-^{-} &: \mathsf{C} \to \mathsf{C} \to \mathsf{C} \\
&: (a + b) + c = a + (b + c)
\end{aligned}$$

$$
\begin{aligned}
&: a + b = b + a \\
&: 0 + a = a \\
&: (a * b) * c = a * (b * c) \\
&: a * b = b * a \\
&: 1 * a = a \\
&: a * (b + c) = a * b + a * c \\
&: a^{b+c} = a^b * a^c \\
&: a^0 = 1 \\
&: (a * b)^c = a^c * b^c \\
&: 1^c = 1 \\
&: a^{b*c} = (a^b)^c \\
&: a^1 = a
\end{aligned}
$$

133. Feladat. *A természetes számok kommutatív exponenciális félgyűrűt alkotnak az összeadással, szorzással, hatványozással. A természetes számok kommutatív félgyűrűt alkotnak a maximum művelettel és az összeadással. A természetes számok a végtelennel kiegészítve kommutatív félgyűrűt alkotnak a minimum művelettel és az összeadással.*

134. Feladat. *Az összeg és szorzat típusokkal rendelkező nyelvben a $\cong$ relációval a típusok kommutatív exponenciális semiring-et alkotnak!*

135. Megjegyzés. *A kommutatív exponenciális félgyűrűt Tarski high school algebrájának [BL93] is nevezik (bár kihagyják a 0 műveletet), ezekkel kapcsolatban az érdekes teljességi problémát Wilkie oldotta meg [Wil01]. A probléma függő típusú általánosítása továbbra is nyitott [Alt08].*

136. Feladat. *Adjuk meg a fenti nyelv sekély beágyazását a metanyelvbe!*

4.5. Generikus programozás

Ebben és a következő alfejezetben tárgyalásunk Robert Harper könyvét követi [Har16].

Tegyük fel, hogy van egy $\mathsf{Bool} \Rightarrow \mathsf{Nat}$ függvényünk, például ami true-hoz 1-t, false-hoz 0-t rendel (például $\mathsf{lam}\,\lambda x.\mathsf{ite}\,x\,(\mathsf{suc}\,\mathsf{zero})\,\mathsf{zero}$). Ezt tudjuk alkalmazni egy Bool típusú kifejezésre. De hasonlóképp tudnánk alkalmazni $\mathsf{Bool} \times \mathsf{Nat}$ típusú kifejezésre is, úgy, hogy a Nat rész nem változik, tehát (t, t')-ből $(\mathsf{ite}\,t\,(\mathsf{suc}\,\mathsf{zero})\,\mathsf{zero}, t')$ lesz. Vagy egy $(\mathsf{Nat} + \mathsf{Bool}) \times \mathsf{Bool}$ kifejezésre is, ahol mindkét Bool-ra alkalmaznánk, a Nat típusú értéket meg változatlanul hagynánk. Általánosságban: tetszőleges $A \Rightarrow B$ függvényt szeretnénk kiterjeszteni egy $F : \mathsf{Ty} \to \mathsf{Ty}$ típusoperátorra (egy típustól függő típusra), úgy, hogy egy $F\,A \Rightarrow F\,B$ függvényt kapjunk. A generikus programozás ezt teszi lehetővé bizonyos F típusoperátorok esetén. Ha $F = \lambda X.A$ alakú, ahol A-ban X-en kívül nulláris és bináris szorzat illetve összeg típusok lehetnek, akkor F-et polinomiális típusoperátornak nevezzük. Ha függvény típus is lehet benne, akkor a pozitív típusoperátorok lesznek azok, melyekre a generikus művelet megadható.

A $\lambda x.2x^3+x^2+3x+1$ polinom típusoperátor-megfelelője a $\lambda X.(\top+\top)\times(X\times X\times X)+X\times X+(\top+\top+\top)\times X+\top$.

137. Definíció (Polinomiális típusoperátorokra map)**.** *Az összeg és szorzat típusok nyelvét (115. definíció) feltételezve megadjuk az alábbi induktív családot. A konstruktoroknak nem adunk nevet.*

$$
\begin{aligned}
\mathsf{Poly} &: (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Set} \\
&: \mathsf{Poly}\,(\lambda X.X) \\
&: \mathsf{Poly}\,(\lambda X.\top) \\
&: \mathsf{Poly}\,F \to \mathsf{Poly}\,G \to \mathsf{Poly}\,(\lambda X.F\,X \times G\,X) \\
&: \mathsf{Poly}\,(\lambda X.\bot) \\
&: \mathsf{Poly}\,F \to \mathsf{Poly}\,G \to \mathsf{Poly}\,(\lambda X.F\,X + G\,X)
\end{aligned}
$$

Poly *szerinti rekurzióval megadjuk a generikus* map *operátort.*

$$
\begin{aligned}
&\mathsf{map} : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{Poly}\,F\} \to (\mathsf{Tm}\,A \to \mathsf{Tm}\,B) \to \mathsf{Tm}\,(F\,A) \to \mathsf{Tm}\,(F\,B) \\
&\mathsf{map}\,(\lambda X.X) \qquad\quad f\,a = f\,a \\
&\mathsf{map}\,(\lambda X.\top) \qquad\quad f\,t = \mathsf{tt} \\
&\mathsf{map}\,(\lambda X.F\,X \times G\,X)\ f\,w = \big(\mathsf{map}\,F\,f\,(\mathsf{fst}\,w), \mathsf{map}\,G\,f\,(\mathsf{snd}\,w)\big)
\end{aligned}
$$

$$\begin{aligned}
&\mathsf{map}\ (\lambda X.\bot) && f\ t &&= \mathsf{exfalso}\ t\\
&\mathsf{map}\ (\lambda X.F\ X + G\ X)\ f\ w &&&&= \mathsf{case}\ w\ \big(\lambda u.\mathsf{inl}\ (\mathsf{map}\ F\ f\ u)\big)\ \big(\lambda v.\mathsf{inr}\ (\mathsf{map}\ G\ f\ v)\big)
\end{aligned}$$

138. Feladat. *Lássuk be az alábbi típusoperátorról hogy polinomiális.*

$$\lambda X.\bot + (\top \times X + X \times X)$$

139. Feladat. *Mutassuk meg, hogy* $\mathsf{map}\ (\lambda X.\top + (\mathsf{Bool} \times X))\ f\ (\mathsf{inr}\ (\mathsf{true}, t)) = \mathsf{inr}\ (\mathsf{true}, f\ t)$.

140. Feladat. *Mutassuk meg, hogy ha* $A : \mathsf{Ty}$*, akkor* $\mathsf{map}\ (\lambda X.A)\ f\ w = w$.

141. Feladat. *A* map *függvény funktor, tehát* $\mathsf{map}\ F\ (\lambda x.f\ (g\ x))\ w = \mathsf{map}\ F\ f\ (\mathsf{map}\ F\ g\ w)$ *illetve* $\mathsf{map}\ F\ (\lambda x.x)\ u = u$.

Láttuk, hogy a polinomiális típusoperátorokra mindig megadható a map függvény. Ha függvény típust is engedünk a típusoperátorban, akkor ez már nem mindig igaz. Például a következő típusoperátorokra nincs map függvény:

- $F\ X = X \Rightarrow \bot$,
- $F\ X = X \Rightarrow X$,

míg a következőkre van:

- $F\ X = \mathsf{Bool} \Rightarrow X$,
- $F\ X = ((X + X) \Rightarrow \mathsf{Bool}) \Rightarrow X$.

A következő nyelvben bevezetjük a pozitív és negatív típusoperátorok fogalmát, a pozitívokra van map operátor, a negatívokra van egy kontravariáns map operátor. Pozitív és negatív helyett szokás kovariánsat és kontravariánsat mondani.

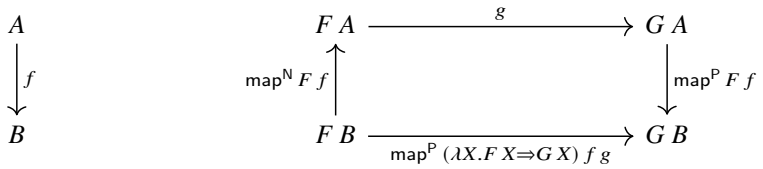
142. Definíció (Pozitív típusoperátorokra map)**.** *Az összeg és szorzat típusok nyelvét (115. definíció) feltételezve megadjuk az alábbi két, kölcsönen megadott induktív családot. A konstruktoroknak nem adunk nevet.*

$$\begin{aligned}
\mathsf{Pos}\ &: (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Set}\\
\mathsf{Neg}\ &: (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Set}\\
&\quad : \mathsf{Pos}\ (\lambda X.X)\\
&\quad : (C : \mathsf{Ty}) \to \mathsf{Pos}\ (\lambda_.C)\\
&\quad : (C : \mathsf{Ty}) \to \mathsf{Neg}\ (\lambda_.C)\\
&\quad : \mathsf{Pos}\ F \to \mathsf{Pos}\ G \to \mathsf{Pos}\ (\lambda X.F\ X + G\ X)\\
&\quad : \mathsf{Neg}\ F \to \mathsf{Neg}\ G \to \mathsf{Neg}\ (\lambda X.F\ X + G\ X)\\
&\quad : \mathsf{Pos}\ F \to \mathsf{Pos}\ G \to \mathsf{Pos}\ (\lambda X.F\ X \times G\ X)\\
&\quad : \mathsf{Neg}\ F \to \mathsf{Neg}\ G \to \mathsf{Neg}\ (\lambda X.F\ X \times G\ X)\\
&\quad : \mathsf{Neg}\ F \to \mathsf{Pos}\ G \to \mathsf{Pos}\ (\lambda X.F\ X \Rightarrow G\ X)\\
&\quad : \mathsf{Pos}\ F \to \mathsf{Neg}\ G \to \mathsf{Neg}\ (\lambda X.F\ X \Rightarrow G\ X)
\end{aligned}$$

A fenti két család szerinti kölcsönös rekurzióval megadjuk az alábbi két map függvényt.

$$\begin{aligned}
&\mathsf{map}^{\mathsf{P}} : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{Pos}\ F\} \to (\mathsf{Tm}\ A \to \mathsf{Tm}\ B) \to \mathsf{Tm}\ (F\ A) \to \mathsf{Tm}\ (F\ B)\\
&\mathsf{map}^{\mathsf{N}} : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{Neg}\ F\} \to (\mathsf{Tm}\ A \to \mathsf{Tm}\ B) \to \mathsf{Tm}\ (F\ B) \to \mathsf{Tm}\ (F\ A)\\
&\mathsf{map}^{\mathsf{P}}\ (\lambda X.X) && f\ a &&= f\ a\\
&\mathsf{map}^{\mathsf{P}}\ (\lambda X.C) && f\ c &&= c\\
&\mathsf{map}^{\mathsf{N}}\ (\lambda X.C) && f\ c &&= c\\
&\mathsf{map}^{\mathsf{P}}\ (\lambda X.F\ X + G\ X) && f\ w &&= \mathsf{case}\ w\ \big(\lambda u.\mathsf{inl}\ (\mathsf{map}^{\mathsf{P}}\ F\ f\ u)\big)\ \big(\lambda v.\mathsf{inr}\ (\mathsf{map}^{\mathsf{P}}\ G\ f\ v)\big)\\
&\mathsf{map}^{\mathsf{N}}\ (\lambda X.F\ X + G\ X) && f\ w &&= \mathsf{case}\ w\ \big(\lambda u.\mathsf{inl}\ (\mathsf{map}^{\mathsf{N}}\ F\ f\ u)\big)\ \big(\lambda v.\mathsf{inr}\ (\mathsf{map}^{\mathsf{N}}\ G\ f\ v)\big)\\
&\mathsf{map}^{\mathsf{P}}\ (\lambda X.F\ X \times G\ X) && f\ w &&= \big(\mathsf{map}^{\mathsf{P}}\ F\ f\ (\mathsf{fst}\ w), \mathsf{map}^{\mathsf{P}}\ G\ f\ (\mathsf{snd}\ w)\big)\\
&\mathsf{map}^{\mathsf{N}}\ (\lambda X.F\ X \times G\ X) && f\ w &&= \big(\mathsf{map}^{\mathsf{N}}\ F\ f\ (\mathsf{fst}\ w), \mathsf{map}^{\mathsf{N}}\ G\ f\ (\mathsf{snd}\ w)\big)\\
&\mathsf{map}^{\mathsf{P}}\ (\lambda X.F\ X \Rightarrow G\ X) && f\ g &&= \mathsf{lam}\ \lambda x.\mathsf{map}^{\mathsf{P}}\ G\ f\ \big(g \cdot (\mathsf{map}^{\mathsf{N}}\ F\ f\ x)\big)\\
&\mathsf{map}^{\mathsf{N}}\ (\lambda X.F\ X \Rightarrow G\ X) && f\ g &&= \mathsf{lam}\ \lambda x.\mathsf{map}^{\mathsf{N}}\ G\ f\ \big(g \cdot (\mathsf{map}^{\mathsf{P}}\ F\ f\ x)\big)
\end{aligned}$$

A $\mathsf{map}^{\mathsf{P}}$ függvény $\Rightarrow$ esetét a következő diagram illusztrálja.



143. Feladat. *Lássuk be a következő típusoperátorokról, hogy pozitívak, negatívak vagy mindkettő.*

- $FX = X \Rightarrow \mathsf{Bool} \Rightarrow \mathsf{Bool}$
- $FX = (X \Rightarrow \mathsf{Bool}) \Rightarrow X$
- $FX = ((X \Rightarrow \mathsf{Bool}) \Rightarrow \mathsf{Bool}) \Rightarrow \mathsf{Bool}$
- $FX = (\mathsf{Bool} \Rightarrow \mathsf{Bool}) \Rightarrow X + X$
- $FX = (X \times (X + \top)) \Rightarrow \mathsf{Bool}$
- $FX = (\mathsf{Bool} \Rightarrow \mathit{Bool}) \Rightarrow \top$

144. Feladat. *Mutassuk meg, hogy ha az $F\,X = X \Rightarrow \bot$ típusoperátorra van $\mathsf{map}^{\mathsf{P}}$ függvény, akkor van $\mathsf{Tm}\,\bot$-nak eleme.*

145. Gondolkodnivaló. *A következő típusoperátorok se nem pozitívak, se nem negatívak. Milyen jellegű ellentmondást kapunk, ha mégis van rájuk $\mathsf{map}^{\mathsf{P}}$ vagy $\mathsf{map}^{\mathsf{N}}$ függvény?*

- $FX = X \Rightarrow X$
- $FX = (\mathsf{Bool} \Rightarrow X) \Rightarrow X$

146. Feladat. *A $\mathsf{map}^{\mathsf{P}}$ függvény kovariáns funktor, míg a $\mathsf{map}^{\mathsf{N}}$ kontravariáns funktor, tehát:*

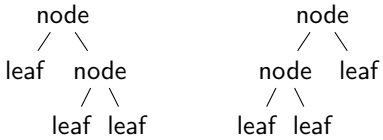
$$\mathsf{map}^{\mathsf{P}}\ F\ (\lambda x.f\ (g\ x))\ w = \mathsf{map}^{\mathsf{P}}\ F\ f\ (\mathsf{map}^{\mathsf{P}}\ F\ g\ w) \qquad \mathsf{map}^{\mathsf{P}}\ F\ (\lambda x.x)\ u = u$$
$$\mathsf{map}^{\mathsf{N}}\ F\ (\lambda x.f\ (g\ x))\ w = \mathsf{map}^{\mathsf{N}}\ F\ g\ (\mathsf{map}^{\mathsf{N}}\ F\ f\ w) \qquad \mathsf{map}^{\mathsf{N}}\ F\ (\lambda x.x)\ u = u$$

4.6. Induktív típusok

Az eddig tárgyalt típusok közül a Bool és a System T-beli Nat induktív típusok voltak. Egy további példa a bináris fák alábbi Tree típusa, melynek két konstruktora van:

$$\begin{aligned}&\mathsf{Tree} &&: \mathsf{Ty}\\ &\mathsf{leaf} &&: \mathsf{Tm}\ \mathsf{Tree}\\ &\mathsf{node} &&: \mathsf{Tm}\ \mathsf{Tree} \to \mathsf{Tm}\ \mathsf{Tree} \to \mathsf{Tm}\ \mathsf{Tree}\end{aligned}$$

A Tree-ben levő fáknak kétfelé ágazó csomópontjai vannak, és sem a csomópontoknál, sem a leveleknél nem tartalmaznak plusz információt. Például a node leaf (node leaf leaf) és a node (node leaf leaf) leaf fák az alábbiak.



A Tree típusra egy algebra a következő komponensekből áll:

$$A : \mathsf{Ty} \qquad u : \mathsf{Tm}\ A \qquad v : \mathsf{Tm}\ A \to \mathsf{Tm}\ A \to \mathsf{Tm}\ A,$$

egy $(\mathsf{Tree}, \mathsf{leaf}, \mathsf{node})$-ból (A, u, v)-be menő homomorfizmus egy Tree-beli termekből A-beli termekbe menő függvény, mely megtartja az algebra műveleteit:

$$\begin{aligned}&t : \mathsf{Tm}\ \mathsf{Tree} \to \mathsf{Tm}\ A\\ &t\ \mathsf{leaf} = u\\ &t\ (\mathsf{node}\ l\ r) = v\ (t\ l)\ (t\ r)\end{aligned}$$

A Tree típus iterátora tetszőleges (A, u, v)-re megad egy ilyen t-t. A homomorfizmus két egyenlőségét nevezzük számítási (β) szabályoknak. A Tree típus egyediségi szabálya azt mondja ki, hogy minden más $(\mathsf{Tree}, \mathsf{leaf}, \mathsf{node})$-ból (A, u, v)-be menő homomorfizmus egyenlő az iterátorral.

A Tree két konstruktorát egyesíteni tudjuk egy konstruktorrá az alábbi izomorfizmusok mentén:

$$\begin{aligned}
&\mathsf{Tm}\,\mathsf{Tree} \times (\mathsf{Tm}\,\mathsf{Tree} \to \mathsf{Tm}\,\mathsf{Tree} \to \mathsf{Tm}\,\mathsf{Tree}) && \cong \\
&(\mathsf{Tm}\,\top \to \mathsf{Tm}\,\mathsf{Tree}) \times (\mathsf{Tm}\,\mathsf{Tree} \times \mathsf{Tm}\,\mathsf{Tree} \to \mathsf{Tm}\,\mathsf{Tree}) && \cong \\
&\mathsf{Tm}\,(\top \Rightarrow \mathsf{Tree}) \times \mathsf{Tm}\,\big((\mathsf{Tree} \times \mathsf{Tree}) \Rightarrow \mathsf{Tree}\big) && \cong \\
&\mathsf{Tm}\,\Big((\top \Rightarrow \mathsf{Tree}) \times \big((\mathsf{Tree} \times \mathsf{Tree}) \Rightarrow \mathsf{Tree}\big)\Big) && \cong \\
&\mathsf{Tm}\,\big((\top + \mathsf{Tree} \times \mathsf{Tree}) \Rightarrow \mathsf{Tree}\big) && \cong \\
&\mathsf{Tm}\,(\top + \mathsf{Tree} \times \mathsf{Tree}) \to \mathsf{Tm}\,\mathsf{Tree} && = \\
&\mathsf{Tm}\,\big((\lambda X.\top + X \times X)\,\mathsf{Tree}\big) \to \mathsf{Tm}\,\mathsf{Tree}
\end{aligned}$$

147. Definíció (Típusoperátor algebrája)**.** *Legyen* $F : \mathsf{Ty} \to \mathsf{Ty}$*. Egy* F*-algebra egy* $A : \mathsf{Ty}$ *típusból és egy* $\mathsf{Tm}\,(F\,A) \to \mathsf{Tm}\,A$ *formájú műveletből áll.*

Egy induktív típus egy F-algebrából (ez az induktív típus maga és a konstruktorok) és az iterátorból áll. Az iterátor a következőt fejezi ki: tetszőleges F-algebrába az induktív típusból pontosan egy F-homomorfizmus megy. Emiatt nevezik az induktív típust az iniciális F-algebrának. Az F-homomorfizmus leírásához már szükségünk van arra, hogy F legalább pozitív legyen. Mi még ennél is szigorúbbak vagyunk, és csak a szigorúan pozitív (strictly positive) F-ekre szorítkozunk, amely megengedőbb kategória, mint a polinomiális, de szigorúbb, mint a pozitív: függvény bal oldalán nem szerepelhet változó.

A Tree típust tehát a $\lambda X.\top + X \times X$ típusoperátor határozza meg, a konstruktora

$$\mathsf{con} : \mathsf{Tm}\,(\top + \mathsf{Tree} \times \mathsf{Tree}) \to \mathsf{Tm}\,\mathsf{Tree}$$

módon elkódolható, míg az iterátora a következő lesz.

$$\begin{aligned}
&\mathsf{ite} : \big(\mathsf{Tm}\,(\top + A \times A) \to \mathsf{Tm}\,A\big) \to \mathsf{Tm}\,\mathsf{Tree} \to \mathsf{Tm}\,A \\
&\mathsf{ite}\,f\,(\mathsf{con}\,w) = \mathsf{case}\,w\,(\lambda_.f\,(\mathsf{inl}\,\mathsf{tt}))\,\Big(\lambda u.f\,\Big(\mathsf{inr}\,\big(\mathsf{ite}\,f\,(\mathsf{fst}\,u), \mathsf{ite}\,f\,(\mathsf{snd}\,u)\big)\Big)\Big)
\end{aligned}$$

A következő nyelvben minden F szigorúan pozitív típusoperátorhoz tartozik egy induktív típus. A homomorfizmus egyenlőségét a map segítségével mondjuk ki.

148. Definíció (Induktív típusok)**.** *Az összeg és szorzat típusok nyelvét (115. definíció) feltételezve megadjuk az alábbi induktív családot. A konstruktoroknak nem adunk nevet.*

$$\begin{aligned}
\mathsf{SPos} &: (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Set} \\
&: \mathsf{SPos}\,(\lambda X.X) \\
&: (A : \mathsf{Ty}) \to \mathsf{SPos}\,(\lambda_.A) \\
&: \mathsf{SPos}\,F \to \mathsf{SPos}\,G \to \mathsf{SPos}\,(\lambda X.F\,X + G\,X) \\
&: \mathsf{SPos}\,F \to \mathsf{SPos}\,G \to \mathsf{SPos}\,(\lambda X.F\,X \times G\,X) \\
&: (C : \mathsf{Ty}) \to \mathsf{SPos}\,F \to \mathsf{SPos}\,(\lambda X.C \Rightarrow F\,X)
\end{aligned}$$

SPos *szerinti rekurzióval megadjuk a generikus* map *operátort:*

$$\begin{aligned}
&\mathsf{map} : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos}\,F\} \to (f : \mathsf{Tm}\,A \to \mathsf{Tm}\,B) \to \mathsf{Tm}\,(F\,A) \to \mathsf{Tm}\,(F\,B) \\
&\mathsf{map}\,(\lambda X.X) \qquad\quad f\,a = f\,a \\
&\mathsf{map}\,(\lambda_.C) \qquad\quad f\,c = c \\
&\mathsf{map}\,(\lambda X.F\,X + G\,X)\,f\,w = \mathsf{case}\,w\,(\lambda u.\mathsf{inl}\,(\mathsf{map}\,F\,f\,u))\,(\lambda v.\mathsf{inr}\,(\mathsf{map}\,G\,f\,v)) \\
&\mathsf{map}\,(\lambda X.F\,X \times G\,X)\,f\,w = \big(\mathsf{map}\,F\,f\,(\mathsf{fst}\,w), \mathsf{map}\,G\,f\,(\mathsf{snd}\,w)\big) \\
&\mathsf{map}\,(\lambda X.C \Rightarrow F\,X) \quad f\,g = \mathsf{lam}\,\lambda a.\mathsf{map}\,F\,f\,(g \cdot a)
\end{aligned}$$

Az összeg és szorzat típusok nyelvét kiegészítjük a következőkkel:

$$\begin{aligned}
&\mathsf{Ind} \quad : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos}\,\mathsf{F}\} \to \mathsf{Ty} \\
&\mathsf{con} \quad : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos}\,\mathsf{F}\} \to \mathsf{Tm}\,\big(F\,(\mathsf{Ind}\,F)\big) \to \mathsf{Tm}\,(\mathsf{Ind}\,F) \\
&\mathsf{ite} \quad : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos}\,\mathsf{F}\} \to \big(\mathsf{Tm}\,(F\,A) \to \mathsf{Tm}\,A\big) \to \mathsf{Tm}\,(\mathsf{Ind}\,F) \to \mathsf{Tm}\,A \\
&\mathsf{Ind}\beta : \mathsf{ite}\,F\,f\,(\mathsf{con}\,F\,x) = f\,\big(\mathsf{map}\,F\,(\mathsf{ite}\,F\,f)\,x\big) \\
&\mathsf{Ind}\eta : (z : \mathsf{Tm}\,(\mathsf{Ind}\,F) \to \mathsf{Tm}\,A) \to \Big(\big(\lambda x.z\,(\mathsf{con}\,F\,x)\big) = \big(\lambda x.f(\mathsf{map}\,F\,z\,x)\big)\Big) \to z = \mathsf{ite}\,F\,f
\end{aligned}$$

Az induktív típusok leírásából gyakran kihagyják az $\mathsf{Ind}\eta$ szabályt, ennek két oka van:

– Ez nem egy számítási szabály, tehát a programok futtatásánál nem szükséges alkalmaznunk.

– Az $\mathsf{Ind}\eta$ szabállyal rendelkező nyelvben az egyenlőség eldönthetetlen (ez függő típusok esetén (lásd a 184. definíció) problémát okoz, mert típusellenőrzéshez szükséges a típusok egyenlőségének eldönthetősége).

Például az 50. és a 93. definíciókban megadott nyelvekben a Bool és Nat típusok nem rendelkeztek η szabállyal.

Az induktív típusok a véges mélységű fák: ez azt jelenti, hogy olyan fák, melyek esetleg végtelen sok felé ágaznak, de ha bármely ágon elindulunk, akkor véges sok lépésen belül levélhez érünk. Szemléletesen: az, hogy az induktív típus eleme véges mélységű, még nem jelenti azt, hogy az adott fa beleférne egy véges mélységű gödröbe: lehet tetszőlegesen hosszú ága, de minden ága véges.

149. Feladat. *Az* $\mathsf{Ind}\,(\lambda X.\top + (\mathsf{Nat} \Rightarrow X))$ *induktív típusban végtelen-felé ágazó fák vannak. Adjunk meg egy olyan fát, melynek* i*. részfájának magassága* i*.*

150. Feladat. *A természetes számokat a* $\lambda X.\top + X$ *típusfüggvény segítségével a* $\mathsf{Nat} := \mathsf{Ind}\,(\lambda X.\top + X)$ *típussal adjuk meg. Adjuk meg ehhez a* zero *és* suc *konstruktorokat, valamint az iterátort és mutassuk meg, hogy a számítási szabályok teljesülnek!*

151. Feladat. *Kódoljuk el a következő induktív típusokat: természetes számok listái, bináris fák, háromfelé ágazó fák, típus nélküli kombinátor kalkulus (egyenlőségek nélkül), Razor nyelv (egyenlőségek és típusok nélkül).*

152. Feladat. *Az* $\mathsf{Ind}\,F$ *típus az* F *operátor fixpontja: mutassuk meg, hogy* $F\,(\mathsf{Ind}\,F) \cong \mathsf{Ind}\,F$*.*

153. Feladat. *Mutassuk meg, hogy minden* A*-ra* $\mathsf{Ind}\,(\lambda X.A) \cong A$*.*

A Nat rekurzora megadható az iterátor segítségével: az ötlet, hogy az iterátor nem csak A-t, hanem a konkrét számot is visszaadja, amire meghívtuk. Emlékeztetőként listázzuk az iterátor típusát és egyenlőség-szabályait.

$$
\begin{aligned}
&\mathsf{ite} &&: \{A : \mathsf{Ty}\} \to \mathsf{Tm}\,A \to (\mathsf{Tm}\,A \to \mathsf{Tm}\,A) \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A\\
&\mathsf{ite}\beta_1 &&: \mathsf{ite}\,z\,s\,\mathsf{zero} = z\\
&\mathsf{ite}\beta_2 &&: \mathsf{ite}\,z\,s\,(\mathsf{suc}\,n) = s\,(\mathsf{ite}\,z\,s\,n)\\
&\mathsf{ite}\eta &&: (z : \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A) \to (z\,\mathsf{zero} = z) \to ((n : \mathsf{Tm}\,\mathsf{Nat}) \to z\,(\mathsf{suc}\,n) = s\,(z\,n)) \to z = \mathsf{ite}\,z\,s\\
&\mathsf{rec} &&: \{A : \mathsf{Ty}\} \to \mathsf{Tm}\,A \to (\mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A) \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A\\
&\mathsf{rec}\,\{A\}\,z\,s\,n := \mathsf{snd}\left(\mathsf{ite}\{\mathsf{Nat} \times A\}\,(\mathsf{zero}, z)\,\left(\lambda w.(\mathsf{suc}\,(\mathsf{fst}\,w), s\,(\mathsf{fst}\,w)\,(\mathsf{snd}\,w))\right)\,n\right)
\end{aligned}
$$

154. Feladat. *Bizonyítsuk be a rekurzor* β *szabályait:*

$$
\begin{aligned}
&\mathsf{rec}\beta_1 : \mathsf{rec}\,z\,s\,\mathsf{zero} = z\\
&\mathsf{rec}\beta_2 : \mathsf{rec}\,z\,s\,(\mathsf{suc}\,n) = s\,n\,(\mathsf{rec}\,z\,s\,n)
\end{aligned}
$$

Utóbbihoz szükségünk van az iterátor η *szabályára is (enélkül* $\mathsf{rec}\beta_2$ *nem vezethető le [Par89]).*

155. Gondolkodnivaló. *A rekurzornak mondjuk ki az* η *szabályát. Levezethető az iterátorból?*

156. Gondolkodnivaló. *Adjuk meg általános induktív típusra a rekurzor fogalmát, és mutassuk meg, hogy az iterátor segítségével definiálható.*

Ha csak polinomiális típusoperátorokra korlátozzuk az induktív típusokat, csak végesen elágazódó fákat kapunk.

157. Gondolkodnivaló. *Minden* F *polinomiális típusoperátorra* $\mathsf{Ind}\,F$ *vagy izomorf egy véges típussal, vagy izomorf a természetes számokkal.*

158. Feladat (Cantor-féle átlós metszés)**.** *Mutassuk meg, hogy ha* $(\mathsf{Nat} \Rightarrow \mathsf{Bool}) \cong \mathsf{Nat}$*, akkor* $\mathsf{true} = \mathsf{false}$*.*

4.7. Koinduktív típusok

A koinduktív típusok az induktív típusok duálisai.

– Az induktív típusokat a konstruktorok, a koinduktív típusokat a destruktorok határozzák meg.

– Míg az induktív típusokból iterátorokkal megadott függvények mindig *terminálnak*, a koinduktív típusokat létrehozó generátorok mindig *produktívak*.

– Induktív típusokkal adjuk meg a véges mélységű fákat, koinduktív típusokkal a potenciálisan végtelen mélységű fákat.

Koinduktív típusok elemei a végtelen sokáig futó programok (például szerverek, operációs rendszerek, interaktív programok), automaták, Turing-gépek és egyéb gépek. Az objektumorientált programozás egy (részleges) modellezése: az osztály egy koinduktív típus, a tagfüggvények a destruktorok, az objektumok a kointuktív típus elemei. Turner [Tur95] már a 90-es években javasolta az erős funkcionális programozási stílust, melyben elkülönítjük az induktív és koinduktív típusokat, és nincs általános rekurzió. Ezt a gyakorlatban azóta sem próbálta ki senki.

A legelső koinduktív típus a végtelen listák (folyam, stream) típusa.

$$
\begin{array}{ll}
\mathsf{Stream} & : \mathsf{Ty} \to \mathsf{Ty} \\
\mathsf{head} & : \mathsf{Tm}\,(\mathsf{Stream}\,A) \to \mathsf{Tm}\,A \\
\mathsf{tail} & : \mathsf{Tm}\,(\mathsf{Stream}\,A) \to \mathsf{Tm}\,(\mathsf{Stream}\,A) \\
\mathsf{gen} & : (\mathsf{Tm}\,C \to \mathsf{Tm}\,A) \to (\mathsf{Tm}\,C \to \mathsf{Tm}\,C) \to \mathsf{Tm}\,C \to \mathsf{Tm}\,(\mathsf{Stream}\,A) \\
\mathsf{Stream}\beta_1 & : \mathsf{head}\,(\mathsf{gen}\,h\,t\,c) = h\,c \\
\mathsf{Stream}\beta_2 & : \mathsf{tail}\,(\mathsf{gen}\,h\,t\,c) = \mathsf{gen}\,h\,t\,(t\,c)
\end{array}
$$

A folyamoknak két destruktora van: fej (head) és farok (tail). A tail-t akárhányszor alkalmazhatjuk a folyamra, így tényleg végtelen sok elemet kapunk egy $u : \mathsf{Tm}\,(\mathsf{Stream}\,A)$-ból:

$$\mathsf{head}\,u,\quad \mathsf{head}\,(\mathsf{tail}\,u),\quad \mathsf{head}\,(\mathsf{tail}\,(\mathsf{tail}\,u)),\quad \mathsf{head}\,(\mathsf{tail}\,(\mathsf{tail}\,(\mathsf{tail}\,u))),\quad \ldots$$

Folyamokat a generátorral (koiterátor) tudunk létrehozni: ez egy mag-típusból (C, seed), egy h és t függvényből, valamint egy magból (a mag-típus eleméből) növeszti ki a folyamot. A h megmondja, hogy a magból hogyan tudjuk kiszámítani az aktuális fej-elemet, míg a t azt mondja el, hogy hogyan kell léptetni a magot: ezt fejezik ki a β szabályok. A természetes számokat tartalmazó $0, 1, 2, 3, \ldots$ folyam például a következőképp adható meg, ahol $C := \mathsf{Nat}$:

$$
\begin{array}{l}
\mathsf{nats} : \mathsf{Tm}\,(\mathsf{Stream}\,\mathsf{Nat}) \\
\mathsf{nats} := \mathsf{gen}\,(\lambda n.n)\,\mathsf{suc}\,\mathsf{zero}
\end{array}
$$

Valóban,

$$
\begin{array}{ll}
\mathsf{head}\,\mathsf{nats} & = \\
\mathsf{head}\,(\mathsf{gen}\,(\lambda n.n)\,\mathsf{suc}\,\mathsf{zero}) & =(\mathsf{Stream}\beta_1) \\
(\lambda n.n)\,\mathsf{zero} & = \\
\mathsf{zero}, & \\
 & \\
\mathsf{head}\,(\mathsf{tail}\,\mathsf{nats}) & = \\
\mathsf{head}\,(\mathsf{tail}\,(\mathsf{gen}\,(\lambda n.n)\,\mathsf{suc}\,\mathsf{zero})) & =(\mathsf{Stream}\beta_2) \\
\mathsf{head}\,(\mathsf{gen}\,(\lambda n.n)\,\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero})) & =(\mathsf{Stream}\beta_1) \\
(\lambda n.n)\,(\mathsf{suc}\,\mathsf{zero}) & = \\
\mathsf{suc}\,\mathsf{zero}, & \\
 & \\
\mathsf{head}\,(\mathsf{tail}\,(\mathsf{tail}\,\mathsf{nats})) & = \\
\mathsf{head}\,(\mathsf{tail}\,(\mathsf{tail}\,(\mathsf{gen}\,(\lambda n.n)\,\mathsf{suc}\,\mathsf{zero}))) & =(\mathsf{Stream}\beta_2) \\
\mathsf{head}\,(\mathsf{tail}\,(\mathsf{gen}\,(\lambda n.n)\,\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero}))) & =(\mathsf{Stream}\beta_2) \\
\mathsf{head}\,(\mathsf{gen}\,(\lambda n.n)\,\mathsf{suc}\,(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero}))) & =(\mathsf{Stream}\beta_1) \\
(\lambda n.n)\,(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero})) & = \\
\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero}), &
\end{array}
$$

és így tovább.

159. Feladat. *Adjuk meg* Stream Nat*-on a pontonkénti összeadást és a* map *függvényt.*

Nem lehet folyamokon megadni a filter függvényt, mert nem produktív.

Általánosságban minden szigorúan pozitív típusfüggvény meghatároz egy koinduktív típust. A következőkben kiegészítjük az induktív típusok nyelvét koinduktív típusokkal (igazából az induktív típusokra nincs szükségünk, csak SPos-ra és map-re).

160. Definíció ((Ko)induktív típusok)**.** *Az induktív típusok nyelvét (148. definíció) kiegészítjük az alábbiakkal.*

$$\begin{aligned}
&\mathsf{Coind} && : \mathsf{SPos\,F} \to \mathsf{Ty}\\
&\mathsf{des} && : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos\,F}\} \to \mathsf{Tm}\,(\mathsf{Coind}\,F) \to \mathsf{Tm}\,\big(F\,(\mathsf{Coind}\,F)\big)\\
&\mathsf{gen} && : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos\,F}\} \to \big(\mathsf{Tm}\,A \to \mathsf{Tm}\,(F\,A)\big) \to \mathsf{Tm}\,A \to \mathsf{Tm}\,(\mathsf{Coind}\,F)\\
&\mathsf{Coind}\beta && : \mathsf{des}\,F\,(\mathsf{gen}\,F\,f\,a) = \mathsf{map}\,F\,(\mathsf{gen}\,F\,f)\,(f\,a)\\
&\mathsf{Coind}\eta && : (z : \mathsf{Tm}\,A \to \mathsf{Tm}\,(\mathsf{Coind}\,F)) \to \Big(\big(\lambda z.\mathsf{des}\,F\,(z\,a)\big) = \big(\lambda z.\mathsf{map}\,F\,z\,(f\,a)\big)\Big) \to z = \mathsf{gen}\,F\,f
\end{aligned}$$

161. Feladat. *Mutassuk meg, hogy a folyamok fent leírt destruktorai és generátora megadható a* $\mathsf{Stream}\,A := \mathsf{Coind}\,(\lambda X.A \times X)$ *definícióval.*

162. Feladat. *A ko-természetes számokon (*$\mathsf{Coind}\,(\lambda X.\top + X)$*) adjuk meg az összeadást, szorzást!*

163. Feladat. *Melyik induktív típus a folyamok duálisa?*

164. Feladat. *Adjuk meg az olyan gépek koinduktív típusát, melyekbe be lehet írni egy (természetes) számot, ki lehet íratni velük egy számot, és meg lehet nyomni rajtuk egy gombot! Ennek a típusnak az elemeként készítsünk egy összeadó-gépet, mely a beírt számokat összeadja, az összeget kiírja, és a gomb megnyomásával nullázza az összeget.*

165. Feladat. *A* $\mathsf{Coind}\,F$ *típus az* F *operátor fixpontja: mutassuk meg, hogy* $F\,(\mathsf{Coind}\,F) \cong \mathsf{Coind}\,F$*.*

166. Feladat. *Mutassuk meg, hogy minden* A*-ra* $\mathsf{Coind}\,(\lambda X.A) \cong A$*.*

167. Gondolkodnivaló. *Az iterátor duálisa a generátor. Mi a rekurzor duálisa?*

4.8. Polimorfizmus

Az eddigi nyelveinkben külön-külön kell megadnunk a $(B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C$ függvénykompozíciót minden A, B és C típusra, és külön kell megadnunk a $\mathsf{filter} : \mathsf{Tm}\,((A \Rightarrow \mathsf{Bool}) \Rightarrow \mathsf{List}\,A \Rightarrow \mathsf{List}\,A)$ függvényt minden A típusra. Ahhoz, hogy egyszerre megadhassuk ezeket a termeket az összes lehetséges típusra, bevezetjük a polimorf típusokat. Az általános függvénykompozíció típusa például a következő lesz: $\forall\lambda A.\forall\lambda B.\forall\lambda C.(B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C$, a filter típusa $\forall\lambda A.(A \Rightarrow \mathsf{Bool}) \Rightarrow \mathsf{List}\,A \Rightarrow \mathsf{List}\,A$.

A következő típusrendszerben megkülönböztetjük a monomorf (MTy) és a polimorf típusokat (Ty), és az univerzális kvantor csak egy polimorf típus elején szerepelhet. Az alaptípusok (például Nat) monomorfak.

168. Definíció (Hindley–Milner-típusrendszer)**.**

$$\begin{aligned}
&\mathsf{MTy} && : \mathsf{Set}\\
&\mathsf{Ty} && : \mathsf{Set}\\
&\mathsf{Tm} && : \mathsf{Ty} \to \mathsf{Set}\\
&- \Rightarrow - && : \mathsf{MTy} \to \mathsf{MTy} \to \mathsf{MTy}\\
&\forall && : (\mathsf{MTy} \to \mathsf{Ty}) \to \mathsf{Ty}\\
&\mathsf{i} && : \mathsf{MTy} \to \mathsf{Ty}\\
&\mathsf{lam} && : \big(\mathsf{Tm}\,(\mathsf{i}\,A) \to \mathsf{Tm}\,(\mathsf{i}\,B)\big) \cong \mathsf{Tm}\,(\mathsf{i}\,(A \Rightarrow B)) : - \cdot -\\
&\mathsf{Lam} && : \big((A : \mathsf{MTy}) \to \mathsf{Tm}\,(B\,A)\big) \cong \mathsf{Tm}\,(\forall\,B) : - \bullet -
\end{aligned}$$

A fenti nyelvben már nem csak termváltozók, hanem típusváltozók is vannak: a $\forall$ és a Lam operátorok nem termváltozót, hanem típusváltozót kötnek.

Például az identitás függvényt implementáló term a következő.

$$\begin{aligned}
&\mathsf{id} : \mathsf{Tm}\,\big(\forall\,\lambda A.\mathsf{i}\,(A \Rightarrow A)\big)\\
&\mathsf{id} := \mathsf{Lam}\,\lambda A.\mathsf{lam}\,\lambda a.a
\end{aligned}$$

Vegyük észre, hogy bár a fenti felírásból nem látszik közvetlenül, az A bemenetet felhasználja a függvény, hiszen a lam mindkét implicit paramétere hivatkozik rá. Az implicit paramétereket is kiírva a következő definíciót kapjuk.

$$\mathsf{id} := \mathsf{Lam}\,\{\lambda A.\mathsf{i}\,(A \Rightarrow A)\}\,\big(\lambda A.\mathsf{lam}\,\{A\}\,\{A\}\,(\lambda a.a)\big)$$

Feltéve, hogy a nyelvet kiegészítjük $\mathsf{Bool} : \mathsf{Ty}$ és $\mathsf{true} : \mathsf{Tm\,Bool}$ elemekkel, akkor ezeken a következőképp működik a fenti definíció.

$$\mathsf{id} \bullet \mathsf{Bool} \cdot \mathsf{true} \qquad\qquad =$$

$$
\begin{array}{ll}
(\mathsf{Lam}\,\lambda A.\mathsf{lam}\,\{A\}\,\{A\}\,\lambda a.a) \bullet \mathsf{Bool} \cdot \mathsf{true} & =(\forall\beta) \\
\big((\lambda A.\mathsf{lam}\,\{A\}\,\{A\}\,\lambda a.a)\,\mathsf{Bool}\big) \cdot \mathsf{true} & = \\
(\mathsf{lam}\,\{\mathsf{Bool}\}\,\{\mathsf{Bool}\}\,\lambda a.a) \cdot \mathsf{true} & =(\Rightarrow\beta) \\
(\lambda a.a)\,\mathsf{true} & = \\
\mathsf{true} &
\end{array}
$$

169. Feladat. *Adjuk meg az alábbi termeket a Hindley-Milner típusrendszerben:*

$$
\begin{aligned}
&\mathsf{const} : \mathsf{Tm}\,\big(\forall\lambda A.\forall\lambda B.\mathsf{i}\,(A \Rightarrow B \Rightarrow A)\big) \\
&\mathsf{comp} : \mathsf{Tm}\,\Big(\forall\lambda A.\forall\lambda B.\forall\lambda C.\mathsf{i}\,\big((B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C\big)\Big) \\
&\mathsf{flip}\ : \mathsf{Tm}\,\Big(\forall\lambda A.\forall\lambda B.\forall\lambda C.\mathsf{i}\,\big((A \Rightarrow B \Rightarrow C) \Rightarrow B \Rightarrow A \Rightarrow C\big)\Big)
\end{aligned}
$$

A Hindley–Milner-típusrendszerben nincsenek induktív és koinduktív típusok, kézzel kell őket hozzáadni, csakúgy mint egyszerű típuselméletben. A szokásos módszer az, hogy minden típusra van egy if-then-else/case jellegű operátorunk, és van egy központi fixpontoperátorunk, mint PCF-ben.

170. Feladat. *Egészítsük ki a Hindley–Milner-típusrendszert fixpont-operátorral, egész számokkal (például* $\mathbb{Z} \to \mathsf{Tm}\,\mathsf{Int}$ *bevezető operátorral), melyekre van néhány primitív operátor: összeadás, szorzás, hatványozás, és* $\leq$ *összehasonlítás, mely C-stílusú logikai értéket ad vissza.*

171. Feladat. *Mutassuk meg, hogy az így megadott nyelven definiálható a faktoriális, a Fibonacci- és az Ackermann-függvény!*

Általánosságban a Hindley-Milner típusok a következőképp néznek ki:

$$
\forall\,\lambda A_1.\forall\,\lambda A_2.\ldots.\forall\,\lambda A_n.\mathsf{i}\,(\underbrace{\ldots\ldots\ldots\ldots\ldots}_{\text{itt már nem szerepel }\forall})
$$

Tehát például a $\mathsf{Bool} \Rightarrow \forall\,\lambda A.A$ típus nem értelmes, ilyenkor a $\forall$ kvantort előre kell vinni. A Hindley-Milner típusrendszer nagyon kényelmes, mert típusok írása nélkül is típushelyesen lehet benne programozni.

172. Gondolkodnivaló. *A Hindley–Milner-nyelvre megadható egy típuskiövetkeztető, mely típusinformáció nélküli pretermekből jól típusozott termeket készít, és mindig a legáltalánosabb típust adja meg. A pretermekben nem szerepel* Lam*, csak* lam*, az algoritmus maga illeszti be a* $\forall$ *és a* Lam *operátorokat. Lásd [Hin69, Mil78, Csö12].*

A következő nyelvben a $\forall$ már bárhol szerepelhet, nem csak a típusok legelején. Jean-Yves Girard [Gir72] és John C. Reynolds [Rey74] fedezték fel egymástól függetlenül, előbbi System F-nek, utóbbi polimorf lambda-kalkulusnak nevezte.

173. Definíció (System F)**.** *Két szortunk van, a típusok és a típusokkal indexelt termek.*

$$
\frac{A \qquad B}{A \Rightarrow B} \qquad \frac{x : A \vdash b : B}{\mathsf{lam}\,x.b : A \Rightarrow B} \qquad \frac{f : A \Rightarrow B \qquad a : A}{f \cdot a : B} \qquad \frac{}{(\mathsf{lam}\,x.b) \cdot a = b[x \mapsto a]} \qquad \frac{f : A \Rightarrow B}{f = \mathsf{lam}\,x.f \cdot x}
$$

$$
\frac{X \vdash A}{\forall X.A} \qquad \frac{X \vdash a : A}{\mathsf{Lam}\,X.a : \forall X.A} \qquad \frac{f : \forall X.A \qquad C}{f \bullet C : A[X \mapsto C]} \qquad \frac{}{(\mathsf{Lam}\,X.a) \bullet C = a[X \mapsto C]} \qquad \frac{f : \forall X.A}{f = \mathsf{Lam}\,X.f \bullet X}
$$

A System F tömörebb megadása:

$$
\begin{array}{ll}
\mathsf{Ty} & : \mathsf{Set} \\
\mathsf{Tm} & : \mathsf{Ty} \to \mathsf{Set} \\
- \Rightarrow - & : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty} \\
\mathsf{lam} & : (\mathsf{Tm}\,A \to \mathsf{Tm}\,B) \cong \mathsf{Tm}\,(A \Rightarrow B) : - \cdot - \\
\forall & : (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Ty} \\
\mathsf{Lam} & : ((X : \mathsf{Ty}) \to \mathsf{Tm}\,(A\,X)) \cong \mathsf{Tm}\,(\forall\,A) : - \bullet -
\end{array}
$$

A System F különlegessége, hogy az összes induktív és koinduktív típus elkódolható benne (β szabályokkal, η szabályokat nem kapunk). A módszert *Church-kódolásnak* nevezzük. Például a kételemű típust a következőképp adjuk meg.

$$
\mathsf{Bool} \quad := \forall\,\lambda A.A \Rightarrow A \Rightarrow A
$$

$$
\begin{aligned}
&\mathsf{true} && := \mathsf{Lam}\,\lambda A.\mathsf{lam}\,\lambda t.\mathsf{lam}\,\lambda f.t\\
&\mathsf{false} && := \mathsf{Lam}\,\lambda A.\mathsf{lam}\,\lambda t.\mathsf{lam}\,\lambda f.f\\
&\mathsf{ite} && : \mathsf{Tm}\,\mathsf{Bool} \to \mathsf{Tm}\,(\forall\,\lambda A.A \Rightarrow A \Rightarrow A)\\
&\mathsf{ite} && := \lambda b.b\\
&\mathsf{Bool}\beta_1 && : \mathsf{ite}\,\mathsf{true} \bullet A \cdot t \cdot f = (\lambda b.b)\,\mathsf{true} \bullet A \cdot t \cdot f = \mathsf{true} \bullet A \cdot t \cdot f = (\mathsf{Lam}\,\lambda A.\mathsf{lam}\,\lambda t.\mathsf{lam}\,\lambda f.t) \bullet A \cdot t \cdot f = t\\
&\mathsf{Bool}\beta_2 && : \mathsf{ite}\,\mathsf{false} \bullet A \cdot t \cdot f = (\lambda b.b)\,\mathsf{false} \bullet A \cdot t \cdot f = \mathsf{false} \bullet A \cdot t \cdot f = (\mathsf{Lam}\,\lambda A.\mathsf{lam}\,\lambda t.\mathsf{lam}\,\lambda f.f) \bullet A \cdot t \cdot f = f
\end{aligned}
$$

Az intuíció, hogy egy Bool típusú term csak kétféle dolgot tud csinálni: visszaadni az első vagy a második A típusú paraméterét. Nem vizsgálhatja meg, hogy mi az az A, egységes (parametrikus, természetes, uniform) módon kell működnie minden A-ra. Ezt az egységes működést fejezi ki a parametricitás elmélete [Rey83, Wad89] és a kategóriaelméletben a természetes transzformációk [Awo10].

Vegyük észre, hogy a Bool Hindley-Milner típusrendszerben nem kódolható el, ehhez már szükségünk van arra, hogy ne csak legkívül lehessen $\forall$.

174. Feladat. *A fenti* Bool *definícióban az* ite *operátor típusát úgy adtuk meg, hogy a legkönnyebb legyen implementálni. Mutassuk meg, hogy ez a típus izomorf a hagyományossal, tehát*

$$\big(\mathsf{Tm}\,\mathsf{Bool} \to \mathsf{Tm}\,(\forall\,\lambda A.A \Rightarrow A \Rightarrow A)\big) \cong \big(\{A : \mathsf{Ty}\} \to \mathsf{Tm}\,\mathsf{Bool} \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A\big).$$

Néhány további típus Church-kódolása:

üres típus	$\bot$	$\forall\,\lambda C.C$
egyelemű típus	$\top$	$\forall\,\lambda C.C \Rightarrow C$
háromelemű típus		$\forall\,\lambda C.C \Rightarrow C \Rightarrow C \Rightarrow C$
összeg típus	$A + B$	$\forall\,\lambda C.(A \Rightarrow C) \Rightarrow (B \Rightarrow C) \Rightarrow C$
szorzat típus	$A \times B$	$\forall\,\lambda C.(A \Rightarrow B \Rightarrow C) \Rightarrow C$
természetes számok	Nat	$\forall\,\lambda C.C \Rightarrow (C \Rightarrow C) \Rightarrow C$
A-kat tartalmazó listák	List A	$\forall\,\lambda C.C \Rightarrow (A \Rightarrow C \Rightarrow C) \Rightarrow C$
bináris fák	Tree	$\forall\,\lambda C.C \Rightarrow (C \Rightarrow C \Rightarrow C) \Rightarrow C$

175. Feladat. *Adjuk meg a fenti típusok konstruktorait, destruktorait, lássuk be a β szabályokat.*

176. Gondolkodnivaló. *Mutassuk meg, hogy tetszőleges szigorúan pozitív funktor által megadott induktív típus Church-kódolható System F-ben (csak β szabállyal).*

System F-ben elkódolható a létezik kvantor is:

$$
\begin{aligned}
&\exists : (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Ty}\\
&\exists F := \forall\,\lambda C.(\forall\,\lambda X.F\,X \Rightarrow C) \Rightarrow C
\end{aligned}
$$

177. Feladat. *Adjuk meg a létezik operátor konstruktorát, destruktorát és β szabályát:*

$$
\begin{aligned}
&\mathsf{mk}\exists && : (X : \mathsf{Ty}) \to \mathsf{Tm}\,(F\,X) \to \mathsf{Tm}\,(\exists F)\\
&\mathsf{case} && : ((X : \mathsf{Ty}) \to \mathsf{Tm}\,(F\,X) \to \mathsf{Tm}\,C) \to \mathsf{Tm}\,(\exists F) \to \mathsf{Tm}\,C\\
&\exists\beta && : \mathsf{case}\,w\,(\mathsf{mk}\exists\,X\,f) = w\,X\,f
\end{aligned}
$$

A koinduktív típusok elkódolását a folyamok példáján illusztráljuk. Egy folyamot egy C típus, a C-n egy koalgebra, valamint a C-nek egy eleme (a mag, amiből kinő a folyam) ad meg.

$$
\begin{aligned}
&\mathsf{Stream} : \mathsf{Ty} \to \mathsf{Ty}\\
&\mathsf{Stream}\,A := \exists\,\lambda C.\big((C \Rightarrow A) \times (C \Rightarrow C)\big) \times C\\
&\mathsf{head} : \mathsf{Tm}\,(\mathsf{Stream}\,A) \to \mathsf{Tm}\,A\\
&\mathsf{head} := \mathsf{case}\,\big(\lambda X\,w.\mathsf{fst}\,(\mathsf{fst}\,w) \cdot \mathsf{snd}\,w\big)\\
&\mathsf{tail} : \mathsf{Tm}\,(\mathsf{Stream}\,A) \to \mathsf{Tm}\,(\mathsf{Stream}\,A)\\
&\mathsf{tail} := \mathsf{case}\,\Big(\lambda X\,w.\mathsf{mk}\exists\,X\,\big(\mathsf{fst}\,w, \mathsf{snd}\,(\mathsf{fst}\,w) \cdot \mathsf{snd}\,w\big)\Big)\\
&\mathsf{gen} : \{C : \mathsf{Ty}\} \to (\mathsf{Tm}\,C \to \mathsf{Tm}\,A) \to (\mathsf{Tm}\,C \to \mathsf{Tm}\,C) \to \mathsf{Tm}\,C \to \mathsf{Tm}\,(\mathsf{Stream}\,A)\\
&\mathsf{gen}\,\{C\}\,h\,t\,c := \mathsf{mk}\exists\,C\,\big((\mathsf{lam}\,h, \mathsf{lam}\,t), c\big)
\end{aligned}
$$

178. Feladat. *Bizonyítsuk be a fenti Church-kódolt folyam típus számítási szabályait:*

$$\mathsf{Stream}\beta_1 : \mathsf{head}\,(\mathsf{gen}\,h\,t\,c) = h\,c$$
$$\mathsf{Stream}\beta_2 : \mathsf{tail}\,(\mathsf{gen}\,h\,t\,c) = \mathsf{gen}\,h\,t\,(t\,c)$$

179. Gondolkodnivaló. *Mutassuk meg, hogy tetszőleges szigorúan pozitív funktor által megadott koinduktív típus Church-kódolható System F-ben (csak β szabállyal).*

180. Gondolkodnivaló. *Ha van belső parametricitás [ACKS24], az η szabályok is teljesülnek. Meg lehet-e a belső parametricitást fogalmazni függő típusok nélkül?*

System F-ben vannak polimorf függvények, de nincsenek típus szintű függvények. Például a listák nem adhatók meg, ami egy típusról típusra képező függvény. A következő rendszer ezt kijavítja. A típusokon és a termeken túl van egy újabb szortunk, a fajták (Kind-ok) szortja, a típusok pedig indexelve vannak az ő „típusukkal" (fajtájukkal). A $*$ fajtájú típusok a tulajdonképpeni típusok, nekik lehetnek termei. A $* \Rrightarrow *$ fajtájú típusok a típus-függvények, melyek egy tulajdonképpeni típusból képeznek egy tulajdonképpeni típusba.

181. Definíció (System F_ω)**.**

$$\begin{array}{ll}
\mathsf{Kind} & : \mathsf{Set} \\
\mathsf{Ty} & : \mathsf{Kind} \to \mathsf{Set} \\
- \Rrightarrow - & : \mathsf{Kind} \to \mathsf{Kind} \to \mathsf{Kind} \\
\mathsf{LAM} & : (\mathsf{Ty}\,K \to \mathsf{Ty}\,L) \cong \mathsf{Ty}\,(K \Rrightarrow L) : - \bullet - \\
* & : \mathsf{Kind} \\
\mathsf{Tm} & : \mathsf{Ty} * \to \mathsf{Set} \\
\forall & : (\mathsf{Ty}\,K \to \mathsf{Ty} *) \to \mathsf{Ty} * \\
\mathsf{Lam} & : ((X : \mathsf{Ty}\,K) \to \mathsf{Tm}\,(A\,X)) \cong \mathsf{Tm}\,(\forall\,A) : - \bullet - \\
- \Rightarrow - & : \mathsf{Ty} * \to \mathsf{Ty} * \to \mathsf{Ty} * \\
\mathsf{lam} & : (\mathsf{Tm}\,A \to \mathsf{Tm}\,B) \cong \mathsf{Tm}\,(A \Rightarrow B) : - \cdot -
\end{array}$$

A fajták és típusok a $\Rrightarrow$-val úgy viselkednek, mint egyszerű típuselméletben a típusok és a termek. A System F_ω érdekessége, hogy az egyik szort felhasznál egy operátort ($*$-ot), tehát a szortok nem adhatók meg az operátoroktól elkülönülten.

A lista típus a következőképp Church-kódolható:

$$\mathsf{List} : \mathsf{Ty}\,(* \Rrightarrow *)$$
$$\mathsf{List} := \mathsf{LAM}\,\lambda A.\forall\,\lambda B.B \Rrightarrow (A \Rightarrow B \Rightarrow B) \Rightarrow B$$

A Haskell bind operátorának a következő a típusa:

$$\forall\,\lambda M.\forall\,\lambda A.\forall\,\lambda B.(A \Rrightarrow M \bullet B) \Rrightarrow M \bullet A \Rrightarrow M \bullet B$$

A λ-kat kihagyva és a $\forall$ által kötött típusok fajtáit odaírva a következőt kapjuk:

$$\forall(M : * \Rrightarrow *).\forall(A : *).\forall(B : *).(A \Rrightarrow M \bullet B) \Rrightarrow M \bullet A \Rrightarrow M \bullet B$$

182. Feladat. *Definiáljuk a Church-kódolt lista típus konstruktorait és iterátorát (fold operátorát)!*

183. Feladat. *Adjuk meg a Church-kódolt lista típus return és bind operátorait és vizsgáljuk meg, hogy teljesítik-e a monád törvényeket!*

4.9. Függő típusok

A függő típuselméletben a típusok termektől is függhetnek (System F-ben a típusok csak típusoktól függhettek). Az első ilyen típuselméleteket Per Martin-Löf [ML98] és Nicolas De Bruijn [GN22] dolgozta ki.

184. Definíció (Martin-Löf típuselmélete)**.** *Két szortunk van, a típusok és a típusokkal indexelt termek. A típusok továbbá a szintjükkel vannak indexelve, ami egy természetes szám. A 79. jelölést használjuk.*

$$\begin{array}{ll}
\mathsf{Ty} & : \mathbb{N} \to \mathsf{Set} \\
\mathsf{Tm} & : \mathsf{Ty}\,i \to \mathsf{Set} \\
\Pi & : (A : \mathsf{Ty}\,i) \to (\mathsf{Tm}\,A \to \mathsf{Ty}\,i) \to \mathsf{Ty}\,i
\end{array}$$

$$
\begin{array}{ll}
\mathsf{lam} & : ((a : \mathsf{Tm}\, A) \to \mathsf{Tm}\,(B\, a)) \cong \mathsf{Tm}\,(\Pi\, A\, B) : - \cdot - \\
\Sigma & : (A : \mathsf{Ty}\, i) \to (\mathsf{Tm}\, A \to \mathsf{Ty}\, i) \to \mathsf{Ty}\, i \\
(-,-) & : (a : \mathsf{Tm}\, A) \times \mathsf{Tm}\,(B\, a) \cong \mathsf{Tm}\,(\Sigma\, A\, B) : \mathsf{fst}, \mathsf{snd} \\
\mathsf{U} & : (i : \mathbb{N}) \to \mathsf{Ty}\,(i+1) \\
\mathsf{c} & : \mathsf{Ty}\, i \cong \mathsf{Tm}\,(\mathsf{U}\, i) : \mathsf{El} \\
\mathsf{Lift} & : \mathsf{Ty}\, i \to \mathsf{Ty}\,(i+j) \\
\mathsf{mk} & : \mathsf{Tm}\, A \cong \mathsf{Tm}\,(\mathsf{Lift}\, A) : \mathsf{un} \\
\bot & : \mathsf{Ty}\, 0 \\
\mathsf{exfalso} & : \mathsf{Tm}\, \bot \to \mathsf{Tm}\, A \\
\top & : \mathsf{Ty}\, 0 \\
\mathsf{tt} & : \top \cong \mathsf{Tm}\, \top \\
\mathsf{Bool} & : \mathsf{Ty}\, 0 \\
\mathsf{true} & : \mathsf{Tm}\, \mathsf{Bool} \\
\mathsf{false} & : \mathsf{Tm}\, \mathsf{Bool} \\
\mathsf{indBool} & : (C : \mathsf{Tm}\, \mathsf{Bool} \to \mathsf{Ty}\, i) \to \mathsf{Tm}\,(C\, \mathsf{true}) \to \mathsf{Tm}\,(C\, \mathsf{false}) \to \\
& \quad (b : \mathsf{Tm}\, \mathsf{Bool}) \to \mathsf{Tm}\,(C\, b) \\
\mathsf{Bool}\beta_1 & : \mathsf{indBool}\, t\, f\, \mathsf{true} = t \\
\mathsf{Bool}\beta_2 & : \mathsf{indBool}\, t\, f\, \mathsf{false} = f \\
\mathsf{Id} & : (A : \mathsf{Ty}\, i) \to \mathsf{Tm}\, A \to \mathsf{Tm}\, A \to \mathsf{Ty}\, i \\
\mathsf{refl} & : (a : \mathsf{Tm}\, A) \to \mathsf{Tm}\,(\mathsf{Id}\, a\, a) \\
\mathsf{J} & : \big(C : (x : \mathsf{Tm}\, A) \to \mathsf{Tm}\,(\mathsf{Id}\, A\, a\, x) \to \mathsf{Ty}\, i\big) \to \\
& \quad \mathsf{Tm}\,\big(C\, a\,(\mathsf{refl}\, a)\big) \to (x : \mathsf{Tm}\, A)(e : \mathsf{Tm}\,(\mathsf{Id}\, A\, a\, x)) \to \mathsf{Tm}\,(C\, x\, e) \\
\mathsf{Id}\beta & : \mathsf{J}\, C\, w\, a\,(\mathsf{refl}\, a) = w \\
\mathsf{W} & : (S : \mathsf{Ty}\, i) \to (\mathsf{Tm}\, S \to \mathsf{Ty}\, i) \to \mathsf{Ty}\, i \\
\mathsf{sup} & : (s : \mathsf{Tm}\, S) \to (\mathsf{Tm}\,(P\, s) \to \mathsf{Tm}\,(\mathsf{W}\, S\, P)) \to \mathsf{Tm}\,(\mathsf{W}\, S\, P) \\
\mathsf{indW} & : (C : \mathsf{Tm}\,(\mathsf{W}\, S\, P) \to \mathsf{Ty}\, i) \to \\
& \quad \Big(\big((p : \mathsf{Tm}\,(P\, s)) \to \mathsf{Tm}\,(C\,(f\, p))\big) \to \mathsf{Tm}\,\big(C\,(\mathsf{sup}\, s\, f)\big)\Big) \to \\
& \quad (w : \mathsf{Tm}\,(\mathsf{W}\, \mathsf{S}\, \mathsf{P})) \to \mathsf{Tm}\,\big(C\, w\big) \\
\mathsf{W}\beta & : \mathsf{indW}\, C\, h\,(\mathsf{sup}\, s\, f) = h\,(\lambda p.\mathsf{indW}\, C\, h\,(f\, p))
\end{array}
$$

185. Feladat. *Adjuk meg a Martin-Löf-típuselmélet sekély beágyazását a metanyelvbe!*

186. Jelölés. *Rövidítések:*

$$
\begin{aligned}
A \Rightarrow B &:= \Pi\, A\, (\lambda_.B) \\
A \times\ \ B &:= \Sigma\, A\, (\lambda_.B)
\end{aligned}
$$

187. Feladat. *Adjuk meg a nem-függő függvény típus szokásos konstruktorát és destruktorát, és bizonyítsuk be a β és η szabályokat!*

188. Feladat. *Az összeg típusok megadhatók Σ és* Bool *segítségével.*

189. Feladat. *Adjuk meg az üres típus függő eliminátorát!*

190. Feladat. *Adjuk meg a nem-függő szorzat ($\times$) típus konstruktorát, destruktorait, és bizonyítsuk be a β és η szabályokat!*

191. Feladat. *Mutassuk meg, hogy minden szigorúan pozitív típusoperátor (lásd a 148. definíciót) megadható S, P segítségével.*

192. Megjegyzés. *A* W *típusok S, P paraméterét konténernek (container) hívjuk [AAG05].*

193. Megjegyzés. *Aref. definíció a típuselmélet à la Coquand változata. Ha nincs* $\mathsf{c} : \mathsf{Ty}\, i \to \mathsf{Tm}\,(\mathsf{U}\, i)$ *operátorunk, csak a másik irányú* El*, akkor à la Tarski-nak nevezzük. Ha ezek helyett a* $\mathsf{Ty}\, i = \mathsf{Tm}\,(\mathsf{U}\, i)$ *fajta-egyenlőségünk van, azt à la Russell-nek nevezzük.*

194. Feladat. *Adjunk meg* $\mathsf{Tm}\,(\mathsf{Id}\, \mathsf{Bool}\, \mathsf{true}\, \mathsf{false} \Rightarrow \bot)$ *egy elemét!*

195. Feladat. *Fogalmazzuk meg, majd bizonyítsuk be, hogy az* $\mathsf{Id}\,A$ *tetszőleges* A*-ra egy ekvivalencia-reláció!*

196. Feladat. *Adjuk meg a természetes számokat a* W *típus segítségével! Bizonyítsuk be a teljes indukció elvét! (Szükség van Hugunin trükkjére [Hug20].)*

197. Feladat. *Adjuk meg a koinduktív típusok szabályait* M *típusok segítségével!*

198. Feladat. *Mutassuk meg, hogy ha az univerzum szabálya* $\mathsf{U}\,i : \mathsf{Ty}\,i$ *lenne, akkor van* $\mathsf{Tm}\,\bot$*-nak eleme.*

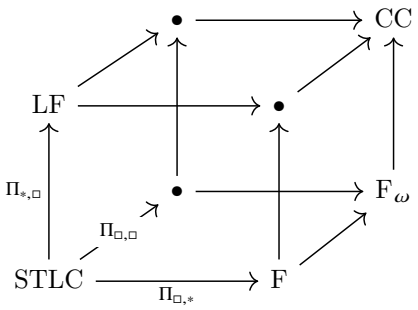
199. Feladat. *Fogalmazzuk meg, majd mutassuk meg, hogy az elsőrendű logika beágyazható Martin-Löf-típuselméletbe!*

200. Definíció (Russell)**.** *A Martin-Löf-típuselmélet Russell típusú, ha a következő szort-egyenlőség teljesül:*

$$\mathsf{Ty}\,i = \mathsf{Tm}\,(\mathsf{U}\,i)$$

201. Feladat. *Mutassunk egy másodrendű modellt, amelyben ez teljesül! Mutassuk meg, hogy tetszőleges másodrendű modellből képezhető olyan nemtriviális másodrendű modell, mely Russell.*

A következő definíció segítségével megadható Barendregt lambda-kockájának (lambda-cube, [Bar91]) összes nyelve. Az egyszerű típusos lambda kalkulus (STLC, 92. definíció) csak a $\Pi_{*,*}$ típust tartalmazza, és minden dimenzió egy újfajta Π típust ad hozzá a nyelvhez (F-el jelöljük a 173. definíciót, F_ω-val a 181. definíciót, az LF a logical framework [HHP93]). A calculus of constructions (CC, [CH88]) mind a négyféle Π típust tartalmazza. Az



A különböző Π típusok univerzális tulajdonságainak (lambda, applikáció) nem adunk nevet.

202. Definíció (CC)**.**

$$
\begin{array}{lll}
\square & : \mathsf{Set} & \\
\mathsf{Ty} & : \square \to \mathsf{Set} & \\
* & : \square & \\
\mathsf{Tm} & : \mathsf{Ty}\,* \to \mathsf{Set} & \\
\Pi_{*,*} & : (A : \mathsf{Ty}\,*) \to (\mathsf{Tm}\,A \to \mathsf{Ty}\,(*)) \to \mathsf{Ty}\,* & \mathsf{Tm}\,(\Pi_{*,*}\,A\,B) \cong (a : \mathsf{Tm}\,A) \to \mathsf{Tm}\,(B\,a) \\
\Pi_{*,\square} & : (A : \mathsf{Ty}\,*) \to (\mathsf{Tm}\,A \to \square) \to \square & \mathsf{Ty}\,(\Pi_{*,\square}\,A\,L) \cong (a : \mathsf{Tm}\,A) \to \mathsf{Ty}\,(L\,a) \\
\Pi_{\square,*} & : (K : \square) \to (\mathsf{Ty}\,K \to \mathsf{Ty}\,*) \to \mathsf{Ty}\,* & \mathsf{Tm}\,(\Pi_{\square,*}\,K\,B) \cong (A : \mathsf{Ty}\,K) \to \mathsf{Tm}\,(B\,A) \\
\Pi_{\square,\square} & : (K : \square) \to (\mathsf{Ty}\,K \to \square) \to \square & \mathsf{Ty}\,(\Pi_{\square,\square}, K\,L) \cong (A : \mathsf{Ty}\,K) \to \mathsf{Ty}\,(L\,A)
\end{array}
$$

203. Gondolkodnivaló. *A következő nyelvek (elsőrendű) szintaxisában minden termnek van normálformája: a lambda-kocka összes nyelve, Hindley–Milner, Martin-Löf-típuselmélet.*

4.10. Elsőrendű logika

Az alábbiakban megadjuk a minimális elsőrendű intuicionista logika nyelvét természetes levezetéses bizonyításelmélettel. A „minimális” azt jelenti, hogy csak implikációnk, univerzális kvantorunk van, és egy relációnk, az egyenlőség. Az egyszerűség kedvéért a bizonyítások szerkezetével nem foglalkozunk, ezt az az egyenlőség fejezi ki, amely azt mondja, hogy tetszőleges két bizonyítás egyenlő. Emiatt a bizonyításokra vonatkozó hipotéziseknek nem adunk nevet, tehát nem írunk kettőspontot és elé nem írunk nevet (sem a $\vdash$ előtt, sem a feltételekben és a konklúziókban).

204. Definíció (Minimális intuicionista logika)**.**

$$\frac{}{\mathsf{For} : \mathsf{Set}} \quad \frac{}{\mathsf{Tm} : \mathsf{Set}} \quad \frac{A : \mathsf{For} \qquad B : \mathsf{For}}{A \supset B : \mathsf{For}} \quad \frac{x : \mathsf{Tm} \vdash A : \mathsf{For}}{\forall x.A : \mathsf{For}}$$

$$\frac{t : \mathsf{Tm} \qquad t' : \mathsf{Tm}}{\mathsf{Eq}\,t\,t' : \mathsf{For}} \quad \frac{A : \mathsf{For}}{\mathsf{Pf}\,A : \mathsf{Set}} \quad \frac{p : \mathsf{Pf}\,A \qquad q : \mathsf{Pf}\,A}{p = q}$$

$$\frac{\mathsf{Pf}\,A \vdash \mathsf{Pf}\,B}{\mathsf{Pf}\,(A \supset B)} \qquad \frac{\mathsf{Pf}\,(A \supset B) \qquad \mathsf{Pf}\,A}{\mathsf{Pf}\,B}$$

$$\frac{x : \mathsf{Tm} \vdash \mathsf{Pf}\,A}{\mathsf{Pf}\,(\forall x.A)} \qquad \frac{\mathsf{Pf}\,(\forall x.A) \qquad t : \mathsf{Tm}}{\mathsf{Pf}\,(A[x \mapsto t])} \qquad \frac{t : \mathsf{Tm}}{\mathsf{Pf}\,(\mathsf{Eq}\,t\,t)} \qquad \frac{\mathsf{Pf}\,(\mathsf{Eq}\,t\,t')}{t = t'}$$

Példaként bebizonyítjuk, hogy $(A \supset B \supset C) \supset (B \supset A \supset C)$ minden A, B, C állításra. A leveleknél változók vannak.

$$\dfrac{\dfrac{\dfrac{\dfrac{\overline{\Gamma \vdash \mathsf{Pf}\,(A \supset B \supset C)} \qquad \overline{\Gamma \vdash \mathsf{Pf}\,A}}{\Gamma \vdash \mathsf{Pf}\,(B \supset C)} \qquad \overline{\Gamma \vdash \mathsf{Pf}\,B}}{\underbrace{\mathsf{Pf}\,(A \supset B \supset C), \mathsf{Pf}\,B, \mathsf{Pf}\,A}_{=:\Gamma} \vdash \mathsf{Pf}\,C}}{\mathsf{Pf}\,(A \supset B \supset C), \mathsf{Pf}\,B \vdash \mathsf{Pf}\,(A \supset C)}}{\mathsf{Pf}\,(A \supset B \supset C) \vdash \mathsf{Pf}\,(B \supset A \supset C)}}{\mathsf{Pf}\,\big((A \supset B \supset C) \supset (B \supset A \supset C)\big)}$$

205. Feladat. *Adjuk meg az intuicionista logika másodrendű Tarski-szemantikáját! A formulák (metanyelvi) állításokkal, a termek valamilyen posztulált halmazzal, a bizonyítások metanyelvi bizonyításokkal vannak modellezve.*

206. Feladat. *Egészítsük ki a nyelvet általános reláció- és termszimbólumokkal, melyeket egy elsőrendű szignatúrából veszünk!*

207. Feladat. *Egészítsük ki a nyelvet a logikai konjunkció, diszjunkció, tagadás, igaz, hamis és egzisztenciális kvantor szabályaival!*

208. Feladat. *Mutassuk meg, hogy ha az így kiegészített nyelvhez hozzávesszük az alábbi szabályt, akkor a formulák Heyting-algebrát alkotnak!*

$$\frac{\mathsf{Pf}\,(A \supset B) \qquad \mathsf{Pf}\,(B \supset A)}{A = B}$$

209. Feladat. *Vezessük le tetszőleges A-ra, hogy* $\mathsf{Pf}\,(\neg\neg(A \vee \neg A))$ *!*

210. Feladat. *Mutassuk meg, hogy akkor és csak akkor vezethető le minden A-ra* $\mathsf{Pf}\,(A \vee \neg A)$*, ha minden A-ra* $\mathsf{Pf}\,(\neg\neg A \supset A)$ *levezethető!*

211. Feladat. *Tegyük az előző feladat nyelvét klasszikussá: adjuk hozzá a kizárt harmadik elvét, tehát az*

$$\frac{}{\mathsf{Pf}\,(A \vee \neg A)}$$

szabályt! Mutassuk meg, hogy ha a klasszikus nyelvhez hozzávesszük az alábbi szabályt, akkor a formulák Boole-algebrát alkotnak!

$$\frac{\mathsf{Pf}\,(A \supset B) \qquad \mathsf{Pf}\,(B \supset A)}{A = B}$$

5. Másodrendű általánosított algebra szignatúrák megadása másodrendű általánosított algebrai elmélettel

Ebben a fejezetben univerzális algebrával foglalkozunk. Ahogy a 33. definícióban megadtuk az algebrai elméletek szignatúráit, most megint megadjuk az AT szignatúrákat, de a szignatúrák nyelve egy konkrét SOGAT szintaxisa lesz. Látni fogjuk, hogy ez kényelmes definícióhoz vezet, ahol kevesebb kódolási költség van. A SOGAT-tal megadott szignatúra módszer továbbá nem csak AT szignatúrákra, hanem GAT és SOGAT szignatúrákra is működik.

A következő SOGAT szintaxisa írja le az algebrai elméletek szignatúráit, ez egy alternatívája a 33. definíciónak. Egy típus felel meg egy szignatúrának. Az algebrai szignatúrák nyelve Martin-Löf típuselméletének egy megszorítása: a Π típusnak például fix az értelmezési tartománya. Ezzel biztosítjuk, hogy ne lehessenek magasabbrendű függvények, és a függvények bemenete mindig az adott algebrai elmélet egyetlen szortja legyen. Egy alaptípusuk van, az algebrai elmélet szortja (Srt).

212. Definíció (AT)**.**

$$\begin{aligned} \mathsf{Ty} &: \mathsf{Set} \\ \mathsf{Tm} &: \mathsf{Ty} \to \mathsf{Set} \end{aligned}$$

$$
\begin{aligned}
\Sigma &: (A : \mathsf{Ty}) \to (\mathsf{Tm}\, A \to \mathsf{Ty}) \to \mathsf{Ty}\\
\mathsf{Srt} &: \mathsf{Ty}\\
\Pi\mathsf{Srt} &: (\mathsf{Tm}\,\mathsf{Srt} \to \mathsf{Ty}) \to \mathsf{Ty}\\
- \cdot - &: \mathsf{Tm}\,(\Pi\mathsf{Srt}\, B) \to (x : \mathsf{Tm}\,\mathsf{Srt}) \to \mathsf{Tm}\,(B\,x)\\
\mathsf{Id} &: \mathsf{Tm}\,\mathsf{Srt} \to \mathsf{Tm}\,\mathsf{Srt} \to \mathsf{Ty}
\end{aligned}
$$

Minden $B : \mathsf{Ty}$-ra bevezetjük a $\mathsf{Srt} \Rightarrow B := \Pi\mathsf{Srt}\,(\lambda_.B)$ rövidítést.
Például a félcsoportok (4. definíció) a következőképp adhatók meg, a szignatúra Ty egy eleme.

$$
\begin{aligned}
&\Sigma\,(\mathsf{Srt} \Rightarrow \mathsf{Srt} \Rightarrow \mathsf{Srt})\lambda op.\\
&\quad \Pi\mathsf{Srt}\,\lambda x.\Pi\mathsf{Srt}\,\lambda y.\Pi\mathsf{Srt}\,\lambda z.\mathsf{Id}\,\big(op \cdot (op \cdot x \cdot y) \cdot z\big)\,\big(op \cdot x \cdot (op \cdot y \cdot z)\big)
\end{aligned}
$$

Összehasonlítva a 4. definícióval, a szortot nem kell megadnunk, hiszen algebrai elméletekben (AT-ben) csak egy szort van, a Σ típus első komponense a bináris operátor lesz, melynek curry-zett típusát megadjuk és op-nak nevezzük el. A Σ típus második komponense kvantifikál három Srt-beli elem fölött, majd visszaadja az egyenlőség (identitás) típus megfelelő elemét. a 4. definícióban megadott verzió tekinthető ennek a formális definíciónak a kényelmesebb megadásának.

213. Feladat. *Adjuk meg az 1. fejezetben megadott összes AT szignatúráját a fenti módon!*

214. Megjegyzés. *A definíciónk szerint az üres algebrai elmélet (egy szort, nulla operátor) nem AT. Ez is bevehető, ha az AT elmélethez hozzávesszük az egyelemű $\top$ típust.*

A következő SOGAT szintaxisa írja le a zárt GAT-ok szignatúráit. Egy típus felel meg egy szignatúrának, a szignatúra szortjai U-ban vannak, az operátorait (és az indexelt szortokat) a Π függvénytípussal írjuk le.

215. Definíció (Zárt GAT)**.**

$$
\begin{aligned}
\mathsf{Ty} &: \mathsf{Set}\\
\mathsf{Tm} &: \mathsf{Ty} \to \mathsf{Set}\\
\Sigma &: (A : \mathsf{Ty}) \to (\mathsf{Tm}\, A \to \mathsf{Ty}) \to \mathsf{Ty}\\
(-,-) &: (a : \mathsf{Tm}\, A) \times \mathsf{Tm}\,(B\,a) \cong \mathsf{Tm}\,(\Sigma\, A\, B) : \mathsf{fst}, \mathsf{snd}\\
\mathsf{U} &: \mathsf{Ty}\\
\mathsf{El} &: \mathsf{Tm}\,\mathsf{U} \to \mathsf{Ty}\\
\Pi &: (a : \mathsf{Tm}\,\mathsf{U}) \to (\mathsf{Tm}\,(\mathsf{El}\, a) \to \mathsf{Ty}) \to \mathsf{Ty}\\
- \cdot - &: \mathsf{Tm}\,(\Pi\, a\, B) \to (x : \mathsf{Tm}\,(\mathsf{El}\, a)) \to \mathsf{Tm}\,(B\,x)\\
\mathsf{Id} &: (a : \mathsf{Tm}\,\mathsf{U}) \to \mathsf{Tm}\,(\mathsf{El}\, a) \to \mathsf{Tm}\,(\mathsf{El}\, a) \to \mathsf{Ty}\\
\mathsf{reflect} &: \mathsf{Tm}\,(\mathsf{Id}\, a\, u\, v) \to u = v
\end{aligned}
$$

Itt is használjuk a Martin-Löf-típuselméletnél bevezetett $A \Rightarrow B := \Pi\, A\,(\lambda_.B)$ és $A \times B := \Sigma\, A\,(\lambda_.B)$ rövidítéseket.
Például az előrendezés (65. definíció) szignatúrája a fenti SOGAT szintaxisában a következő típus (Ty egy eleme):

$$
\begin{aligned}
&\Sigma\,\mathsf{U}\,\lambda Ob.\Sigma\\
&\quad (Ob \Rightarrow Ob \Rightarrow \mathsf{U})\,\lambda Mor.\\
&\quad \big(\Pi\, Ob\,\lambda I.\Pi\, Ob\,\lambda J.\Pi\, Ob\,\lambda K.Mor \cdot J \cdot I \Rightarrow Mor \cdot K \cdot J \Rightarrow\\
&\quad\ \mathsf{El}\,(Mor \cdot K \cdot I)\big)\times\\
&\quad \big(\Pi\, Ob\,\lambda I.\mathsf{El}\,(Mor \cdot I \cdot I)\big)\times\\
&\quad \big(\Pi\, Ob\,\lambda I.\Pi\, Ob\,\lambda J.\Pi\,(Mor \cdot J \cdot I)\,\lambda f.\Pi\,(Mor \cdot J \cdot I)\,\lambda g.\\
&\quad\ \mathsf{Id}\,(Mor \cdot J \cdot I)\, f\, g\big)
\end{aligned}
$$

Látjuk, hogy a felsorolás helyett Σ típusokat használunk, kötéseknél metanyelvi λ-t, nincsenek implicit paramétereink, és ki kell írnunk az El-t, amikor egy U típusú termből típust csinálunk. Mindez csak egy nagyon explicit leírás a 65. definícióra.

216. Feladat. *Adjuk meg a 2. fejezetben megadott összes GAT szignatúráját a fenti módon!*

217. Definíció (Nyílt GAT)**.** *A zárt GAT szignatúrák elméletét az alábbiakkal egészítjük ki:*

$$
\begin{aligned}
\hat{\Pi} &: (T : \mathsf{Set}) \to (T \to \mathsf{Ty}) \to \mathsf{Ty}\\
- \hat{\cdot} - &: \mathsf{Tm}\,(\hat{\Pi}\, T\, B) \to (\alpha : T) \to \mathsf{Tm}\,(B\,\alpha)
\end{aligned}
$$

A nyílt általánosított algebrai elméletek már nem adhatók meg nyílt másodrendű általánosított algebrai elméletként, csak végtelenül elágazódó (infinitary) másodrendű általánosított algebrai elméletként, de ezzel semmi gond nincs, lásd [Kov23].

A másodrendű általánosított algebrai elméleteket a következő szignatúrával adjuk meg. Van egy új szortunk, U^+, az ebben levő szortokat lehet másodrendű függvénytérben használni, más néven az U^+-ban levő szortjainkhoz tartoznak változók. Persze tehetjük az összes szortot U^+-ba, ezzel a szemlélettel írtuk le a korábbi SOGAT-jainkat.

218. Definíció (SOGAT)**.** *A zárt GAT szignatúrák elméletét az alábbiakkal egészítjük ki:*

$$\begin{aligned}
&\mathsf{U}^+ && : \mathsf{Ty}\\
&\mathsf{el}^+ && : \mathsf{Tm}\,\mathsf{U}^+ \to \mathsf{Tm}\,\mathsf{U}\\
&\pi^+ && : (a^+ : \mathsf{Tm}\,\mathsf{U}^+) \to \Big(\mathsf{Tm}\,\big(\mathsf{El}\,(\mathsf{el}^+\,a^+)\big) \to \mathsf{Tm}\,\mathsf{U}\Big) \to \mathsf{Tm}\,\mathsf{U}\\
&\mathsf{lam}^+ && : \Big(\big(x : \mathsf{El}\,(\mathsf{el}^+\,a^+)\big) \to \mathsf{Tm}\,(\mathsf{El}\,(b\,x))\Big) \cong \mathsf{Tm}\,\big(\mathsf{El}\,(\pi^+\,a^+\,b)\big) : _\cdot^+_
\end{aligned}$$

Például az egyszerű típuselmélet (92. definíció) szignatúrája a következő (mivel típusok nem szerepelnek nyíl bal oldalán, más szóval nincsenek típusváltozóink, ezért a típusok szortja U-ban van).

$$\begin{aligned}
&\Sigma\,\mathsf{U}\lambda Ty.\Sigma\\
&(Ty \Rightarrow \mathsf{U}^+)\,\lambda Tm.\\
&\mathsf{El}\,Ty \times \Sigma\\
&(Ty \Rightarrow Ty \Rightarrow \mathsf{El}\,Ty).\lambda arr.\Sigma\\
&\Big(\Pi\,Ty\,\lambda A.\Pi\,Ty\,\lambda B.\\
&\ (Tm\cdot A \Rightarrow^+ \mathsf{el}^+\,(Tm\cdot B)) \Rightarrow \mathsf{El}\,\big(\mathsf{el}^+\,(Tm\cdot(arr\cdot A\cdot B))\big)\Big)\,\lambda lam.\Sigma\\
&\Big(\Pi\,Ty\,\lambda A.\Pi\,Ty\,\lambda B.\\
&\ \mathsf{el}^+\,(Tm\cdot(arr\cdot A\cdot B)) \Rightarrow \mathsf{el}^+\,(Tm\cdot A) \Rightarrow \mathsf{El}\,\big(\mathsf{el}^+\,(Tm\cdot B)\big)\Big)\,\lambda app.\\
&\Big(\Pi\,Ty\,\lambda A.\Pi\,Ty\,\lambda B.\Pi\,\big(Tm\cdot A \Rightarrow^+ \mathsf{el}^+\,(Tm\cdot B)\big)\,\lambda b.\\
&\ \Pi\,\big(\mathsf{el}^+\,(Tm\cdot A)\big)\,\lambda a.\\
&\ \mathsf{Id}\,\big(\mathsf{el}^+\,(Tm\cdot B)\big)\,\big(app\cdot A\cdot B\cdot(lam\cdot A\cdot B\cdot b)\cdot a\big)\,(b\cdot^+ a)\Big)\times\\
&\Big(\Pi\,Ty\,\lambda A.\Pi\,Ty\,\lambda B.\Pi\,\big(Tm\cdot(arr\cdot A\cdot B)\big)\,\lambda f.\\
&\ \mathsf{Id}\,\Big(\mathsf{el}^+\,\big(Tm\cdot(arr\cdot A\cdot B)\big)\Big)\\
&\quad f\,\big(lam\cdot A\cdot B\cdot(\mathsf{lam}^+\,\lambda x.app\cdot A\cdot B\cdot f\cdot x)\big)\Big)
\end{aligned}$$

219. Feladat. *Mutassuk meg, hogy a 4. fejezetben megadott összes nyelv leírható SOGAT szignatúrával. Csak azokat a szortokat tegyük U^+-ba, melyeket feltétlenül szükséges (például egyszerű típuselméletnél vagy Martin-Löf-típuselméletnél csak Tm van U^+-ban, míg System F-nél Ty is).*

220. Feladat. *Mutassuk meg, hogy a SOGAT-okat leíró SOGAT megadható SOGAT-ként (saját farkába harap a kígyó).*

221. Gondolkodnivaló. *Adjuk meg a SOAT szignatúrákat SOGAT-leírással!*

222. Megjegyzés. *Tetszőleges másodrendű általánosított algebrai elmélet másodrendű modelljéből képezhető nemtriviális elsőrendű modell. Lásd [KX24].*

223. Gondolkodnivaló. *A Martin-Löf-típuselméletet egyszer szeretnénk SOGAT-okkal kiegészíteni: például olyan szabályokkal, melyek tetszőleges SOGAT szignatúrához megadják annak elsőrendű szintaxisát.*

Hivatkozások

[AAG05] Michael Gordon Abbott, Thorsten Altenkirch, and Neil Ghani. Containers: Constructing strictly positive types. *Theor. Comput. Sci.*, 342(1):3–27, 2005. URL: https://doi.org/10.1016/j.tcs.2005.06.002, doi:10.1016/J.TCS.2005.06.002.

[ACKS24] Thorsten Altenkirch, Yorgo Chamoun, Ambrus Kaposi, and Michael Shulman. Internal parametricity, without an interval. *Proc. ACM Program. Lang.*, 8(POPL):2340–2369, 2024. doi:10.1145/3632920.

[AKSV23] Thorsten Altenkirch, Ambrus Kaposi, Artjoms Sinkarovs, and Tamás Végh. Combinatory logic and lambda calculus are equal, algebraically. In Marco Gaboardi and Femke van Raamsdonk, editors, *8th International Conference on Formal Structures for Computation and Deduction, FSCD 2023, July 3-6, 2023, Rome, Italy*, volume 260 of *LIPIcs*, pages 24:1–24:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. doi:10.4230/LIPICS.FSCD.2023.24.

[Alt08] Thorsten Altenkirch. From high school to university algebra. https://people.cs.nott.ac.uk/psztxa/publ/unialg.pdf, 2008.

[Atk18] Robert Atkey. Syntax and semantics of quantitative type theory. In Anuj Dawar and Erich Grädel, editors, *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2018, Oxford, UK, July 09-12, 2018*, pages 56–65. ACM, 2018. doi:10.1145/3209108.3209189.

[Awo10] S. Awodey. *Category Theory*. Oxford Logic Guides. OUP Oxford, 2010. URL: http://books.google.co.uk/books?id=-MCJ6x2lC7oC.

[Bar91] Henk Barendregt. Introduction to generalized type systems. *J. Funct. Program.*, 1(2):125–154, 1991. doi:10.1017/S0956796800020025.

[BL93] Stanley Burris and Simon Lee. Tarski's high school identities. *The American Mathematical Monthly*, 100(3):231–236, 1993. URL: http://www.jstor.org/stable/2324454.

[Car86] John Cartmell. Generalised algebraic theories and contextual categories. *Ann. Pure Appl. Log.*, 32:209–243, 1986. doi:10.1016/0168-0072(86)90053-9.

[CH88] Thierry Coquand and Gérard P. Huet. The calculus of constructions. *Inf. Comput.*, 76(2/3):95–120, 1988. doi:10.1016/0890-5401(88)90005-3.

[Csö12] Zoltán Csörnyei. *Bevezetés a típusrendszerek elméletébe*. ELTE Eötvös Kiadó, 2012.

[Gir72] Jean-Yves Girard. *Interprétation functionelle et élimination des coupures dans l'arithmétique d'ordre supérieure*. PhD thesis, Université Paris VII, 1972.

[Gir87] Jean-Yves Girard. Linear logic. *Theor. Comput. Sci.*, 50:1–102, 1987. doi:10.1016/0304-3975(87)90045-4.

[GKNB21] Daniel Gratzer, G. A. Kavvos, Andreas Nuyts, and Lars Birkedal. Multimodal dependent type theory. *Log. Methods Comput. Sci.*, 17(3), 2021. doi:10.46298/LMCS-17(3:11)2021.

[GN22] Herman Geuvers and Rob Nederpelt. Characteristics of de bruijn's early proof checker automath. *Fundam. Informaticae*, 185(4):313–336, 2022. doi:10.3233/FI-222112.

[Har16] Robert Harper. *Practical Foundations for Programming Languages*. Cambridge University Press, New York, NY, USA, 2nd edition, 2016.

[HHP93] Robert Harper, Furio Honsell, and Gordon D. Plotkin. A framework for defining logics. *J. ACM*, 40(1):143–184, 1993. doi:10.1145/138027.138060.

[Hin69] R. Hindley. The principal type-scheme of an object in combinatory logic. *Transactions of the American Mathematical Society*, 146:29–60, 1969. URL: http://www.jstor.org/stable/1995158.

[Hug20] Jasper Hugunin. Why not W? In Ugo de'Liguoro, Stefano Berardi, and Thorsten Altenkirch, editors, *26th International Conference on Types for Proofs and Programs, TYPES 2020, March 2-5, 2020, University of Turin, Italy*, volume 188 of *LIPIcs*, pages 8:1–8:9. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. doi:10.4230/LIPICS.TYPES.2020.8.

[Hut23] Graham Hutton. Programming Language Semantics: It's Easy As 1,2,3. *Journal of Functional Programming*, 33, October 2023.

[Kov23] András Kovács. Type-theoretic signatures for algebraic theories and inductive types. *CoRR*, abs/2302.08837, 2023. arXiv:2302.08837, doi:10.48550/ARXIV.2302.08837.

[KX24] Ambrus Kaposi and Szumi Xie. Second-order generalised algebraic theories: Signatures and first-order semantics. In Jakob Rehof, editor, *9th International Conference on Formal Structures for Computation and Deduction, FSCD 2024, July 10-13, 2024, Tallinn, Estonia*, volume 299 of *LIPIcs*, pages 10:1–10:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024. URL: `https://doi.org/10.4230/LIPIcs.FSCD.2024.10`, `doi:10.4230/LIPICS.FSCD.2024.10`.

[LN15] John Longley and Dag Normann. *Higher-Order Computability*. Theory and Applications of Computability. Springer, 2015. `doi:10.1007/978-3-662-47992-6`.

[Mil78] Robin Milner. A theory of type polymorphism in programming. *J. Comput. Syst. Sci.*, 17(3):348–375, 1978. `doi:10.1016/0022-0000(78)90014-4`.

[ML98] Per Martin-Löf. An intuitionistic theory of types. In *Twenty-five years of constructive type theory (Venice, 1995)*, volume 36 of *Oxford Logic Guides*, pages 127–172. Oxford Univ. Press, New York, 1998.

[Par89] Michel Parigot. On the representation of data in lambda-calculus. In Egon Börger, Hans Kleine Büning, and Michael M. Richter, editors, *CSL '89, 3rd Workshop on Computer Science Logic, Kaiserslautern, Germany, October 2-6, 1989, Proceedings*, volume 440 of *Lecture Notes in Computer Science*, pages 309–321. Springer, 1989. `doi:10.1007/3-540-52753-2\_47`.

[PdAC+23] Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hriţcu, Vilhelm Sjöberg, and Brent Yorgey. *Logical Foundations*, volume 1 of *Software Foundations*. Electronic textbook, 2023. Version 6.5, `http://softwarefoundations.cis.upenn.edu`.

[Pie02] Benjamin C. Pierce. *Types and programming languages*. MIT Press, 2002.

[Plo77] Gordon D. Plotkin. LCF considered as a programming language. *Theor. Comput. Sci.*, 5(3):223–255, 1977. `doi:10.1016/0304-3975(77)90044-5`.

[Rey74] John C. Reynolds. Towards a theory of type structure. In Bernard J. Robinet, editor, *Programming Symposium, Proceedings Colloque sur la Programmation, Paris, France, April 9-11, 1974*, volume 19 of *Lecture Notes in Computer Science*, pages 408–423. Springer, 1974. `doi:10.1007/3-540-06859-7\_148`.

[Rey83] John C. Reynolds. Types, abstraction and parametric polymorphism. In R. E. A. Mason, editor, *Information Processing 83, Proceedings of the IFIP 9th World Computer Congress, Paris, France, September 19-23, 1983*, pages 513–523. North-Holland/IFIP, 1983.

[Sch24] Moses Schönfinkel. Über die bausteine der mathematischen logik. *Mathematische Annalen*, 92(3):305–316, Sep 1924. `doi:10.1007/BF01448013`.

[Tur95] D. A. Turner. Elementary strong functional programming. In Pieter H. Hartel and Rinus Plasmeijer, editors, *Funtional Programming Languages in Education*, pages 1–13, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.

[Wad89] Philip Wadler. Theorems for free! In Joseph E. Stoy, editor, *Proceedings of the fourth international conference on Functional programming languages and computer architecture, FPCA 1989, London, UK, September 11-13, 1989*, pages 347–359. ACM, 1989. `doi:10.1145/99370.99404`.

[Wil01] Alex Wilkie. On exponentiation - a solution to tarski's high school algebra problem. *Quaderni di Matematica*, 6, 02 2001.

[WKS22] Philip Wadler, Wen Kokke, and Jeremy G. Siek. *Programming Language Foundations in Agda*. August 2022. URL: `https://plfa.inf.ed.ac.uk/22.08/`.